

AI & Machine Learning Bootcamp

20 Sessions × 2 Hours · Foundations → GenAI

Interactive Roadmap

ML Core

S1 – S6

Foundations, regression, classification, trees & SVM

Deep Learning

S7

Neural networks, CNNs, RNNs, optimization

NLP

S8 – S13

NLP: fundamentals → tokenization → analysis → LM → embeddings → Seq2Seq (S8–S13)

GenAI

S14 – S16

BERT, GPT, LLMs, RAG & MLOps

BOOTCAMP TIMELINE

ML Core

S1 – S6

Deep Learning

S7

NLP

S8 – S13

GenAI

S14 – S16

ETRA

PRE-WEEK 1 · CHAPTER 1 · Foundations

1

Introduction to Artificial Intelligence

What Is AI — a simple, comprehensive definition

How Does a Computer Think? — traditional vs. AI programming

Data — the real fuel of AI

Three Types of AI — Narrow, General, and Super

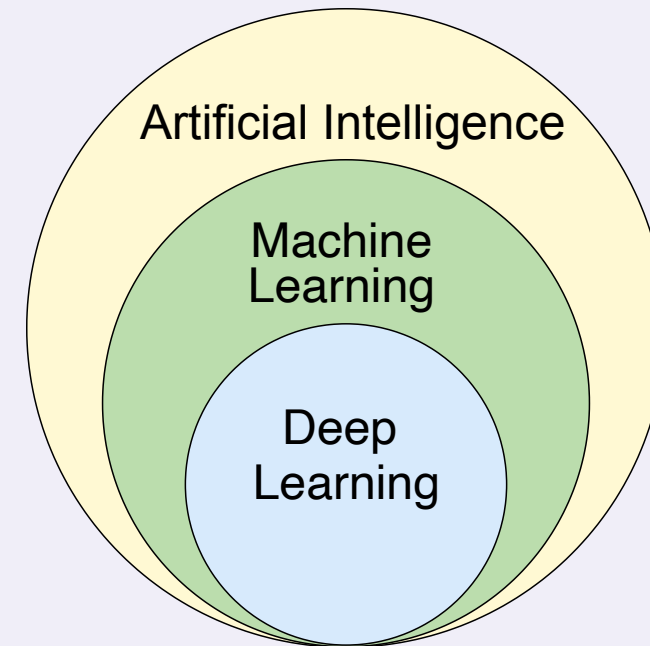
► 6 slides · ~15 min

What Is Artificial Intelligence?

A Simple, Comprehensive Definition

ETRA

Artificial Intelligence (AI) refers to computer systems designed to perform tasks that typically require human intelligence — such as learning from experience, recognizing patterns, reasoning about information, understanding language, and making decisions under uncertainty.



Learning: improve performance from data and feedback (e.g., recommendation engines).

Reasoning: draw conclusions from incomplete or noisy information.

Perception: interpret images, speech, and sensor signals (e.g., face unlock).

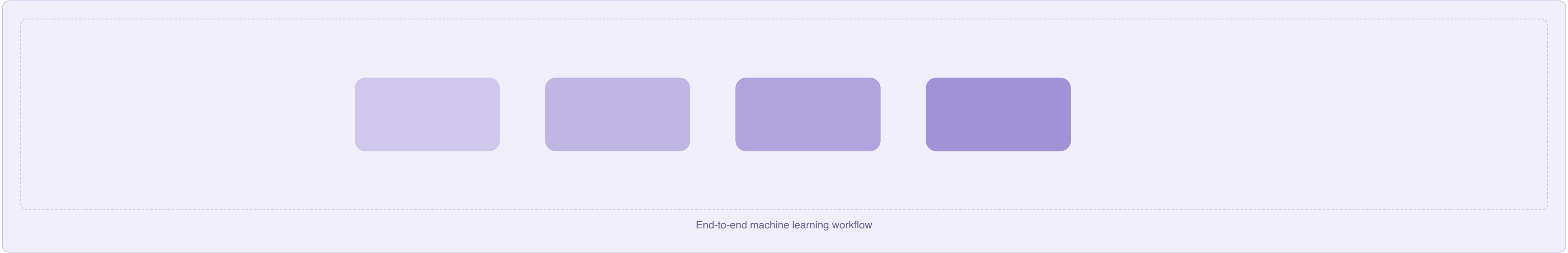
Language: understand and generate text or speech (e.g., voice assistants, chatbots).

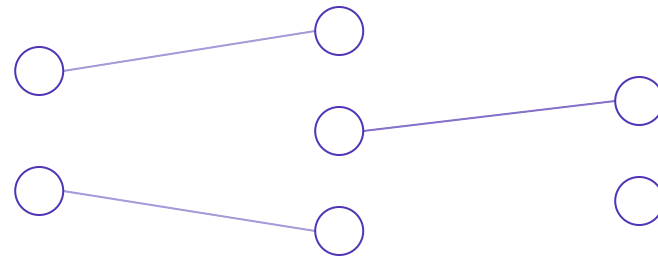
Everyday examples: spam filters, navigation apps, fraud detection, product suggestions.

Key phrase: tasks that typically require human intelligence.

How Does a Computer "Think"?

Traditional Programming vs. Artificial Intelligence





Traditional Programming

- Developer writes explicit rules (if/else, formulas).
- Logic is fixed before the program runs.
- Output is deterministic for the same input.
- Best when rules are known and stable.

AI / Machine Learning

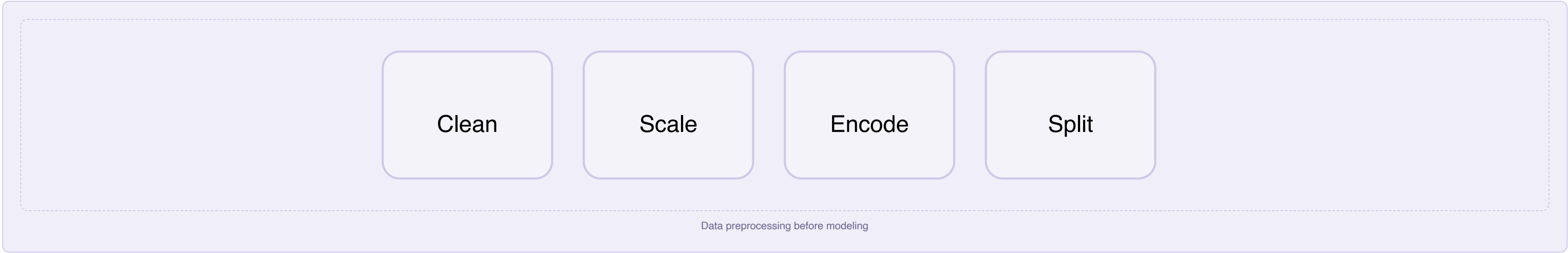
- | System learns patterns from examples (data).
- | Rules emerge from training — not hand-coded.
- | Output is probabilistic (confidence scores).
- | Best when rules are too complex to write manually.

The Core Difference

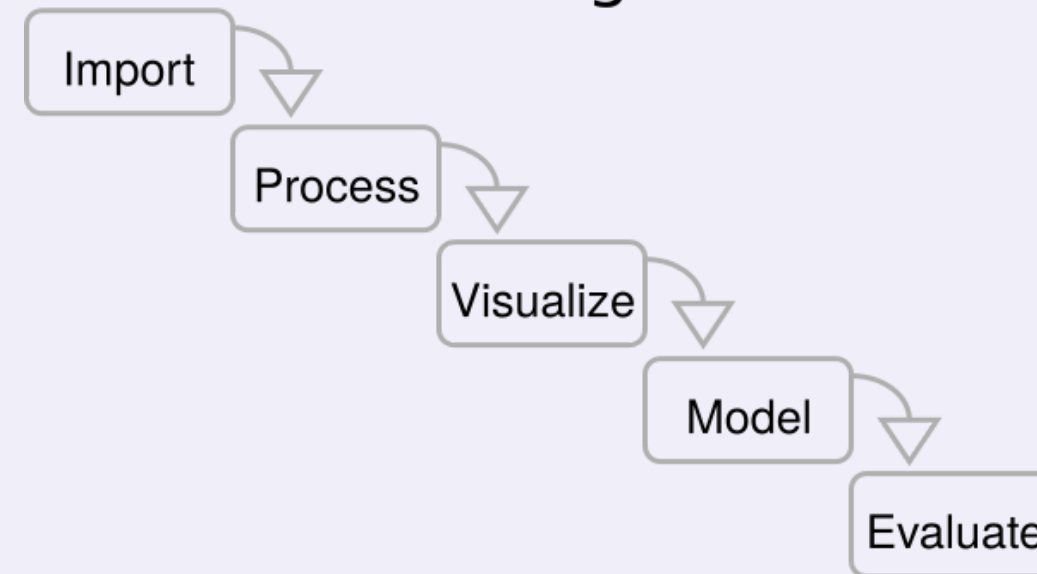
- | Traditional: Input + Rules → Output.
- | ML: Input + Output (labels) → Learned Rules (model).
- | You teach the machine by showing examples.
- | This bootcamp teaches you the ML path.

Why Data Quality Drives Every Model

Data is not just an input — it is the engine of artificial intelligence. A model can only learn what its training data represents. Poor, biased, or mislabeled data produces unreliable systems regardless of algorithm choice.



Machine learning workflow



Quality beats quantity: clean, representative data outperforms massive noisy datasets.

Labeled data: each example has a known answer (supervised learning — classification, regression).

Unlabeled data: patterns discovered without explicit answers (clustering, representation learning).

Features: measurable properties extracted from raw data (columns, pixels, words).

Garbage in, garbage out (GIGO): bad preprocessing → bad predictions.

Bridge to Section 1: we start with data cleaning, scaling, and train/test splits.

Link forward: Section 1 covers the preprocessing pipeline in depth.

Narrow AI (also called Weak AI or ANI) is designed and trained for one specific task or domain. It can match or exceed human performance in that narrow scope but cannot transfer its skills to unrelated problems.



What It Is

- | Specialized systems optimized for a single purpose.
- | No general reasoning or common-sense understanding.

Real-World Examples

| Chess engines (Deep Blue, Stockfish) — beat grandmasters at chess only.

| Face recognition on phones — identifies faces, not general vision.

| Spam filters — classify email, cannot write poetry.

| ChatGPT — excels at language tasks, not physical robotics.

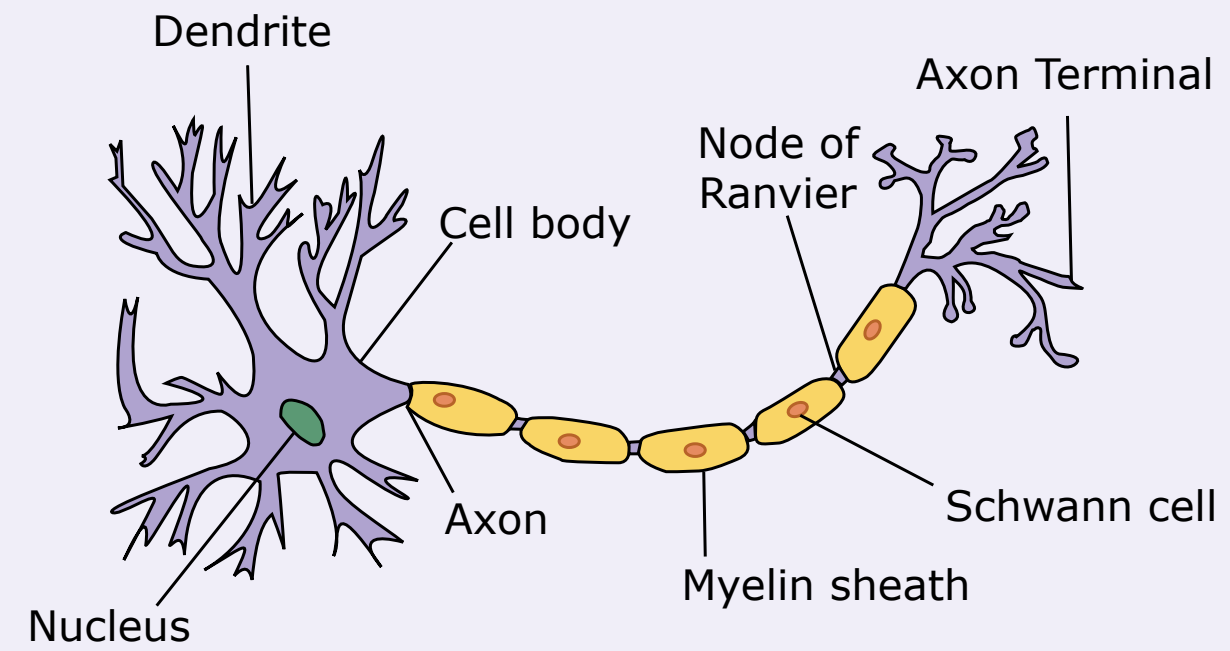
Limitations & Status

| Fails outside its training domain (no true understanding).

| Status: everywhere today — this is what industry builds and deploys.

Emphasize: ANI = practical, deployable, industry-ready.

General AI (AGI) would match human-level cognitive ability across a wide range of tasks — learning, reasoning, and adapting to new domains without retraining from scratch, much like a person who can switch from medicine to music.



What It Would Be

Human-level versatility across diverse tasks and domains.

Transfer learning: skills from one area apply to unrelated areas.

Common-sense reasoning and abstract problem solving.

Hypothetical Examples

A single agent that diagnoses patients, writes code, and drives a car.

Learns a new language from a few examples, like a child.

Current Status

Not achieved yet — active research at OpenAI, DeepMind, Anthropic, and academia.

Large language models show sparks of generality but lack robust reasoning.

Timeline estimates vary widely (years to decades).

Status: research goal — not available as a product today.

Super AI (ASI) refers to hypothetical systems that surpass human intelligence in virtually every field — scientific creativity, social skills, wisdom, and problem-solving — and could improve themselves at an accelerating rate.

What It Would Mean

- Outperforms the best humans in all cognitive tasks.
- Self-improvement loop: each upgrade makes the next upgrade faster.

Unpredictable impact on science, economy, and society.

Why It Matters (Even If Theoretical)

AI safety and alignment: ensuring AI goals match human values.

Ethics frameworks guide today's Narrow AI deployment.

Responsible AI: bias, privacy, transparency, and accountability.

Bootcamp Focus

ASI is science fiction today — but ethics applies to Narrow AI now.

This bootcamp builds practical Narrow AI: ML, DL, NLP, and GenAI.

Three types of AI — at a glance

TYPE	ABBREV.	SCOPE	STATUS
Narrow AI	ANI	One specific task	Exists today — industry standard
General AI	AGI	Human-level, all domains	Research goal — not yet achieved
Super AI	ASI	Beyond human, all domains	Theoretical — ethics still relevant

Bridge to Chapter 2 — History of AI.

PRE-WEEK 1 · CHAPTER 2 · Foundations

2

History of Artificial Intelligence

Philosophical & mathematical roots — before computers Birth of the field (1956) — Dartmouth & Alan Turing

First golden age & first AI winter (1970s) Expert systems boom & second AI winter (1980s)

Modern renaissance (1990s–2012) & deep learning era

▸ 8 slides · ~15 min

Before Electronic Computers

Long before the word 'Artificial Intelligence' existed, thinkers asked whether reasoning could be mechanized — laying the intellectual foundation for modern AI.

 Turing machine — foundational model of computation

17th–19th Century: Machines & Logic

Gottfried Leibniz (1646–1716): dreamed of a universal calculus of reasoning.

Charles Babbage & Ada Lovelace: Analytical Engine — first programmable machine concept.

George Boole (1847): Boolean algebra — logic as mathematics (AND, OR, NOT).

Early 20th Century: Formal Computation

Kurt Gödel (1931): limits of formal systems — some truths are unprovable.

Alonzo Church & Alan Turing (1936): what can be computed in principle?

| Turing Machine: abstract model of any computer — the blueprint for AI research.

Why It Matters

| AI is not about robots — it starts with logic, math, and the limits of computation.

| Every ML model today runs on Turing-equivalent hardware.

Pre-computer era — no electronic AI yet, only ideas and math.



Birth of the Field (1956)

Dartmouth Conference & Alan Turing's Legacy

ETRA

Artificial Intelligence was officially born as a research discipline at the Dartmouth Summer Research Project on Artificial Intelligence (1956), organized by John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude Shannon.



John McCarthy coined the term 'Artificial Intelligence' at Dartmouth (1956).

Proposal: 'every aspect of learning or intelligence can in principle be precisely described and simulated by a machine.'

Alan Turing (1912–1954): WWII code-breaker; asked 'Can machines think?' in his 1950 paper.

Turing Test: if a human cannot distinguish machine from human in conversation, the machine is intelligent.

Turing's work on computation and the Turing Test remain touchstones for AI ethics and evaluation.

1956 = official birth year of AI as a field.

High Expectations & Early Programs (1950s–1960s)

The first decade after Dartmouth was marked by enormous enthusiasm. Researchers believed human-level AI was only 20 years away — a prediction that would prove premature.

 IBM 7030 Stretch — mainframe era when early AI programs ran

| Logic Theorist (1956, Newell & Simon): first program to prove mathematical theorems — mimicked human problem-solving.

| General Problem Solver (1957): heuristic search for any well-defined problem.

| ELIZA (1966, Joseph Weizenbaum): chatbot simulating a Rogerian psychotherapist — fooled many users.

| Perceptron (1958, Rosenblatt): early neural network for pattern recognition — initial excitement.

| Funding flowed from DARPA, NSF, and governments — AI was the 'next big thing.'

| Hubris: Minsky (1967) — 'Within a generation ... the problem of creating AI will substantially be solved.'

Golden age = symbolic AI + early neural nets + massive optimism.

By the early 1970s, AI research had failed to deliver on its boldest promises. Governments and agencies cut funding sharply — the first 'AI winter' had arrived.

What Went Wrong

Combinatorial explosion: search spaces grew faster than computers could explore.

Brittle systems: symbolic programs failed on real-world noise and ambiguity.

Minsky & Papert (1969): Perceptron book showed single-layer nets cannot solve XOR — neural net funding dropped.

Key Events

Lighthill Report (1973, UK): criticized AI research; led to major UK funding cuts.

DARPA reduced grants for undirected 'general AI' research in the mid-1970s.

Speech recognition and machine translation projects underperformed expectations.

Lesson for Today

Hype cycles are real — capability must match promises.

Narrow, practical wins survive winters; grand claims do not.

First AI winter — causes vs. effects

CAUSE	EFFECT
Overpromised timelines	Loss of public and funder trust
Limited compute (1970s hardware)	Could not scale symbolic search
Perceptron limitations exposed	Neural network research stalled ~15 years
No big data yet	Statistical methods lacked fuel

First winter $\approx$ 1974–1980 — funding drought, not end of research.

Expert Systems

- IF-THEN rule engines encoding human domain expertise.
- MYCIN (1970s–80s): diagnosed blood infections — rivaled human experts.
- XCON (1980): configured DEC VAX computers — saved millions annually.
- Commercial boom: AI market reached ~\$1B by mid-1980s.

Neural Networks Return

- Backpropagation popularized (Rumelhart, Hinton, Williams, 1986).
- Multi-layer networks could learn non-linear patterns (including XOR).
- Parallel Distributed Processing (PDP) books reignited connectionist research.
- Still limited by data and compute — not yet mainstream.

Global Context

- Japan Fifth Generation Computer Project (1982): \$400M push for AI supercomputers.
- US and Europe responded with competing AI funding programs.
- AI returned to fashion — but fragility of expert systems would soon show.

● Second AI Winter (Late 1980s)

Market Collapse & Hardware Limits

The expert-system bubble burst in the late 1980s. Maintaining rule bases proved expensive, hardware could not keep pace with ambitions, and symbolic AI hit a ceiling — triggering a second funding and market pullback.

- | Expert systems were costly to update — rules drifted as domains changed.
- | Desktop PCs replaced specialized Lisp machines — AI hardware startups failed.
- | Japan Fifth Generation project fell short of goals by the early 1990s.

DARPA shifted funding toward specific applications, not general intelligence.

AI company valuations collapsed; 'AI' became a risky label in boardrooms.

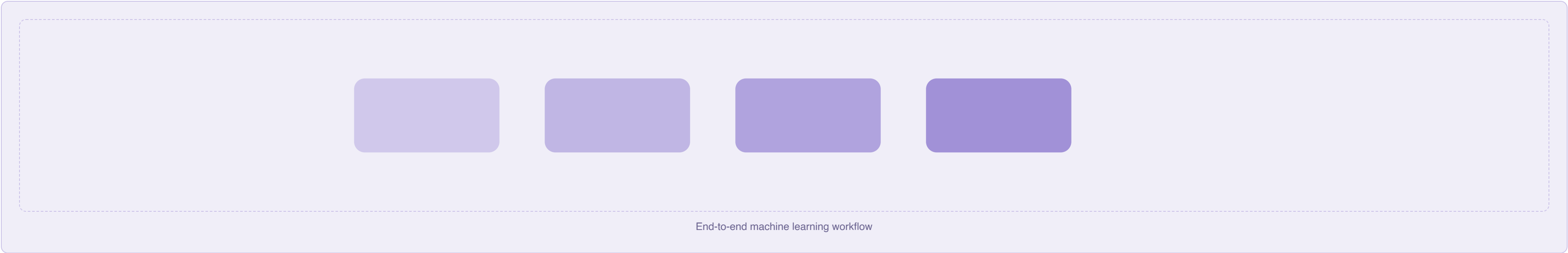
Research continued quietly in universities — seeds for the next wave were planted.

Two AI winters compared

	FIRST WINTER (1970S)	SECOND WINTER (LATE 1980S)
Trigger	Lighthill Report, perceptron limits	Expert system market crash
Main victim	General AI / neural nets	Commercial AI / Lisp machines
What survived	Academic AI, expert systems niche	Statistics, ML, backprop research
Lesson	Don't overpromise timelines	Don't overpromise commercial ROI

Late 1980s–early 1990s — AI rebranded as ML in many labs.

AI recovered not through grand symbolic promises but through statistics, data, and faster hardware — culminating in deep learning's breakthrough on ImageNet in 2012.





Landmark Victories

Deep Blue beats Garry Kasparov at chess (1997, IBM) — Narrow AI triumph.

IBM Watson wins Jeopardy! (2011) — NLP + search at scale.

Enablers of the Renaissance

| Internet era: massive datasets became available (web text, images, logs).

| GPUs: parallel matrix ops made neural net training 10–100× faster.

| Statistical ML: SVMs, random forests, boosting dominated Kaggle and industry.

The 2012 Turning Point

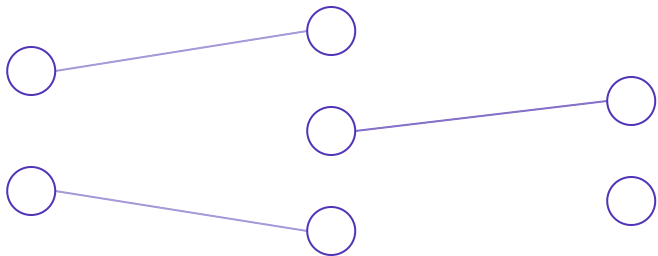
| ImageNet (2009, Fei-Fei Li): 14M labeled images — benchmark for vision.

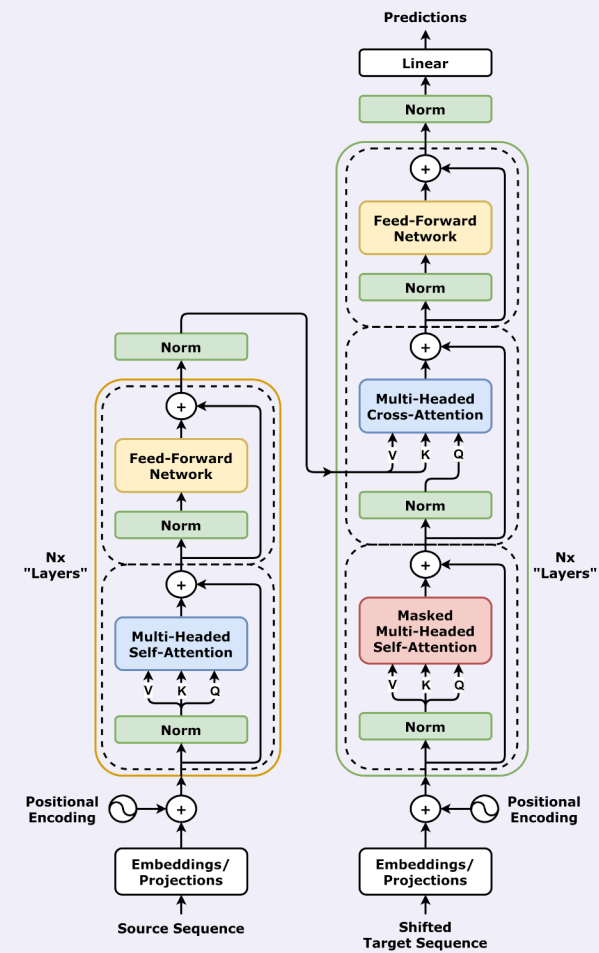
AlexNet (2012, Krizhevsky et al.): deep CNN crushed ImageNet — error rate halved.

Deep learning era officially began — industry pivoted en masse.

1997 Deep Blue → 2012 AlexNet = path from symbolic wins to deep learning.

Since 2012, AI has advanced at unprecedented speed — from real-time object detection to transformers and chatbots used by hundreds of millions. This is the era this bootcamp prepares you to build in.





Computer Vision & Scale

YOLO (2016): real-time object detection — AI in cameras, drones, autonomous vehicles.

ResNet, EfficientNet: ever-deeper and more efficient image classifiers.

AlphaGo (2016): deep RL beats world Go champion — beyond explicit search.

Language & Generative AI

Transformers (2017, 'Attention Is All You Need'): parallel sequence modeling.

BERT (2018), GPT-2/3 (2019–2020): pre-trained language models at scale.

ChatGPT (2022): 100M users in 2 months — GenAI enters mainstream.

| GPT-4, Claude, Gemini, multimodal models (2023–2025): text, image, code, voice.

Bootcamp Bridge

| You will learn the stack behind this era: ML → DL → NLP → GenAI → RAG → MLOps.

| History teaches patience: winters follow hype — solid skills outlast trends.

| Next: AI domains and branches — how the field splits into specializations.

Bridge to Chapter 3 — AI Domains.

AI Domains Branches

Machine Learning — patterns & algorithms

Deep Learning — neural network simulation

NLP — how machines understand language

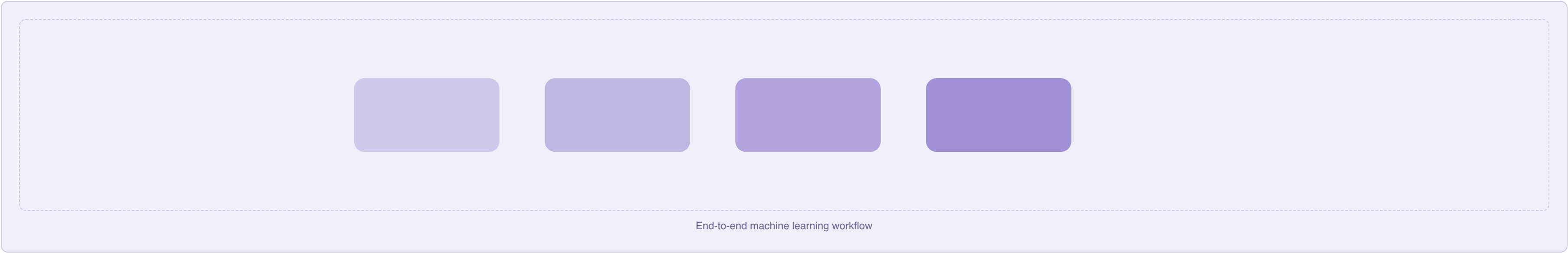
Computer Vision — images, video & security

Robotics & autonomous systems

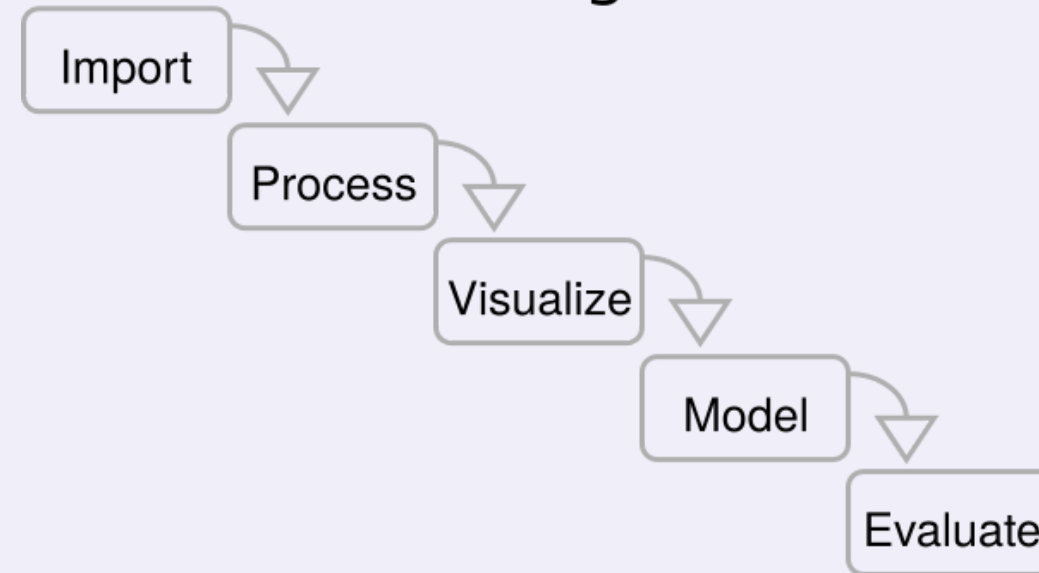
Expert systems & decision making

▸ 6 slides · ~15 min

Machine Learning (ML) enables computers to learn patterns from data without being explicitly programmed for every scenario. It is the foundation of modern AI and the starting point of this bootcamp.



Machine learning workflow



Core Idea

Learn a mapping from inputs (features) to outputs (labels or clusters).

Generalize from training examples to unseen data.

Evaluate with held-out test sets and proper metrics.

Learning Types

Supervised: labeled data — classification & regression.

Unsupervised: no labels — clustering, dimensionality reduction.

Reinforcement: reward signals — game playing, robotics control.

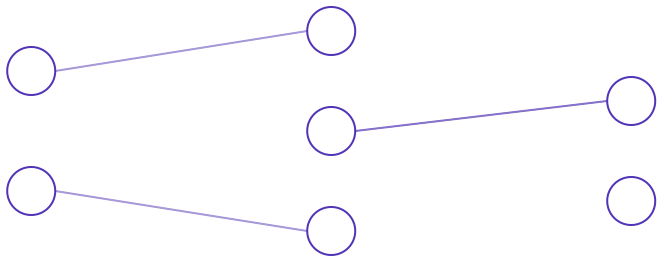
Algorithms & Bootcamp

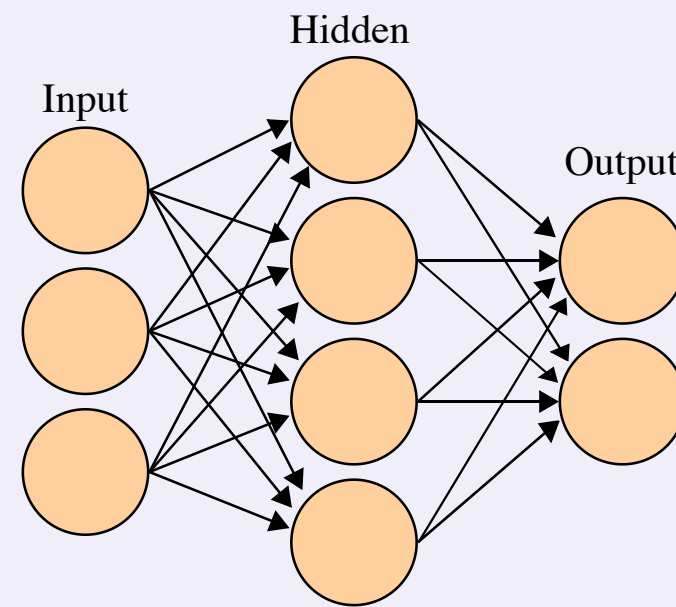
| Linear & logistic regression, decision trees, SVM, k-means, PCA.

| This bootcamp: Sections S1–S6 — preprocessing through classical ML.

ML = patterns and algorithms — bootcamp Weeks/Days 1 focus.

Deep Learning (DL) uses multi-layer artificial neural networks to learn hierarchical representations — from edges in images to words in sentences. It powers the breakthroughs of the 2012–today era.





How It Works

Artificial neurons: weighted inputs $\rightarrow$ activation $\rightarrow$ layered composition.

Backpropagation adjusts weights to minimize prediction error.

Deep = many layers extract features automatically (no hand-crafted rules).

Key Architectures

CNNs: spatial patterns in images and video.

RNNs / LSTMs: sequential data (text, time series).

Transformers: parallel attention — foundation of GPT and BERT.

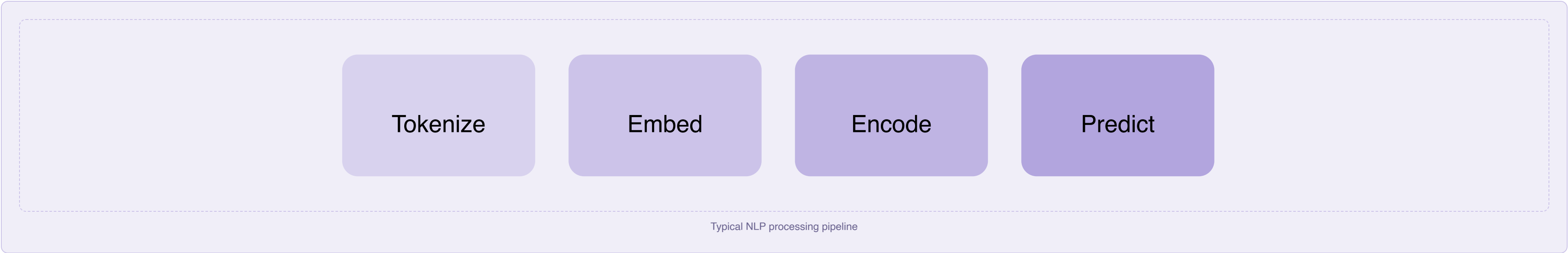
Requirements & Bootcamp

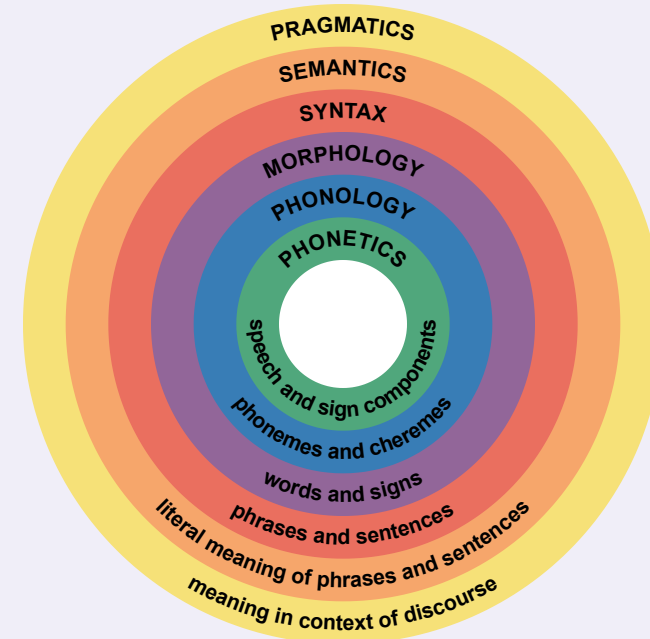
Needs large datasets, GPUs, and careful regularization.

This bootcamp: Section S7 — 7 labs from MLP to CNN and RNN.

DL $\subset$ ML — neural networks with many layers.

NLP teaches computers to read, interpret, and generate human language — from tokenizing raw text to building chatbots and machine translation systems.





Core Tasks

| Text cleaning & tokenization: raw text → structured units.

| Understanding: POS tagging, NER, sentiment analysis.

Generation: summarization, translation, conversational AI.

Evolution

Classical: bag-of-words, n-grams, TF-IDF.

Neural: word embeddings (Word2Vec), RNNs, attention.

Modern: pre-trained Transformers (BERT, GPT) — transfer learning for NLP.

Applications & Bootcamp

| Chatbots, search, spam detection, document classification, code assistants.

| This bootcamp: Sections S8–S13 — cleaning through Seq2Seq and attention.

NLP — language understanding and generation.

Computer Vision (CV) enables machines to interpret visual data — photographs, video streams, medical scans, and surveillance feeds — extracting meaning from pixels.

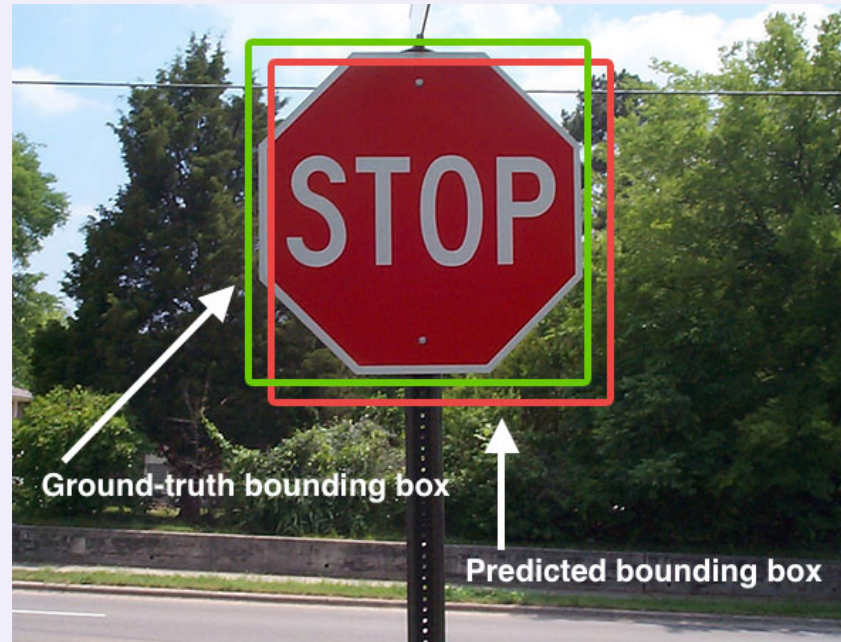


Image classification: what object is in this image? (ResNet, EfficientNet).

Object detection: where are objects and what are they? (YOLO, Faster R-CNN).

Segmentation: pixel-level boundaries for medical imaging and autonomous driving.

Face recognition & biometrics: security systems, phone unlock, access control.

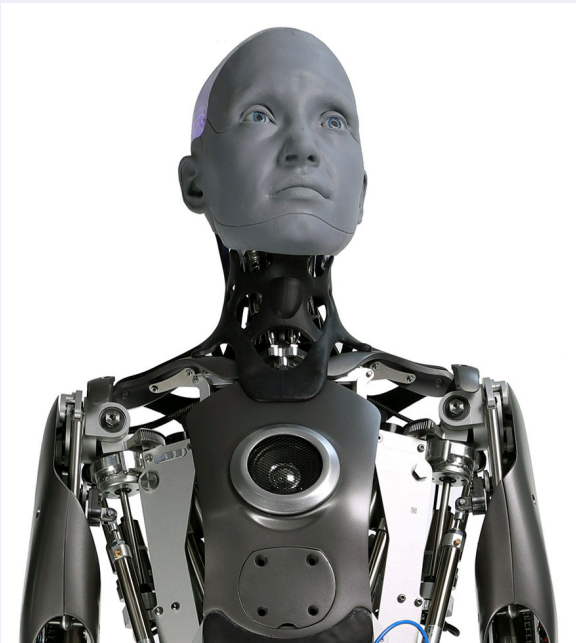
Video analytics: motion detection, crowd monitoring, anomaly alerts in CCTV.

Powered by CNNs and increasingly Vision Transformers (ViT).

Bootcamp touchpoint: CNN labs and architectures in Section S7.

CV — images, video, and visual security analytics.

Robotics combines AI with mechanical systems — sensors, actuators, and real-time control — to create machines that perceive, decide, and act autonomously in dynamic environments.



Core Components

Perception: cameras, LiDAR, radar → CV + sensor fusion.

Planning: path finding, obstacle avoidance, task scheduling.

Control: motor commands executed in milliseconds.

Applications

Self-driving cars: detect lanes, pedestrians, signs — decide in real time.

Drones: aerial mapping, delivery, inspection.

Warehouse robots (Amazon, logistics): pick, pack, navigate shelves.

Surgical & industrial robots: precision tasks alongside humans.

AI Domains Combined

Robotics = CV + ML/DL + reinforcement learning + embedded systems.

| Bootcamp: cross-cutting skills from S1–S7 prepare you for robotics foundations.

Robotics — perception, planning, and autonomous action.

Expert Systems encode human domain knowledge as explicit IF-THEN rules — one of the first commercial AI successes. Understanding them clarifies why statistical ML and deep learning eventually superseded pure rule-based approaches.

MYCIN expert system — early rule-based medical AI

How Expert Systems Work

- Knowledge base: facts and rules written by domain experts.

| Inference engine: applies rules to reach conclusions.

| Examples: MYCIN (medical diagnosis), XCON (computer configuration).

Strengths & Limits

| Strengths: interpretable, auditable, no training data required.

| Limits: brittle, expensive to maintain, cannot learn from data.

1980s boom → late-1980s collapse (see AI history slides).

Today & Bootcamp

Modern systems blend rules + ML (hybrid AI in finance, healthcare).

Decision trees in S4 bridge symbolic and statistical approaches.

Next: Generative AI and modern applications — the current frontier.

Expert systems vs. machine learning

ASPECT	EXPERT SYSTEMS	MACHINE LEARNING
Knowledge source	Human-written rules	Learned from data
Maintenance	Manual rule updates	Retrain on new data
Interpretability	High (explicit rules)	Varies (trees high, DL low)
Best for	Stable, well-defined domains	Noisy, high-dimensional data

Bridge to Chapter 4 — GenAI & Modern Applications.

PRE-WEEK 1 · CHAPTER 4 · GenAI

4

Generative AI Modern Applications

The Generative AI revolution

Large Language Models — conversation & context

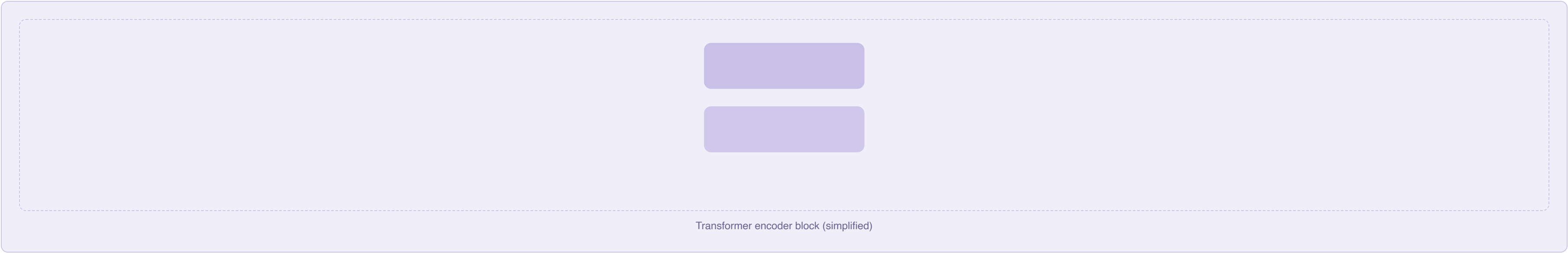
Industry applications — business, medicine & public security

Daily productivity — text, image & code generation

► 4 slides · ~15 min

Why Creation Changes Everything

Traditional AI answers 'What is this?' or 'What will happen?' Generative AI answers 'Create something new that matches this intent' — drafts, designs, code, and synthetic data on demand.





Discriminative vs. Generative

Discriminative: classify spam vs. ham, predict house prices.

Generative: write emails, compose images, synthesize speech.

GenAI learns data distributions — then samples new outputs.

Key Breakthroughs

Transformers (2017): scalable sequence modeling at web scale.

Diffusion models: high-quality image generation (DALL·E, Stable Diffusion, Midjourney).

ChatGPT (2022): conversational interface made GenAI accessible to everyone.

Why It Matters Now

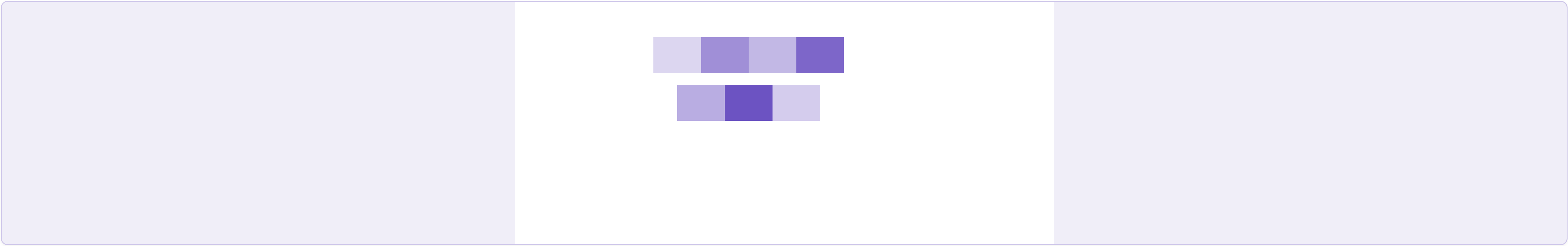
Lower barrier: natural language replaces complex ML pipelines for many tasks.

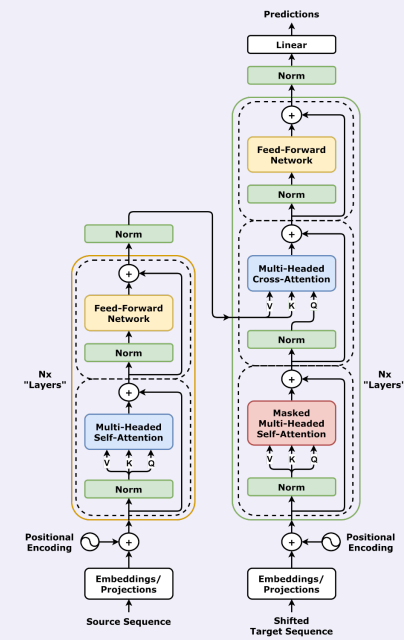
Risks: hallucinations, bias, copyright, deepfakes — need guardrails.

Bootcamp: Section S14 covers BERT, GPT, attention, and LLM fundamentals.

GenAI revolution — prediction → creation paradigm shift.

Large Language Models are neural networks trained on vast text corpora to predict the next token — enabling fluent conversation, reasoning over context, and task completion from natural-language prompts.





How LLMs Work

Pre-training: learn language patterns from billions of web pages, books, code.

Self-attention: weigh every word against every other — capture long-range context.

Fine-tuning & RLHF: align outputs with human preferences (helpful, harmless, honest).

Context & Conversation

Context window: how much prior text the model 'remembers' in one session.

Multi-turn dialogue: model tracks conversation history for coherent replies.

Prompt engineering: instructions, examples, and role-setting steer behavior.

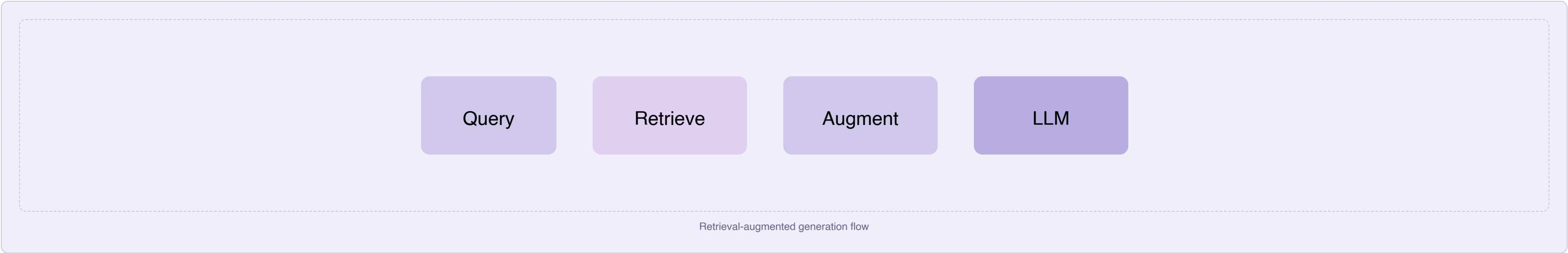
Examples & Bootcamp

| GPT-4, Claude, Gemini, Llama — general-purpose assistants and APIs.

| Bootcamp path: S8–S13 (NLP foundations) → S14 (BERT, GPT, attention) → S15 (RAG).

LLMs — conversation, context windows, and prompt steering.

Generative AI is moving from demos to production across high-impact sectors — with distinct opportunities and governance requirements in each.



Business & Enterprise

| Customer support: AI agents resolve tickets, draft responses, summarize calls.

| Marketing: personalized content, A/B copy variants, market research synthesis.

| Finance: report generation, fraud narrative analysis, compliance document review.

| RAG over company docs: grounded answers from internal knowledge bases (S15).

Medicine & Healthcare

Clinical documentation: draft notes from doctor-patient conversations.

Literature review: summarize research papers and trial results.

Medical imaging assist: report generation from radiology scans (human-in-the-loop).

Critical: HIPAA/privacy, validation, never replace clinician judgment.

Public Security & Safety

| Intelligence analysis: summarize large document sets, translate multilingual sources.

| Threat detection: pattern recognition in communications and surveillance metadata.

| Deepfake detection: counter AI-generated misinformation and synthetic media.

| Ethics: bias, civil liberties, audit trails — MLOps monitoring essential (S16).

Industry applications — opportunities and governance in each sector.

GenAI is already embedded in everyday workflows — not just research labs. Knowing these tools helps trainees use AI responsibly while building their own systems in the bootcamp.



Text Generation

| ChatGPT, Claude, Gemini: drafting, summarizing, translating, brainstorming.

| Copilot in Word/Outlook: inline writing assistance in familiar apps.

| Notion AI, Grammarly: notes, editing, and tone adjustment.

Image Generation

| DALL·E, Midjourney, Stable Diffusion: marketing visuals, prototypes, storyboards.

Canva AI, Adobe Firefly: design workflows for non-designers.

Use with care: copyright, consent, and disclosure of AI-generated content.

Code Generation

GitHub Copilot, Cursor, Codeium: autocomplete, refactor, explain code.

ChatGPT/Claude for debugging, unit tests, and documentation.

Next: ethics, the future of work, and your learning roadmap.

Bridge to Chapter 5 — Ethics & the Future.

PRE-WEEK 1 · CHAPTER 5 · GenAI

5

Ethics the Future

AI ethics — bias, privacy & deepfakes Future of jobs — human–machine integration

Your learning roadmap — start & grow in AI/ML

▸ 3 slides · ~15 min

As AI systems influence hiring, healthcare, law enforcement, and public discourse, ethical design is not optional — it is a core engineering requirement.

TP	FP
FN	TN

Confusion matrix layout



Bias & Fairness

| Training data reflects historical bias → models can discriminate by race, gender, age.

| Examples: biased hiring filters, unequal loan approvals, skewed facial recognition.

Mitigation: diverse datasets, fairness metrics, human review, audit trails.

Privacy

Models memorize sensitive training data — risk of leakage in outputs.

Regulations: GDPR (EU), HIPAA (health), sector-specific data governance.

Best practices: anonymization, consent, data minimization, secure pipelines (S1, S16).

Deepfakes & Misinformation

| Synthetic video/audio/text indistinguishable from real — fraud, political manipulation.

| Detection tools emerging; watermarking and provenance standards in progress.

| Always disclose AI-generated content; verify before sharing.

AI ethics — bias, privacy, and synthetic media risks.

AI will transform work — not by replacing every job overnight, but by reshaping tasks, roles, and the skills that matter most in a human–machine partnership.





What Changes

| Automation of repetitive tasks: data entry, basic reporting, first-draft content.

| Augmentation: AI assists professionals — doctors, lawyers, engineers, analysts.

New roles: ML engineer, prompt engineer, AI ethicist, MLOps specialist, data curator.

Human + Machine Model

Humans: judgment, empathy, creativity, accountability, domain expertise.

Machines: speed, scale, pattern detection, 24/7 availability.

Best outcomes: human-in-the-loop — AI proposes, human approves.

Skills That Grow in Value

| Critical thinking, communication, and interdisciplinary problem-solving.

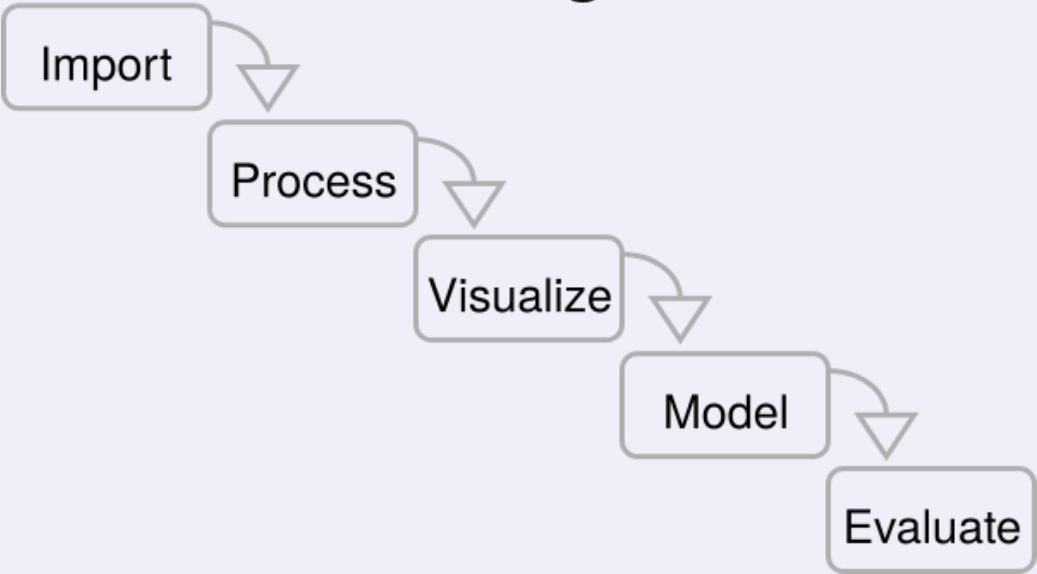
| Technical: Python, ML pipelines, data literacy — exactly what this bootcamp builds.

| Adaptability: AI tools evolve fast — learn how to learn.

Future of work — augmentation over replacement.

This bootcamp is your launchpad — not your finish line. A structured path after these five days keeps momentum and builds a portfolio that employers recognize.

Machine learning workflow



Phase 1 — During Bootcamp (Now)

| Follow the daily arc: S1–S6 (ML) → S7 (DL) → S8–S13 (NLP) → S14–S16 (GenAI, RAG, MLOps).

| Run every lab notebook — modify hyperparameters, break things, fix them.

| Take notes on concepts you cannot explain — revisit after each section.

Phase 2 — First 3 Months After

| Build 2–3 portfolio projects: classification, NLP, or a small RAG app.

Practice on Kaggle competitions and open datasets.

Read papers: start with distill.pub and Hugging Face blog posts.

Contribute to open source: scikit-learn docs, Hugging Face models.

Phase 3 — Long-Term Growth

Specialize: vision, NLP, RL, or MLOps based on interest and job market.

Stay current: follow arXiv summaries, AI newsletters, and vendor docs.

Network: local meetups, GitHub, LinkedIn — share what you build.

Bootcamp → career milestones

MILESTONE	TARGET
Week 1 (bootcamp)	Complete S1–S16 + core labs
Month 1	1 end-to-end ML project on GitHub
Month 3	NLP or DL project + Kaggle entry
Month 6–12	Specialization + internship or junior role

Final intro slide — bridge to Section 1 (S1). Total intro ~70–75 min.

WEEK 1 · SESSION 1 · Foundations

1

Foundations

Data Pre-Processing

Train/test splits

Feature scaling

Encoding & imputation

Data leakage

▸ 20 slides · ~32 min

Where Are We in the Bootcamp?



Build the end-to-end ML workflow: clean data → train → evaluate.

- **S1 Foundations & Preprocessing**
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

The 3 Main Steps

Step 1: Data Pre-Processing

- Import the data
- Clean the data
- Split into training and test sets
- Feature scaling

Step 2: Modelling

- Build the model
- Train the model
- Make predictions

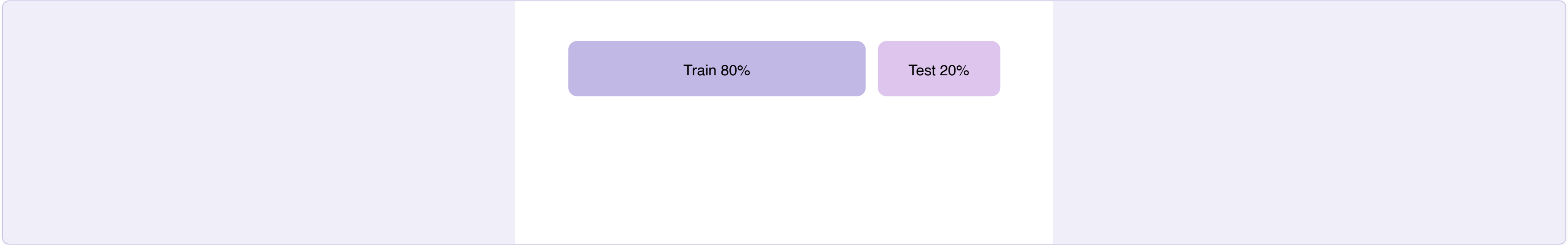
Step 3: Evaluation

- Calculate performance metrics
- Make a final prediction

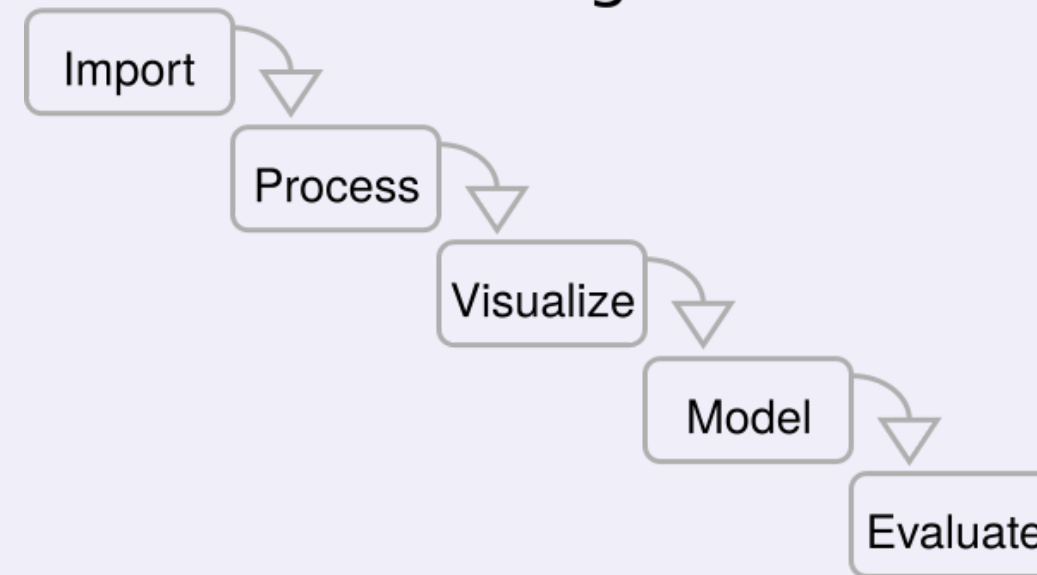
Training Set & Test Set

ETRA

Key Idea



Machine learning workflow



Training Set: Data used by the model to learn patterns.

Test Set: New, unseen data used to check model performance.

Never tune hyperparameters on the test set — it becomes part of training indirectly.

Stratified splits preserve class balance in classification tasks.

We train on one part of the data, then test on another part to make sure the model generalizes well.

What is Feature Scaling?

Feature scaling means adjusting feature values to a similar range using methods like standardization and normalization.



- It prevents large-value features from dominating smaller-value features.
- It improves performance for distance-based and gradient-based algorithms (e.g., KNN, SVM, Gradient Descent).
- It helps training converge faster.

Example (Before vs After Scaling)

STUDENT	HEIGHT (CM) - BEFORE	WEIGHT (KG) - BEFORE	HEIGHT (SCALED)	WEIGHT (SCALED)
A	150	50	0.25	0.22
B	170	70	0.58	0.67
C	185	85	0.83	1.00

Before scaling, height and weight have different numeric ranges. After scaling, both features are on a comparable scale and contribute more fairly.



- Normalization scales to [0, 1] using min–max — sensitive to outliers.
- Standardization uses mean 0 and std 1 — preferred when features are roughly Gaussian.

Robust scaling uses median and IQR when outliers dominate.

1) Normalization (Min-Max Scaling)

Normalization transforms each value to a range between 0 and 1.

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Useful for models that depend on distances or absolute magnitudes.

Common choice for algorithms such as KNN and K-Means.

2) Standardization (Z-score Scaling)

Standardization centers data around 0 with a standard deviation of 1.

$$z = \frac{x - \mu}{\sigma}$$

mu = mean of the feature, sigma = standard deviation of the feature.

Preferred when features are close to a Gaussian distribution.

Works well with Logistic Regression, SVM, and Linear Regression.



Choosing the Right Scaling Method

Use standardization for SVM, logistic regression, PCA, and gradient-based training.

Tree-based models are scale-invariant — scaling is optional.

When in doubt, fit the scaler on train only, then transform train and test.

Normalization

Use Normalization when feature values have very different ranges and you want all values mapped to [0, 1].

Normalization

Typical method: Min-Max Scaling.

Good for distance-based models where magnitude differences strongly affect results.

Standardization

Use Standardization when features are expected to follow (or be close to) a Gaussian distribution, or when the dataset contains outliers.

It keeps the data centered around 0 and scales variability to 1.

Often preferred for linear models and optimization-based algorithms.

Feature Scaling Example (Normalization)

Given feature values: [20, 40, 60, 80, 100]



$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

$x_{\min} = 20$

$x_{\max} = 100$

Before vs After Normalization

ORIGINAL VALUE (X)	NORMALIZED VALUE (X_{NORM})
20	0.00
40	0.25
60	0.50
80	0.75
100	1.00

This table shows how all values are scaled to the range [0, 1].

Feature Scaling Example (Standardization)

Given feature values: [20, 40, 60, 80, 100]



$$z = \frac{x - \mu}{\sigma}$$

Mean: $\mu = 60$

Standard deviation: $\sigma = 28.28$

Before vs After Standardization

ORIGINAL VALUE (X)	STANDARDIZED VALUE (Z)
20	-1.41
40	-0.71
60	0.00
80	0.71
100	1.41

This table shows that standardized values are centered around 0 and measured in units of standard deviation.

What is Categorical Data?

Categorical data describes values that belong to named groups rather than continuous numbers.

Category Types

TYPE	DESCRIPTION	EXAMPLE
Ordinal	Categories have a meaningful order.	Education Level: High School < Bachelor's < Master's < PhD
Nominal	Categories have no natural order.	Payment Method: Cash, Card, Bank Transfer

Example 1: Label Encoding

RAW CATEGORY	ENCODED VALUE
Cash	0
Card	1
Bank Transfer	2

Example 2: One-Hot Encoding

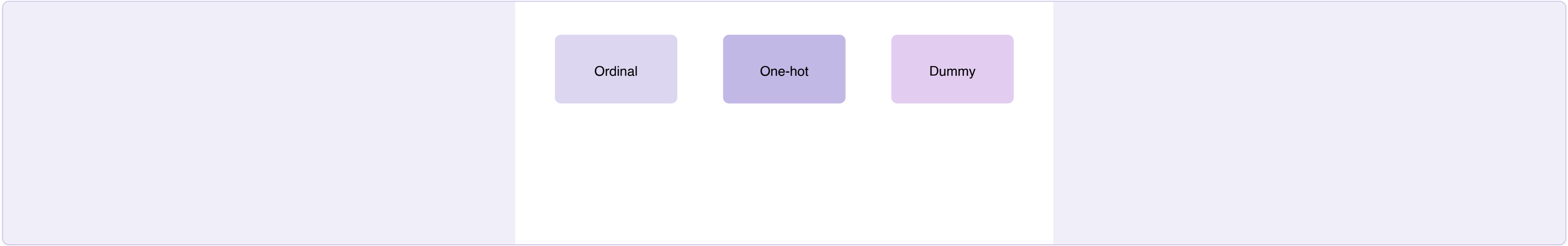
RAW CATEGORY	CASH	CARD	BANK TRANSFER
Cash	1	0	0
Card	0	1	0
Bank Transfer	0	0	1

Most machine learning algorithms work with numerical inputs only, so categorical values must be converted into numbers.

How to Deal with Categorical Data (Ordinal Encoding)

What is Ordinal Encoding?

Ordinal Encoding converts ordered categories into integers that preserve their ranking.



Use Ordinal Encoding only when categories have a true order.

Step 1: Identify the correct category order.

Step 2: Assign an integer to each level based on that order.

Step 3: Replace the original text values with encoded numbers.

Example

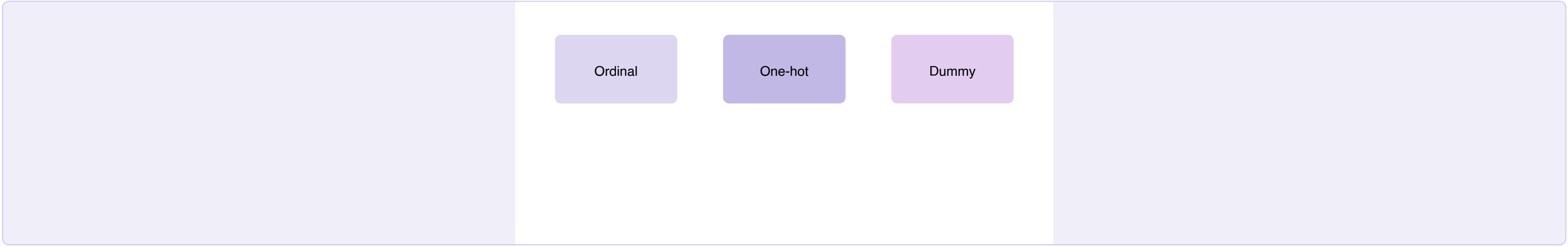
SATISFACTION LEVEL	ENCODED VALUE
Low	1
Medium	2
High	3

This method keeps the order information, which is important for many machine learning models.

How to Deal with Categorical Data (One-Hot Encoding)

What is One-Hot Encoding?

One-Hot Encoding converts each category into a separate binary column (0 or 1).



Use One-Hot Encoding for nominal categories (no natural order).

Step 1: List all unique categories.

Step 2: Create one column for each category.

Step 3: Put 1 in the matching category column and 0 in all others.

Example

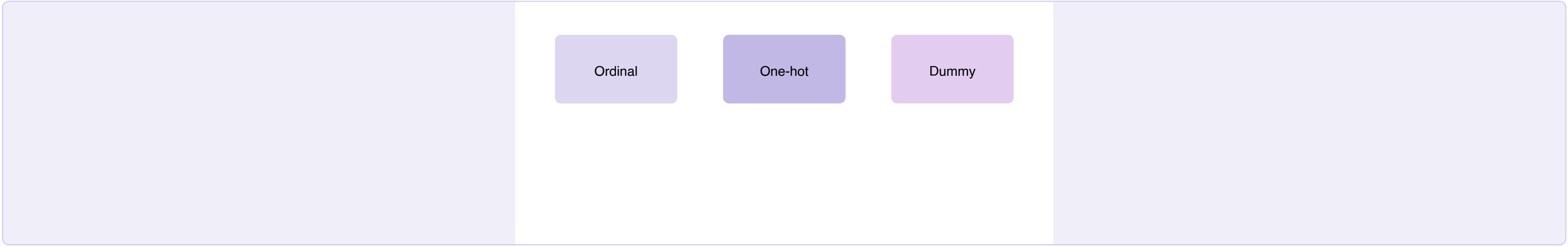
PAYMENT METHOD	CASH	CARD	BANK TRANSFER
Cash	1	0	0
Card	0	1	0
Bank Transfer	0	0	1

This method avoids creating a false ranking between categories.

How to Deal with Categorical Data (Dummy Encoding)

What is Dummy Encoding?

Dummy Encoding is similar to One-Hot Encoding, but it removes one category column to avoid redundancy.



It helps prevent the dummy variable trap (perfect multicollinearity), especially in linear models.

Step 1: Start with One-Hot encoded columns.

Step 2: Drop one category as a reference (baseline).

Step 3: Keep the remaining binary columns.

Original categories: Payment Method = [Cash, Card, Bank Transfer].

Reference category (dropped): Cash.

Example

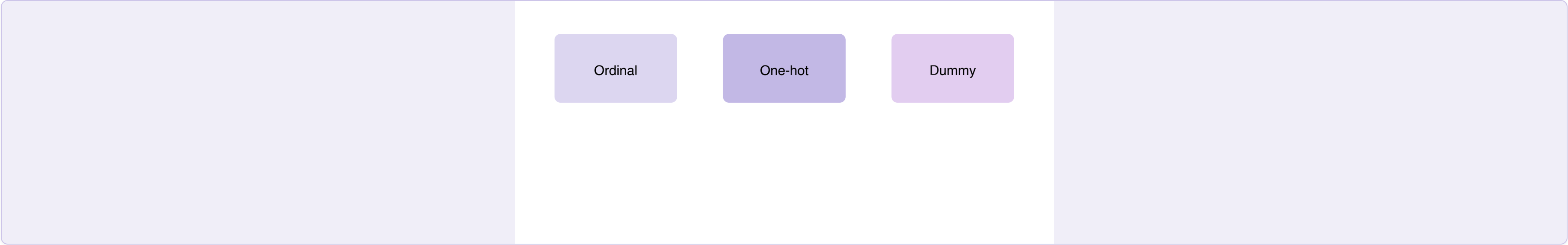
PAYMENT METHOD	CARD	BANK TRANSFER
Cash	0	0
Card	1	0
Bank Transfer	0	1

In this setup, Cash is represented when all remaining columns are 0.

●

How to Deal with Categorical Data (Other Encoding Types)

Besides Ordinal, One-Hot, and Dummy Encoding, there are other useful methods.



Encoding Methods Overview

Encoding Type	Idea	Best Use Case
Frequency Encoding	Replace each category with how often it appears.	Large datasets with many categories
Count Encoding	Similar to frequency, but uses raw counts directly.	Tree-based models with high-cardinality features
Target Encoding	Replace category with target mean for that category.	Supervised tasks (use with care to avoid leakage)
Binary Encoding	Convert category index to binary digits across columns.	High-cardinality data with fewer columns than one-hot
Hash Encoding	Use a hash function to map categories into fixed columns.	Very large and dynamic category sets

1) Frequency Encoding (City)

City	Frequency
Riyadh	0.50
Jeddah	0.33
Dammam	0.17

2) Count Encoding (City)

CITY	COUNT
Riyadh	3
Jeddah	2
Dammam	1

3) Target Encoding (City)

CITY	MEAN TARGET
Riyadh	0.80
Jeddah	0.40
Dammam	0.20

4) Binary Encoding (City)

CITY	CATEGORY ID	BINARY CODE
Riyadh	1	01
Jeddah	2	10
Dammam	3	11

5) Hash Encoding (4 Buckets)

CITY	HASH BUCKET
Riyadh	2
Jeddah	0
Dammam	3

Choose the encoding method based on category order, cardinality, model type, and risk of data leakage.

What is Missing Data?

Missing data refers to values that are absent or incomplete in one or more features of a dataset.

- It can introduce bias into model training.
- It may reduce prediction accuracy.
- It can cause errors in analysis if not handled properly.

Types of Missing Data

TYPE	MEANING
MCAR (Missing Completely at Random)	Missingness happens randomly and is unrelated to any variable in the dataset.
MAR (Missing at Random)	Missingness depends on other observed variables, but not on the missing value itself.
NMAR (Not Missing at Random)	Missingness depends on the missing value itself or unobserved factors.

Handling Missing Data - Why Do We Have Missing Data?

Missing values can appear for many practical reasons during data collection and processing.

Common Causes

Cause	Description	Example
Human Error	Data is skipped, entered incorrectly, or forgotten.	A user leaves the Age field empty in a form.
Device/Sensor Failure	Measurement tools fail or disconnect.	A medical sensor stops recording heart rate temporarily.
Data Integration Issues	Missing fields appear when combining data from multiple sources.	One database has salary, another does not.
Privacy or Refusal	Participants choose not to share sensitive information.	A customer does not provide income details.
System/Transmission Errors	Data is lost during storage, transfer, or export.	Network interruption causes missing rows in logs.
Conditional Missingness	Some fields are only relevant for specific groups.	Pregnancy_Status is empty for male patients.

Understanding why data is missing helps us choose the right treatment method (deletion, imputation, or advanced modeling).



Handling Missing Data - MCAR

Missing Completely at Random (MCAR)

MCAR means missing values occur randomly and are independent of both observed and unobserved variables.

- MCAR has no systematic pattern.
- In practice, MCAR is often an unrealistic assumption in real-world datasets.
- Before assuming MCAR, review data collection workflow and consult domain experts.

If MCAR is reasonable, simple imputation (mean/median/mode) can work.

Illustrative Data Example

STUDENT ID	AGE	SCORE
101	20	85
102	Missing	88
103	21	Missing
104	23	91
105	Missing	76
106	22	Missing

Missing values appear across different rows and columns with no clear pattern, which is consistent with MCAR.

Missing at Random (MAR)

MAR means missingness is related to observed features in the dataset, but not to the missing feature itself.

Imputation Techniques for MAR

METHOD	IDEA	WHY IT WORKS FOR MAR
kNN Imputation	Fills missing values using the nearest similar records.	Uses observed features to find similar patterns.
MissForest	Uses random-forest models iteratively to predict missing values.	Captures non-linear relationships from observed data effectively.

Example: kNN Imputation (k = 2)

For student C, the two nearest students based on Grade and Attendance are A and B.

$$\text{Study Hours}_C = \frac{8 + 7}{2} = 7.5$$

STUDENT	GRADE	ATTENDANCE (%)	STUDY HOURS
A	90	95	8
B	88	92	7
C	91	96	Missing
D	70	75	3

Example: MissForest

MissForest trains a Random Forest model using known rows to predict the missing value for student C.

$\text{Study Hours}_C \approx 7.6$

Example (Table)

STUDENT	GRADE (OBSERVED)	STUDY HOURS
1	95	Missing
2	92	Missing
3	88	4
4	75	6
5	70	7

MAR is common in practice because missingness can often be explained by available variables.



Handling Missing Data - MNAR

Missing Not at Random (MNAR)

ETRA

MNAR is usually the most complex type because missingness depends on the missing value itself or unobserved variables.

- Standard imputation can be unreliable without additional data or strong domain assumptions.
- MNAR has a systematic pattern tied to hidden information.

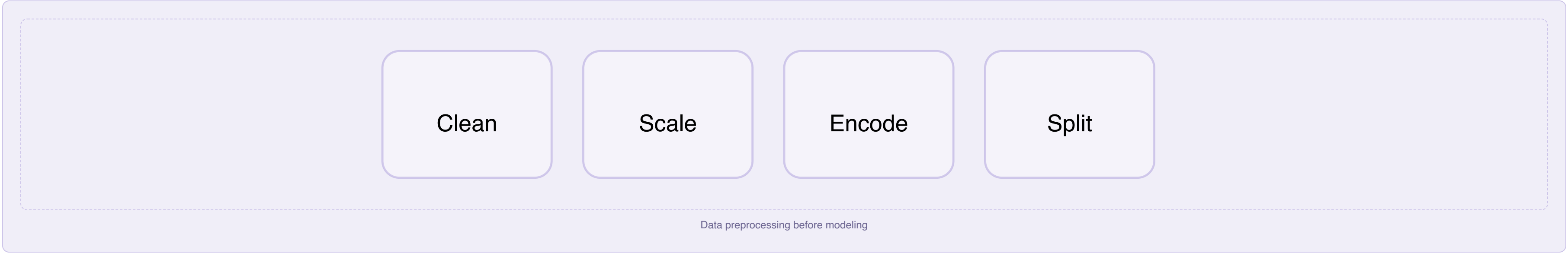
Example (Table)

PARTICIPANT	REPORTED STRESS LEVEL
1	2
2	Missing
3	3
4	Missing
5	1

Practical Tip: Preserve Missingness Pattern

PARTICIPANT	STRESS (IMPUTED)	STRESS_WAS_MISSING
1	2.0	0
2	4.1	1
3	3.0	0
4	4.3	1
5	1.0	0

Standard Workflow



- Load the dataset.
- Detect and handle missing values.
- Encode categorical features into numerical form.
- Split the dataset into training and test sets.
- Scale numerical features when needed.

Pipeline: Raw Data -> Missing Data Treatment -> Categorical Encoding -> Train/Test Split -> Feature Scaling -> ML Model.

Section Takeaways

- 1. Always split data before touching the model — test set simulates production.
- 2. Feature scaling matters for distance- and gradient-based algorithms.
- 3. Preprocessing choices persist through the entire pipeline.

Think: Which preprocessing step would break your model if skipped?

WEEK 1 · SESSION 2 · Foundations

2

Regression Models

Linear regression

OLS & R^2

Residual diagnostics

Ridge / Lasso

▸ 37 slides · ~59 min

Where Are We in the Bootcamp?

Learn how models quantify relationships and when linear assumptions hold.

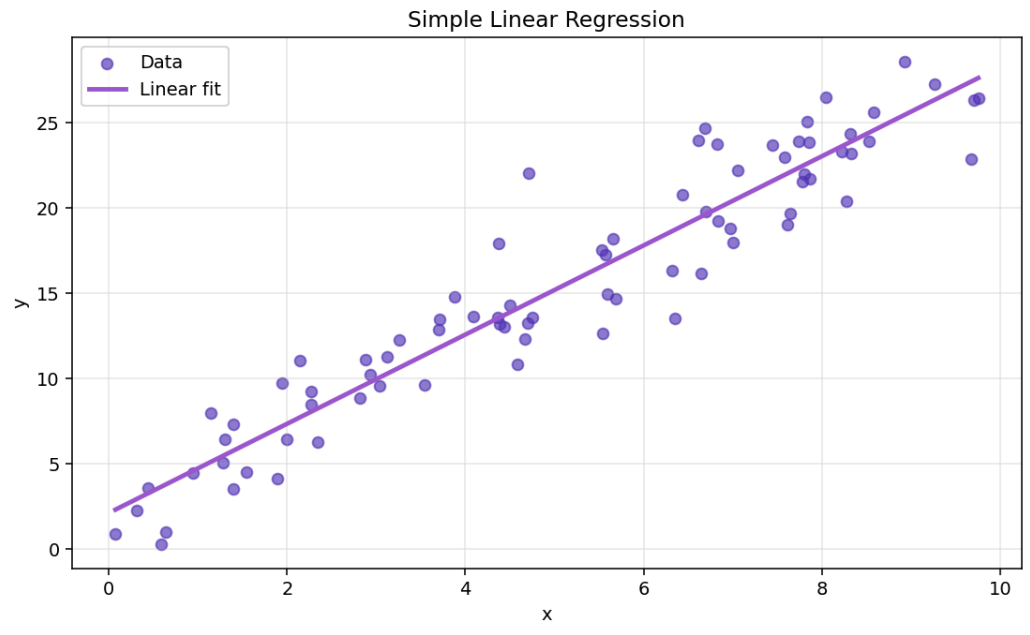
- **S1** Foundations & Preprocessing
- **S2 Regression Models**
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Regression - Simple Linear Regression

What is Simple Linear Regression?

Simple Linear Regression models the relationship between one independent variable x and one dependent variable y using a straight line.

$$y = b_0 + b_1x$$



Practical Example

Suppose x = Hours of Study and y = Exam Score.

$$y = 40 + 5x$$

Practical Example

| Slope ($b_1 = 5$): each extra study hour increases predicted score by 5 points.

| Intercept ($b_0 = 40$): if study hours are 0, predicted score is 40.

Meaning of Each Symbol

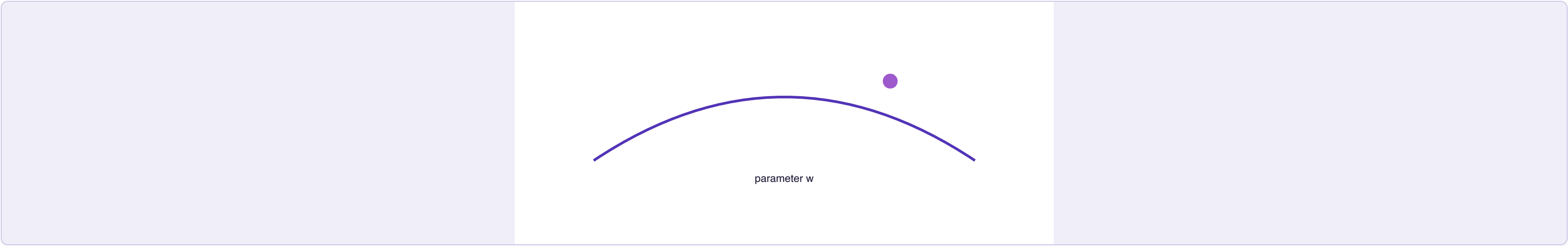
SYMBOL	MEANING
y	Predicted output (dependent variable).
x	Input feature (independent variable).
b_0	Intercept: predicted value of y when x = 0.
b_1	Slope: expected change in y when x increases by 1 unit.

The regression line is the best-fit line that minimizes prediction errors.

Ordinary Least Squares (OLS)

What is OLS?

Ordinary Least Squares is the most common method to fit a linear regression line by minimizing prediction errors.



$$\min \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Actual value: y_i

Predicted value: $\hat{y}_i$

Residual error: $e_i = y_i - \hat{y}_i$

Squared errors make all errors positive, penalize large errors, and provide a clear optimization objective.

Multiple Linear Regression

Definition

Multiple Linear Regression describes the relationship between one dependent variable y and two or more independent variables.

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Purpose: predict the target variable y using multiple input factors.

Compared with simple regression, it models more realistic multi-factor scenarios.

Example

Suppose we want to predict house price using house size and number of bedrooms.

$$y = 50 + 2.5x_1 + 15x_2$$

For each additional 1 m^2 , price increases by 2.5 (holding other variables constant).

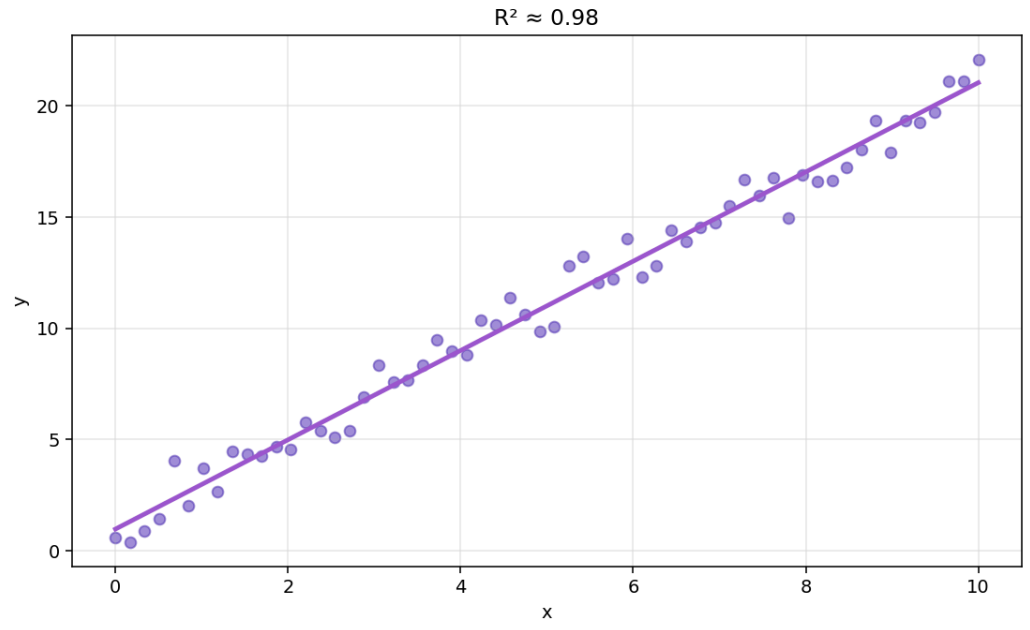
For each additional bedroom, price increases by 15 (holding other variables constant).

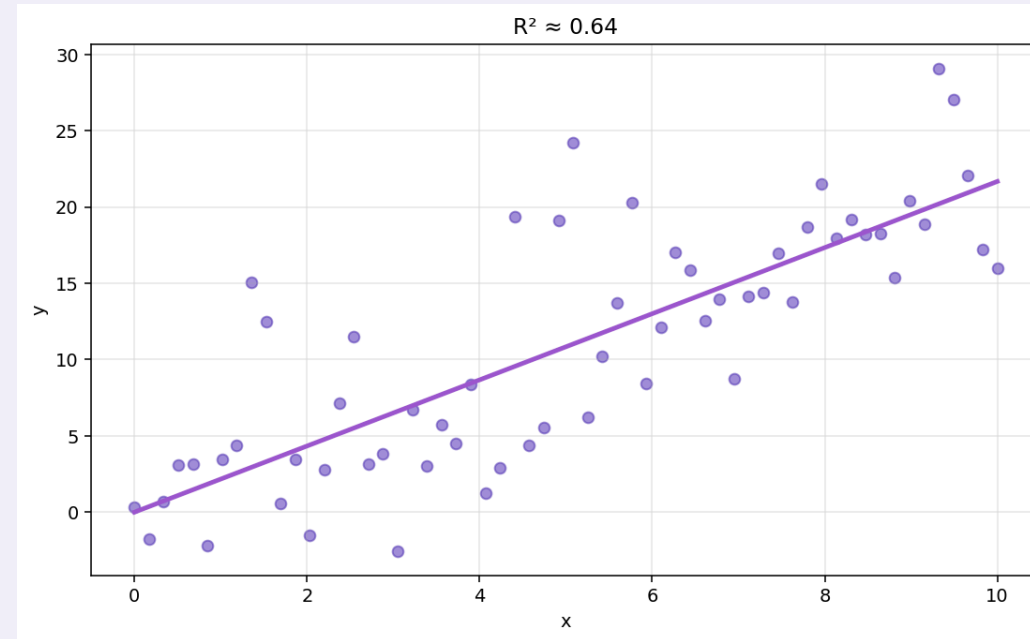
Table for Clarification

HOUSE	SIZE (X_1) M^2	BEDROOMS (X_2)	PREDICTED PRICE (Y IN \$1000)
A	100	2	$50 + 2.5(100) + 15(2) = 330$
B	120	3	$50 + 2.5(120) + 15(3) = 395$
C	80	1	$50 + 2.5(80) + 15(1) = 265$

R^2 is a goodness-of-fit metric in regression. It measures how much variation in y is explained by model inputs.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$





Larger R^2 means better explanatory power.

$R^2 = 1$ means perfect fit; $R^2 = 0$ means mean-level prediction performance.

Formulas

$$SS_{res} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Formulas (continued)

$$SS_{tot} = \sum_{i=1}^n (y_i - \bar{y})^2$$

What Do These Terms Mean?

TERM	DESCRIPTION
SS_res	Residual Sum of Squares: squared distance between actual values and predictions.
SS_tot	Total Sum of Squares: squared distance between actual values and the mean of y.

Why R^2 Alone Is Not Enough

In multiple regression, adding predictors often increases R^2 even when new variables are not useful.

$$\bar{R}^2 = 1 - (1 - R^2) \frac{n - 1}{n - p - 1}$$

Adjusted R^2 penalizes unnecessary predictors.

It is usually a better metric for comparing models with different numbers of predictors.

Quick Example

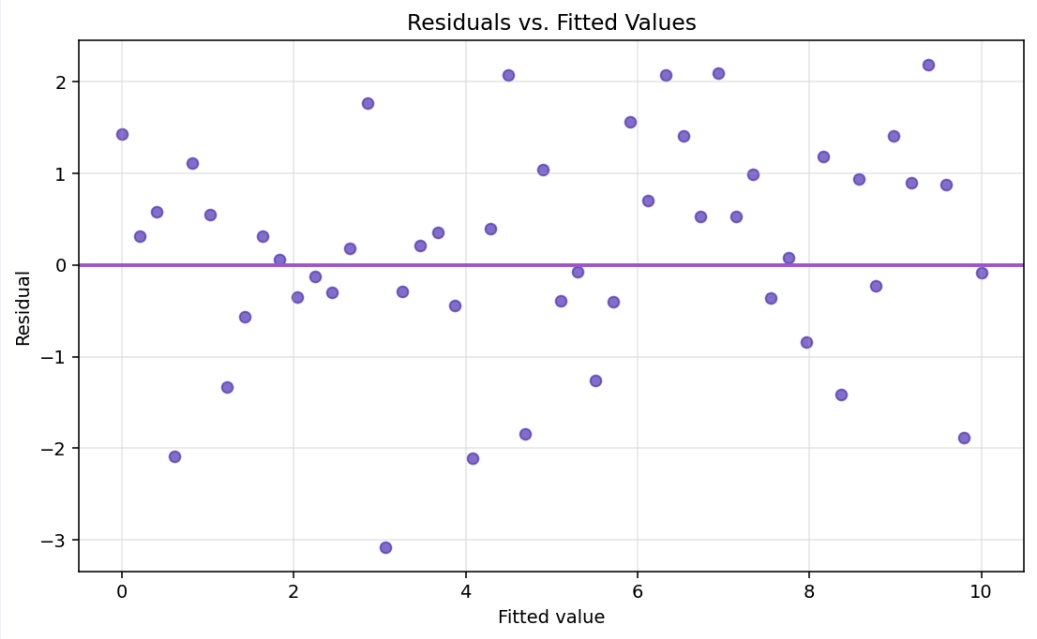
MODEL	PREDICTORS	R^2	ADJUSTED R^2	INTERPRETATION
A	x_1, x_2	0.82	0.81	Strong and efficient model
B	x_1, x_2, x_3, x_4	0.83	0.79	Higher R^2 , but worse after penalty

Model B looks better by R^2 , but Adjusted R^2 shows extra variables are likely not helpful.

Assumptions of Linear Regression (Overview)

Linear regression is most reliable when core assumptions are approximately satisfied.

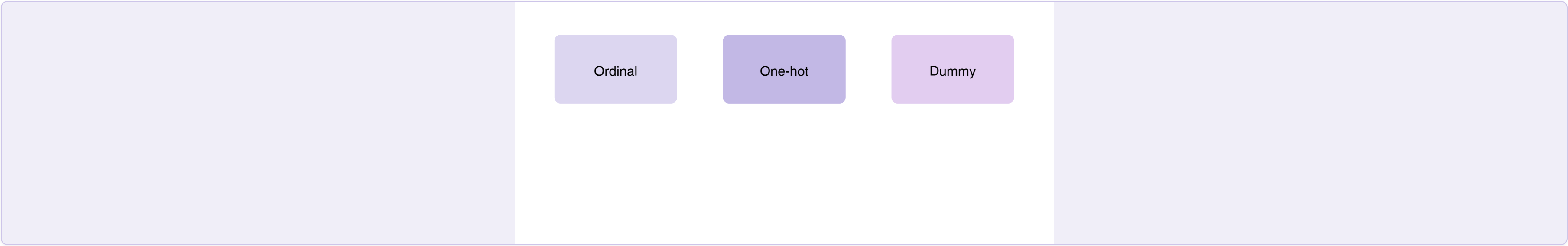
ASSUMPTION	MEANING	IF VIOLATED
Linearity	Relationship between y and each x is roughly linear.	Biased predictions
Homoscedasticity	Error variance is roughly constant across fitted values.	Unstable standard errors
Normality of Errors	Residuals are approximately normal.	Inference becomes less reliable
Independence	Observations/errors are independent.	Biased significance tests
No Severe Multicollinearity	Predictors are not highly redundant.	Coefficients become unstable
Limited Outlier Influence	Extreme points do not dominate the fit.	Distorted regression line



ASSUMPTION	SIMPLE EXAMPLE	PRACTICAL CHECK
Linearity	Sales increase approximately linearly with ad spend.	Scatter plot / residual plot
Homoscedasticity	Prediction errors are similar for low and high sales values.	Residuals vs fitted values
Normal Errors	Most residuals are near zero, few at extremes.	Histogram / Q-Q plot of residuals
Independence	Customer records are independent.	Study design / Durbin-Watson for time data
Low Multicollinearity	TV_ads and radio_ads are not near-duplicates.	Correlation matrix / VIF
Outlier Control	One extreme house price should not define the whole model.	Cook's distance / leverage diagnostics

If one assumption is violated, use transformations, robust methods, feature engineering, or a different model.

Categorical variables are converted into binary columns (0/1).



Dummy Variable Trap

If all dummy columns are included with an intercept, perfect multicollinearity appears.

$$D_2 = 1 - D_1$$

Coefficients become unstable.

Interpretation becomes unreliable.

Statistical tests may be misleading.

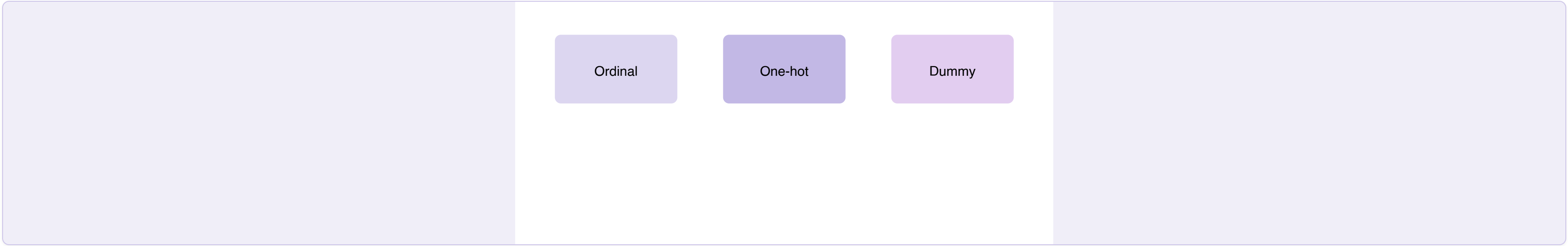
Example: State

STATE	D_1 (NEW YORK)	D_2 (CALIFORNIA)
New York	1	0
California	0	1

Solving Dummy Variable Trap

Solution

Drop one dummy column and keep it as the baseline category.



$$\text{Profit} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + b_4D_1$$

If we drop California, then $D_1 = 1$ means New York and $D_1 = 0$ means California baseline.

This keeps category information while avoiding multicollinearity.

Interpretation Example

CASE	D_1	STATE MEANING
A	1	New York
B	0	California (reference)

Goal

Model-building methods select the most relevant predictors to balance simplicity, interpretability, and predictive accuracy.

Method	Core Idea	Best Use Case
All-in	Use all predictors directly.	Strong prior knowledge that all variables matter
Backward Elimination	Start full and remove least significant variables.	Many candidate predictors
Forward Selection	Start empty and add most significant variables.	Need a compact model from scratch
Bidirectional (Stepwise)	Add and remove dynamically.	Flexible search for balanced model

1) Backward Elimination

Set significance level (e.g., $SL = 0.05$).

Fit model with all predictors.

Remove highest p-value if $p > SL$.

Refit and repeat.

2) Forward Selection

Start with no predictors.

Add variable with lowest p-value below SL.

Continue one-by-one.

Stop when no variable qualifies.

3) Bidirectional (Stepwise)

Add significant variables and remove non-significant ones each iteration.

Mini Example

ITERATION	CANDIDATE PREDICTORS	SELECTED ACTION
Start	x_1, x_2, x_3, x_4	Full model
1	highest p-value: $x_4 = 0.42$	Remove x_4
2	highest p-value: $x_3 = 0.11$	Remove x_3
Final	x_1, x_2 with $p < 0.05$	Keep final model

$$0.03 < 0.05 \Rightarrow \text{Reject } H_0$$

If $p < \alpha$: reject H_0 (statistically significant).

If $p \geq \alpha$: fail to reject H_0 .

CONCEPT	MEANING
Null Hypothesis (H_0)	No effect / no relationship
p-value	Probability of observing current result (or more extreme) if H_0 is true
Significance Level (α)	Maximum tolerated Type I error probability

So the predictor is considered significant at the 5% level.

Why Is $SL = 0.05$ Common?

Standard Choice

$SL = 0.05$ is widely used because it balances sensitivity and false-positive risk.

$$SL = 1 - \text{Confidence Level}$$

- | $\alpha = 0.05$ implies a 5% false-positive rate under the null hypothesis.
- | It is a convention, not a law — adjust for domain risk (medical vs A/B tests).

Report effect size and confidence intervals alongside p-values.

SL	CONFIDENCE LEVEL	STRICTNESS
0.10	90%	Less strict
0.05	95%	Standard
0.01	99%	More strict

How to Choose in Practice

SCENARIO	SUGGESTED SL
High-stakes decisions (medicine, safety, finance)	0.01
General modeling and reporting	0.05
Exploratory analysis	0.10

Start with SL = 0.05, then adjust based on domain risk, dataset size, and model purpose.

Polynomial Regression

Definition

Polynomial Regression extends linear regression to model non-linear relationships by adding polynomial terms.

- Linear regression: $y = b_0 + b_1x$
- Polynomial regression: $y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots$
- It is still linear in parameters because coefficients remain linear.

Why Polynomial Regression?

Simple linear regression fits only straight lines, but many real-world relationships are curved.

- Population growth over time
- Disease progression
- Sales and economic trends
- Rainfall vs crop yield relationships
- Curved or U-shaped data is often underfit by a straight line.

$$y = b_0 + b_1x + b_2x^2 + \cdots + b_dx^d$$

STEP	DESCRIPTION
1	Start with input feature x
2	Create transformed terms: $x^2, x^3, \dots$
3	Fit linear regression on transformed features
4	Estimate coefficients ($b_0, b_1, b_2, \dots$)
5	Use the fitted equation to predict y

d is the polynomial degree.

The polynomial degree controls model complexity.

- Use cross-validation.
- Compare adjusted R^2 .
- Compare validation/test error.

DEGREE CHOICE	EFFECT
Too low	Underfitting (model too simple)
Too high	Overfitting (fits noise, weak generalization)
Balanced degree	Better bias-variance trade-off

When to Use Polynomial Regression

Good Use Cases

- Data shows clear curved trends.
- You need more flexibility than a straight line.
- You still want interpretable behavior.

Applications

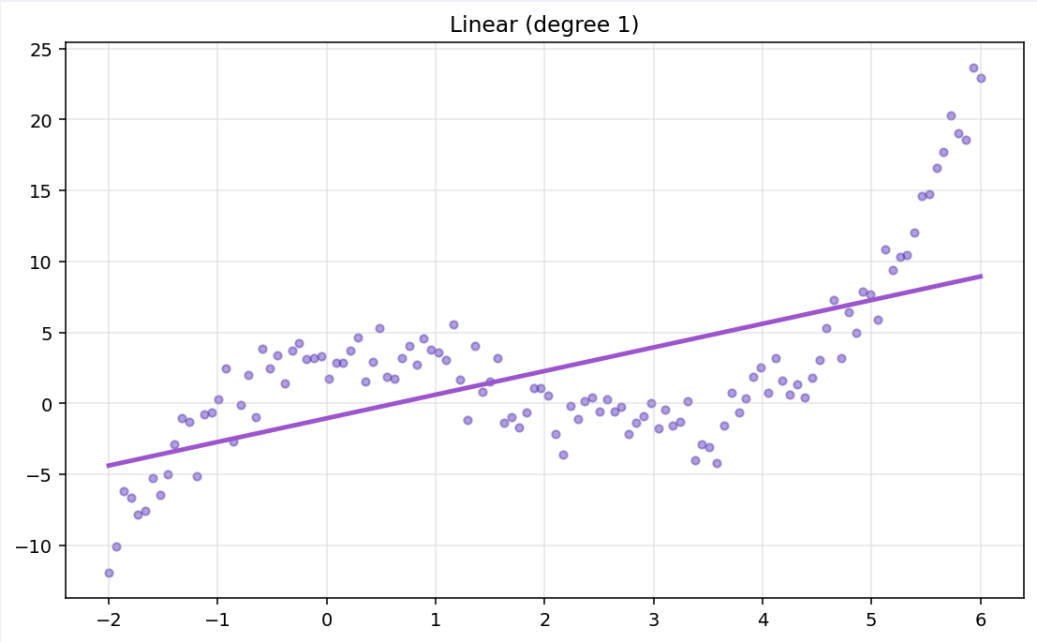
- Forecasting demand and sales trends
- Environmental modeling (rainfall vs yield)
- Medical progression patterns
- Engineering curves (stress-strain relationships)

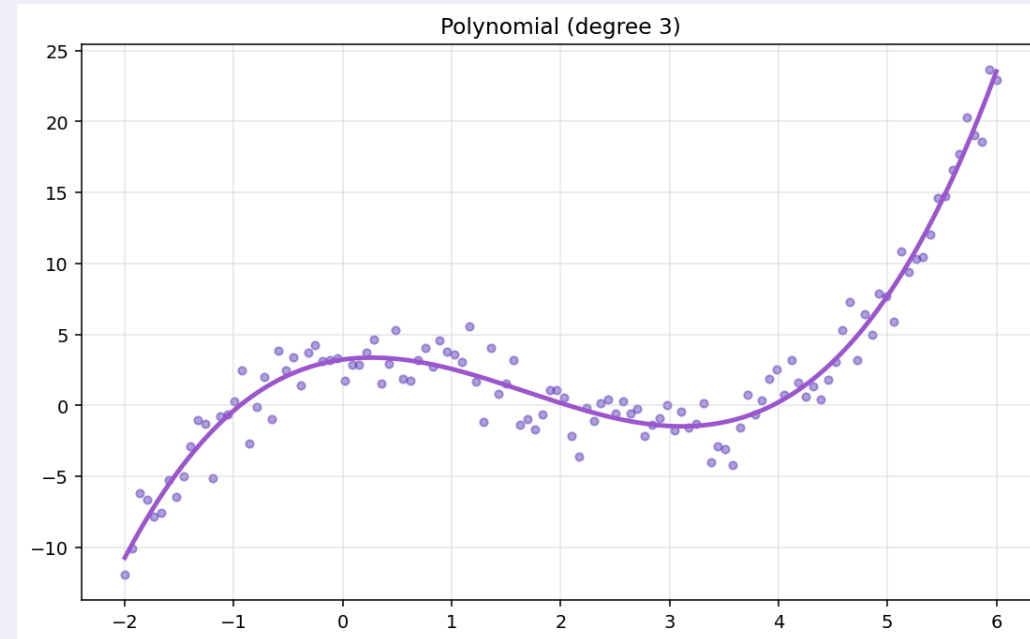
LIMITATION	EXPLANATION
Overfitting risk	High-degree models may memorize noise
Outlier sensitivity	Extreme points can shift the curve strongly
Extrapolation weakness	Predictions outside training range can be unreliable
Limited flexibility for very complex patterns	Some tasks are better handled by tree-based or neural models

Use polynomial regression as a baseline, then compare with alternatives as complexity grows.

Polynomial Regression Visual Explanation

This section compares linear and polynomial fits, and shows how model behavior changes with polynomial degree.





Underfitting: degree too low — high bias.

Overfitting: degree too high — high variance.

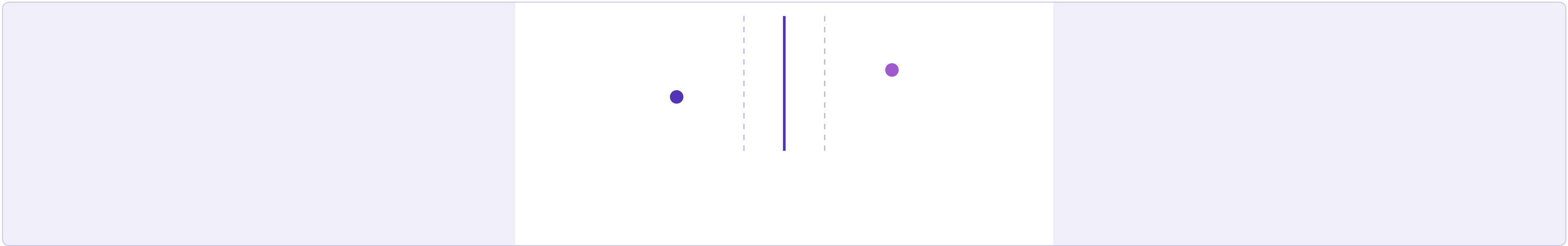
Use validation curves to pick degree — not training error alone.

Comparison plots are shown below.

Support Vector Machine (SVM) and SVR

What is SVR?

Support Vector Regression (SVR) is the regression version of SVM. It predicts a continuous target with a flat function and a tolerance margin.



- Used in disease progression prediction
- Used in engineering curves (e.g., stress-strain)
- Used in demand and trend forecasting
- SVR uses an epsilon-insensitive zone where small errors are ignored.

$$f(x) = w^T \phi(x) + b$$

$\phi(x)$ maps input to a (possibly) higher-dimensional space.

With kernels, we avoid computing $\phi(x)$ explicitly.

Optimization Objective (Soft-Margin SVR)

$$\min_{w,b,\xi_i,\xi_i^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*)$$

Constraints

$$y_i - (w^T \phi(x_i) + b) \leq \varepsilon + \xi_i$$

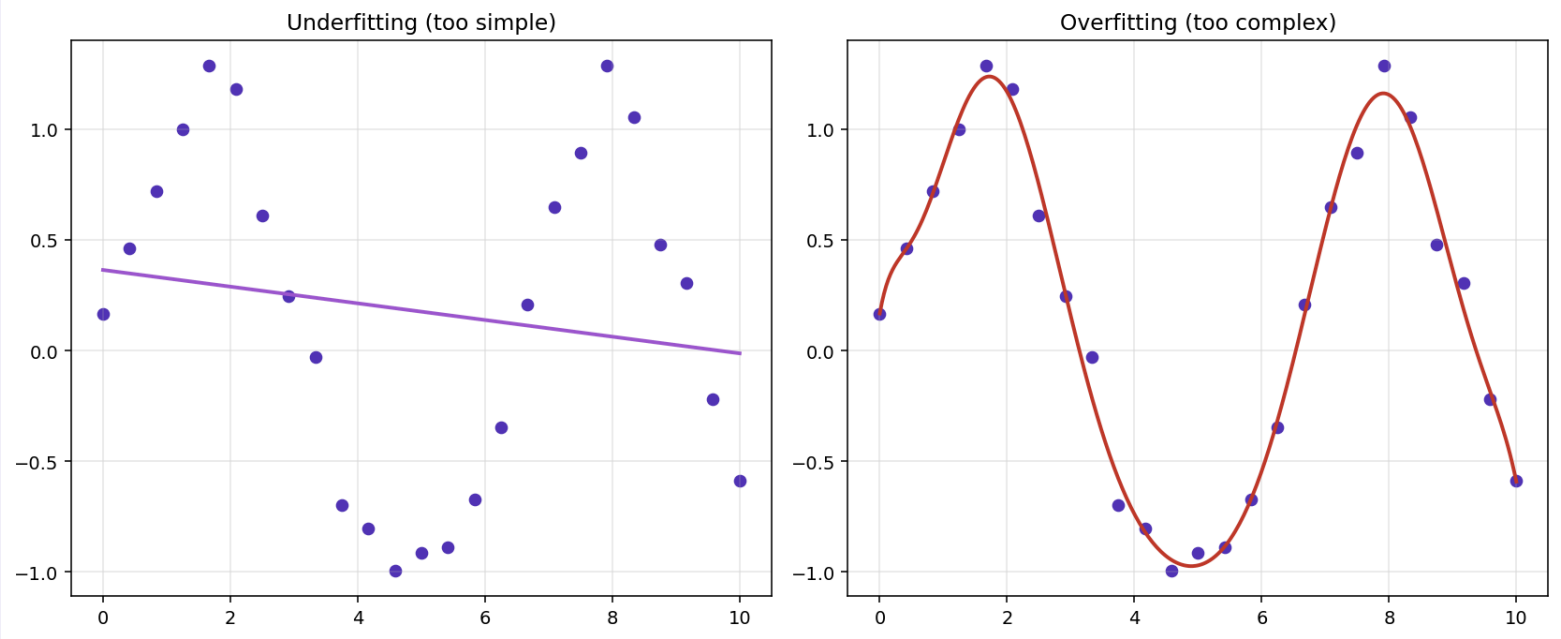
Constraints (continued)

$$(w^T \phi(x_i) + b) - y_i \leq \varepsilon + \xi_i^*$$

| $\xi_i, \xi_i^* \geq 0$

Hyperparameters

HYPERPARAMETER	ROLE
epsilon	Width of the no-penalty tube around prediction
C	Penalty strength for points outside the tube
Kernel	Controls curve shape (linear, RBF, polynomial)





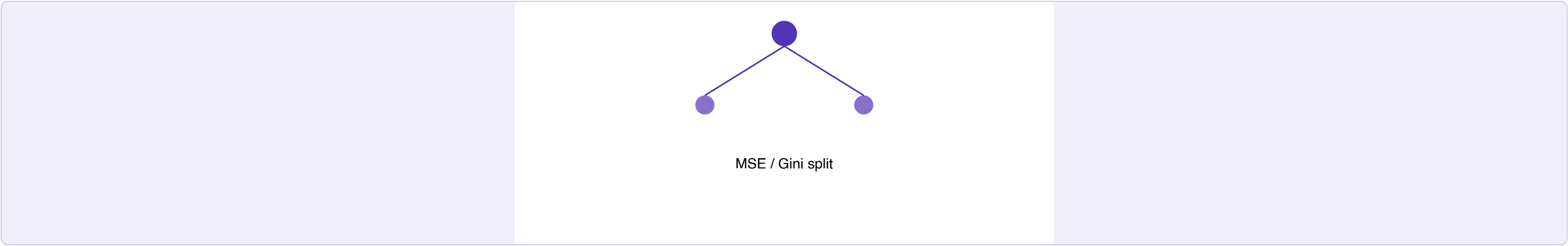
- Use feature scaling before SVR.
- Start with RBF kernel, then tune C , epsilon, and kernel parameters.

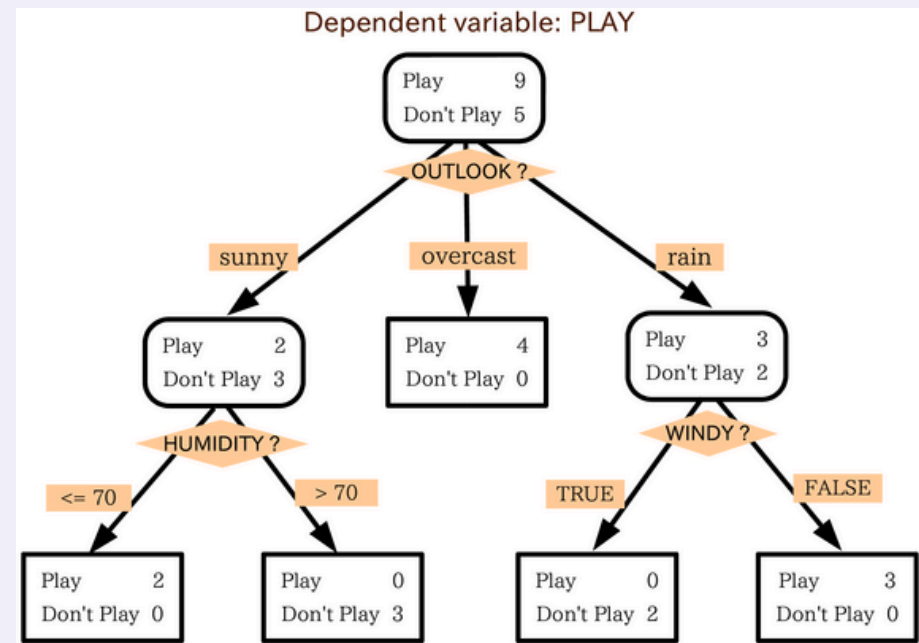
ASPECT	TRADITIONAL REGRESSION	SVR
Error handling	Minimizes all residuals	Ignores errors inside epsilon-tube
Outlier sensitivity	Can be sensitive	More robust when tube and C are tuned
Non-linear handling	Needs explicit feature engineering	Uses kernels directly
Large datasets	Usually faster for very large data	Can become expensive for very large sample sizes

Decision Tree Regression (CART)

What is Decision Tree Regressor?

Decision Tree Regressor predicts continuous values by splitting feature space into rule-based regions and assigning a constant value in each region.





Captures non-linear relationships.

Easy to interpret if tree depth is controlled.

Handles complex feature interactions.

Internal node: decision rule (e.g., $x \leq 4.5$).

Branch: outcome of the rule.

Leaf node: predicted value (usually region mean).

CART Perspective

Tree Type	Target Type	Typical Criterion
Classification Tree	Categorical target	Gini / Entropy
Regression Tree	Continuous target	MSE (or MAE)

This section focuses on the Regression Tree part of CART.

How Splitting Happens (MSE Criterion)

CART picks the split that maximally reduces MSE (regression) or Gini/entropy (classification).

Greedy splits are fast but not globally optimal.

Depth and min-samples-leaf control overfitting on small datasets.

Node MSE

$$\text{MSE}(S) = \frac{1}{|S|} \sum_{i \in S} (y_i - \bar{y}_S)^2$$

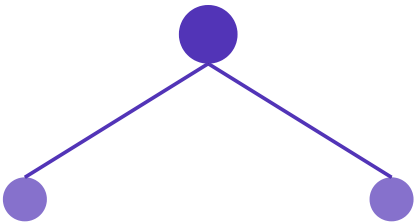
Split Score

$$\text{MSE}_{\text{split}} = \frac{|S_L|}{|S|} \text{MSE}(S_L) + \frac{|S_R|}{|S|} \text{MSE}(S_R)$$

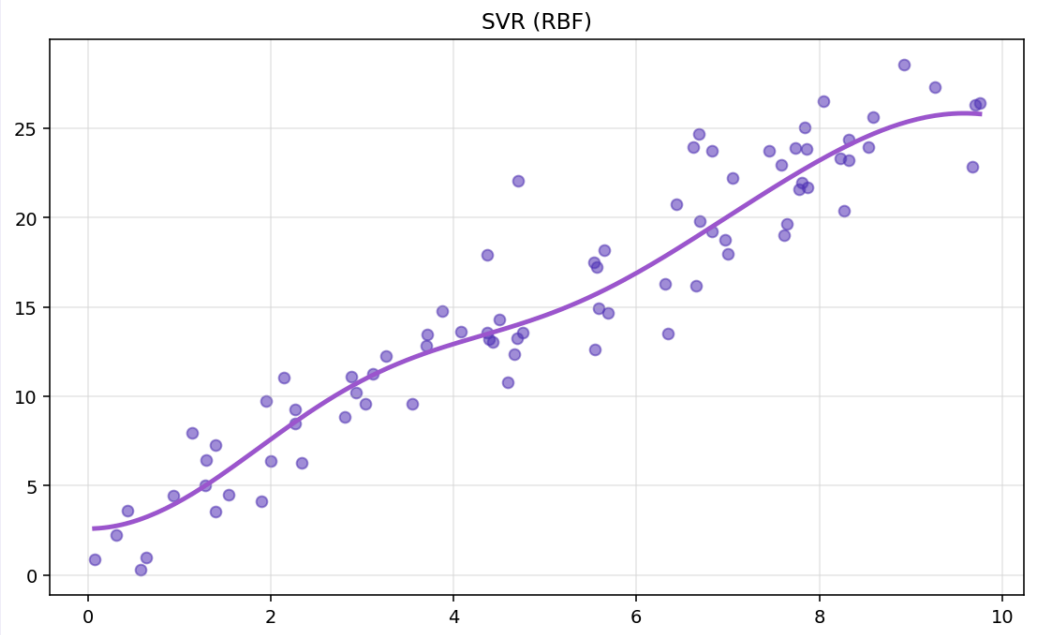
Prediction at Leaf

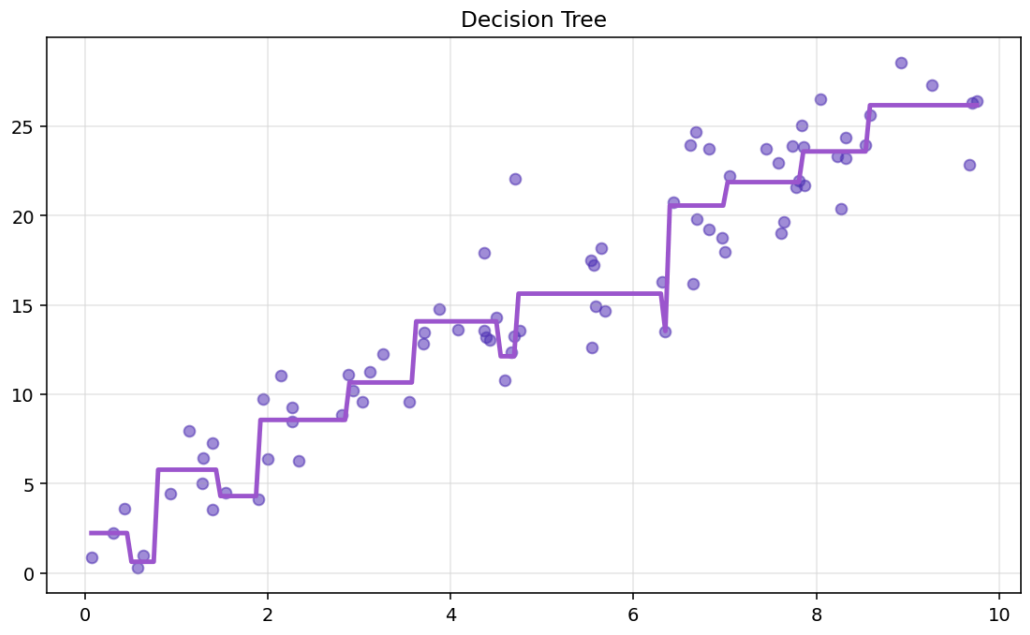
$$\hat{y}_{\text{leaf}} = \frac{1}{|S_{\text{leaf}}|} \sum_{i \in S_{\text{leaf}}} y_i$$

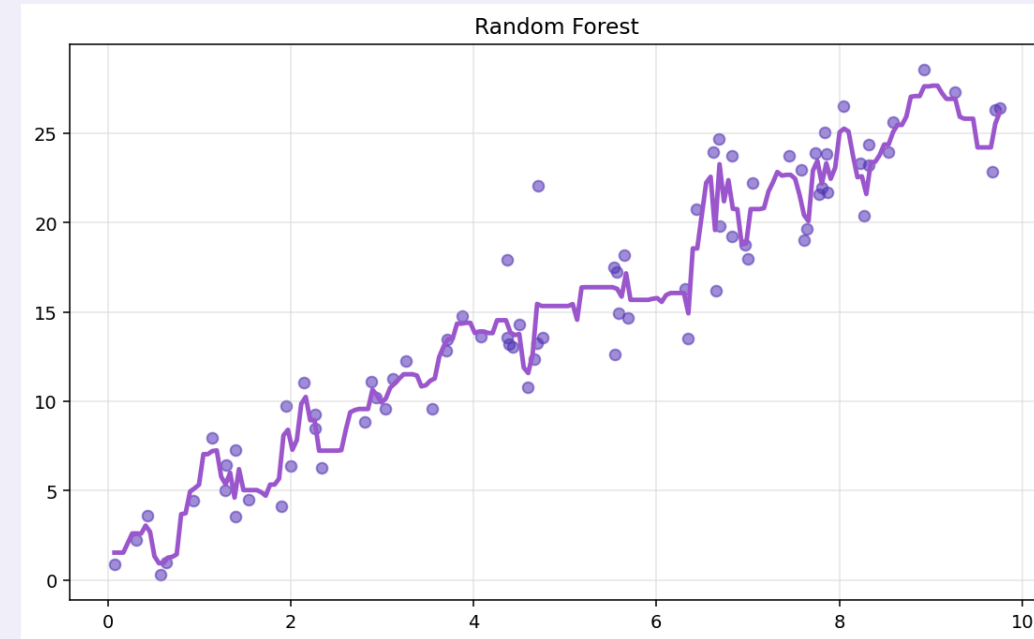
Choose the split with the minimum weighted MSE.



MSE / Gini split







Start with all data at the root node.

Evaluate candidate splits across features.

Compute weighted MSE for each split.

Select the split with minimum error.

Repeat recursively on child nodes.

Stop based on rules (max depth, min samples).

Predict using leaf mean value.

Important Hyperparameters

HYPERPARAMETER	EFFECT
max_depth	Controls model complexity
min_samples_split	Minimum samples required to split a node
min_samples_leaf	Minimum samples in each leaf
max_leaf_nodes	Limits number of terminal regions

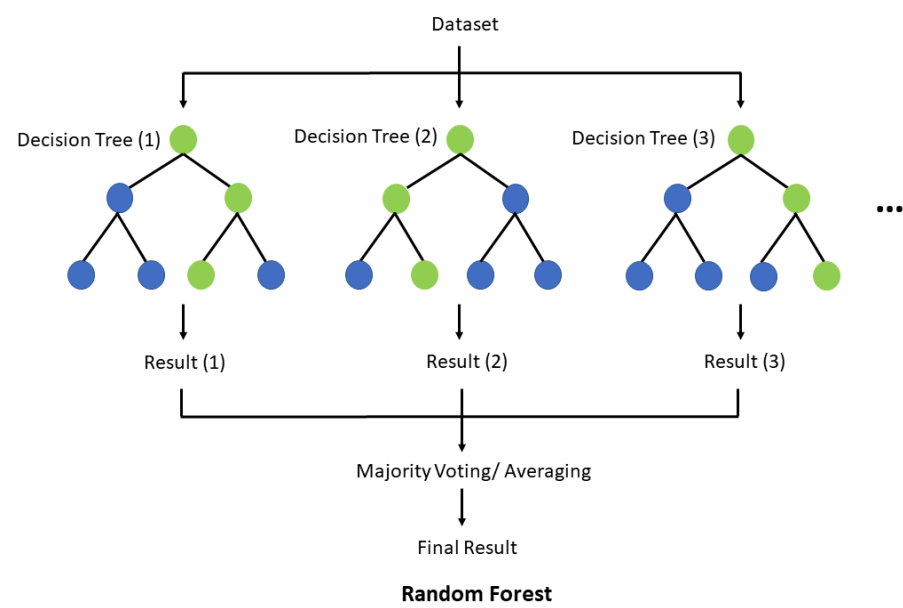
These settings reduce overfitting and improve generalization.

Random Forest Regression

What is Random Forest Regressor?

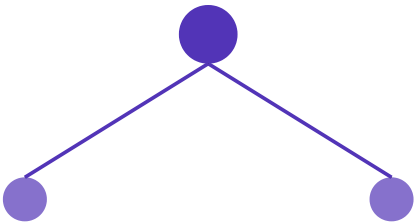
Random Forest Regression is an ensemble method that combines many decision trees and averages their predictions.

$$\hat{y}_{RF}(x) = \frac{1}{N_{trees}} \sum_{t=1}^{N_{trees}} \hat{y}_t(x)$$

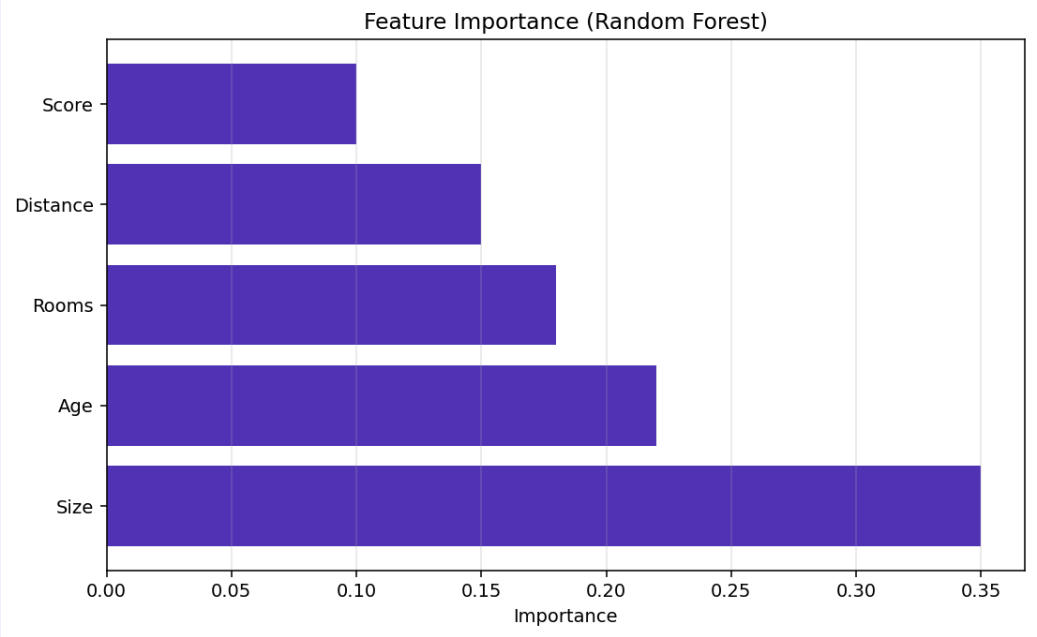


- Sample training data with bootstrap.
- Build many trees on different samples.
- Predict with all trees and average outputs.

Averaging reduces variance and usually improves generalization.



MSE / Gini split



ASPECT	DECISION TREE REGRESSOR	RANDOM FOREST REGRESSOR
Model type	Single tree	Ensemble of many trees
Split criterion	MSE per split	Trees split by MSE, then averaged
Overfitting risk	Higher	Lower (variance reduction)
Stability	Sensitive to data changes	More stable
Final prediction	Output of one tree	Mean of all tree outputs
Interpretability	High	Medium

Use a single tree for interpretability; use random forest for stronger performance.

Key Performance Metrics

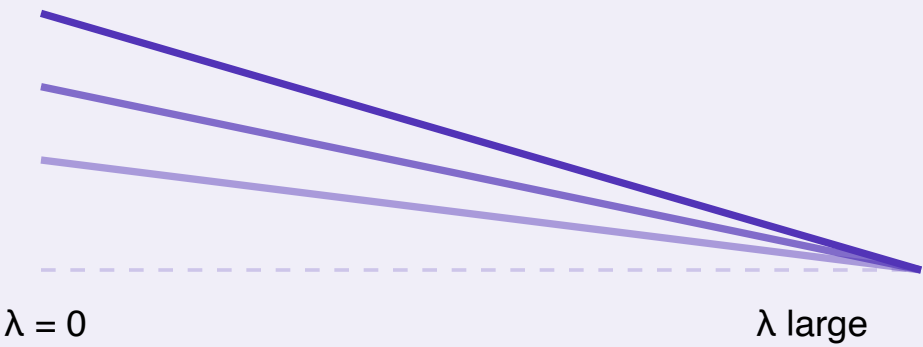
Metric	Formula	What it tells us
MAE	$\frac{1}{n} \sum y_i - \hat{y}_i $	Average prediction error magnitude
MSE	$\frac{1}{n} \sum (y_i - \hat{y}_i)^2$	Penalizes large errors more
RMSE	$\sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2}$	Error in original target scale
R^2	$1 - \frac{SS_{res}}{SS_{tot}}$	Fraction of variance explained

Evaluate using more than one metric (e.g., RMSE with R^2) for balanced judgment.

Regularization Methods (Why Needed?)

Overfitting Problem

When a model is too flexible, it may fit noise instead of true patterns.



Coefficients shrink with regularization strength

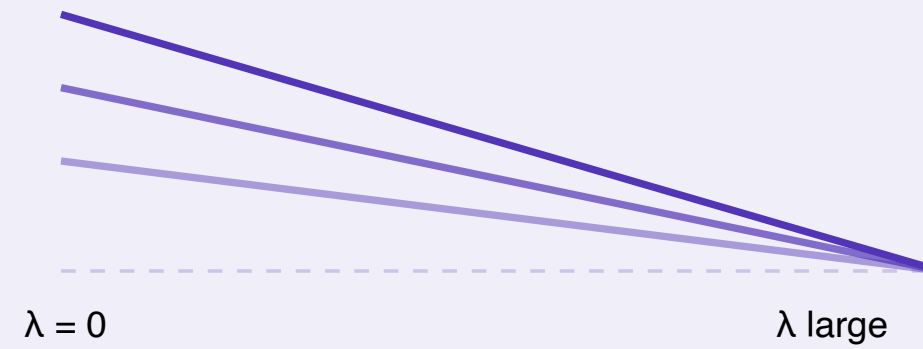
$$\min_{\beta} \underbrace{\sum_{i=1}^n (y_i - \hat{y}_i)^2}_{\text{data loss}} + \lambda \cdot \underbrace{\Omega(\beta)}_{\text{penalty}}$$

Very low training error

Weak test performance

High variance and unstable predictions

Regularization controls complexity by penalizing large coefficients.



Coefficients shrink with regularization strength

Ridge (L2): shrinks all coefficients — good when many features correlate.

Lasso (L1): can zero out coefficients — embedded feature selection.

Elastic Net blends L1 + L2 for correlated sparse settings.

Without Regularization

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Ridge Regression (L2)

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m \beta_j^2$$

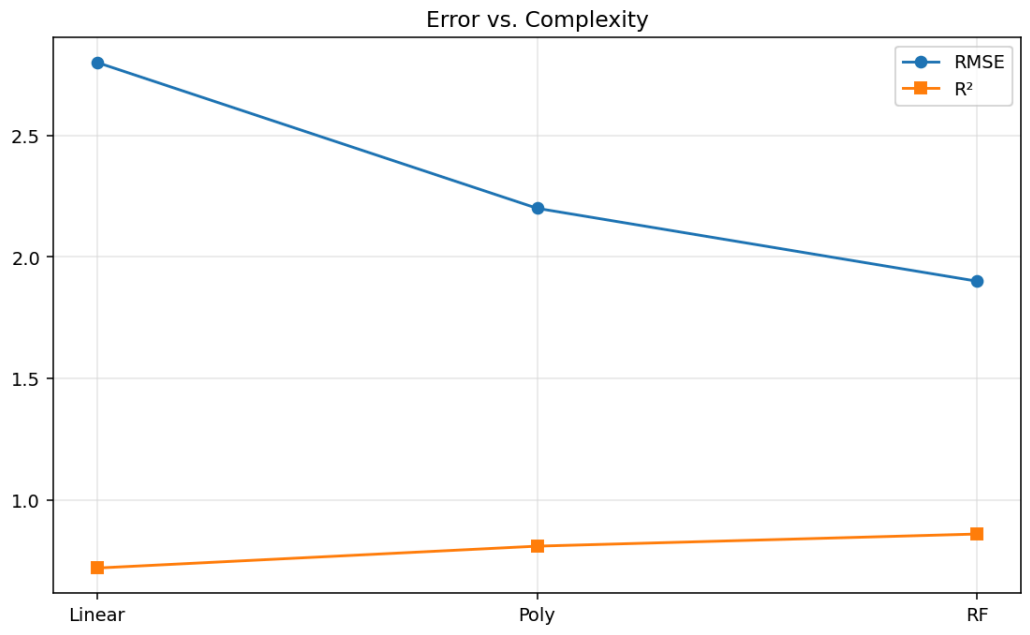
Lasso Regression (L1)

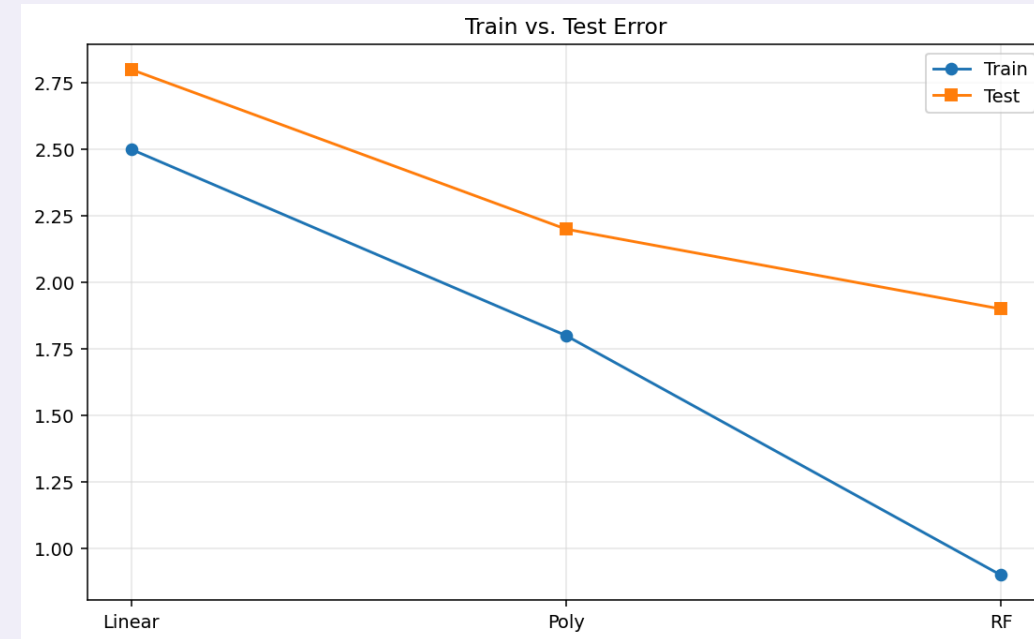
$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^m |\beta_j|$$

Elastic Net (L1 + L2)

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \sum_{j=1}^m |\beta_j| + \lambda_2 \sum_{j=1}^m \beta_j^2$$







Ridge: many small/medium useful features.

Lasso: only few important features expected.

Elastic Net: correlated features and need both stability and sparsity.

METHOD	PENALTY TYPE	MAIN BEHAVIOR	FEATURE SELECTION
Ridge	L2 (sum beta_j^2)	Shrinks coefficients smoothly	No
Lasso	L1 (sum beta_j)	Shrinks and can set some coefficients to zero	Yes
Elastic Net	L1 + L2	Combines shrinkage + selection	Yes

Regression Models - Advantages and Disadvantages

MODEL	ADVANTAGES	DISADVANTAGES
Linear Regression	Simple, fast, easy to interpret, works well for linear patterns.	Assumes linear relationship and is sensitive to outliers.
Polynomial Regression	Captures curved/non-linear relationships better than linear regression.	Choosing degree is sensitive; high degrees may overfit.
SVR	Handles non-linear patterns with kernels and can be robust to noise with proper tuning.	Requires feature scaling and careful hyperparameter tuning; slower on large datasets.
Decision Tree Regression	Interpretable rules, captures non-linear interactions, no feature scaling required.	Can overfit easily and may be unstable with small data changes.
Random Forest Regression	Higher accuracy and stability, reduces overfitting by averaging many trees.	Less interpretable and can be computationally heavier.

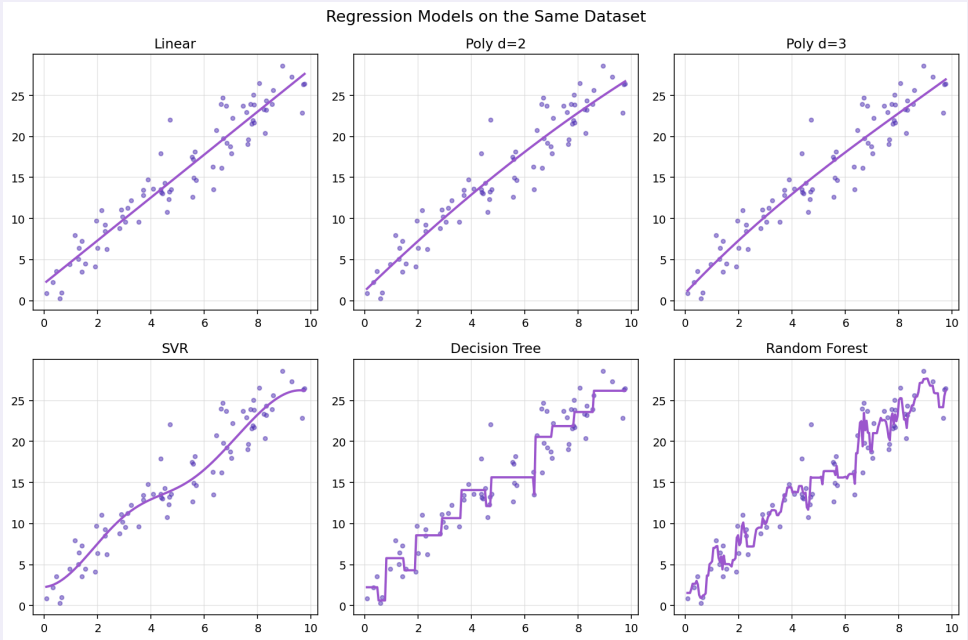
Model Summary (Name + Equation)

MODEL	CORE EQUATION
Linear Regression	$\hat{y} = b_0 + b_1x$
Polynomial Regression	$\hat{y} = b_0 + b_1x + b_2x^2 + \dots + b_dx^d$
SVR	$f(x) = w^T\phi(x) + b$
Decision Tree Regression	$\hat{y}_{leaf} = \frac{1}{ S_{leaf} } \sum_{i \in S_{leaf}} y_i$
Random Forest Regression	$\hat{y}_{RF}(x) = \frac{1}{N_{trees}} \sum_{t=1}^{N_{trees}} \hat{y}_t(x)$

If interpretability is priority: Linear Regression or Decision Tree. If predictive performance is priority: Random Forest.

Visual Comparison of Regression Models

This slide compares model behavior on the same dataset: Linear Regression, Polynomial Regression, SVR, Decision Tree Regression, and Random Forest Regression.



Model families at a glance

MODEL	STRENGTH	WATCH OUT
Linear / Ridge	Interpretable, fast	Nonlinear patterns
SVR	Margin-based, kernels	Slow on huge data
Tree / RF	Nonlinear, little scaling	Extrapolation

Visual output from Python code is shown below.

Section Takeaways

1. R^2 measures explained variance — not accuracy or causation.

2. OLS minimizes squared errors; outliers can dominate the fit.

3. Regularization (Ridge/Lasso) trades bias for stable coefficients.

Think: When would a linear model be misleading even with high R^2 ?

WEEK 1 · SESSION 3 · Classical ML

3

Classification Basics

Logistic regression K-NN Decision boundaries Threshold tuning

▸ 13 slides · ~21 min

Where Are We in the Bootcamp?

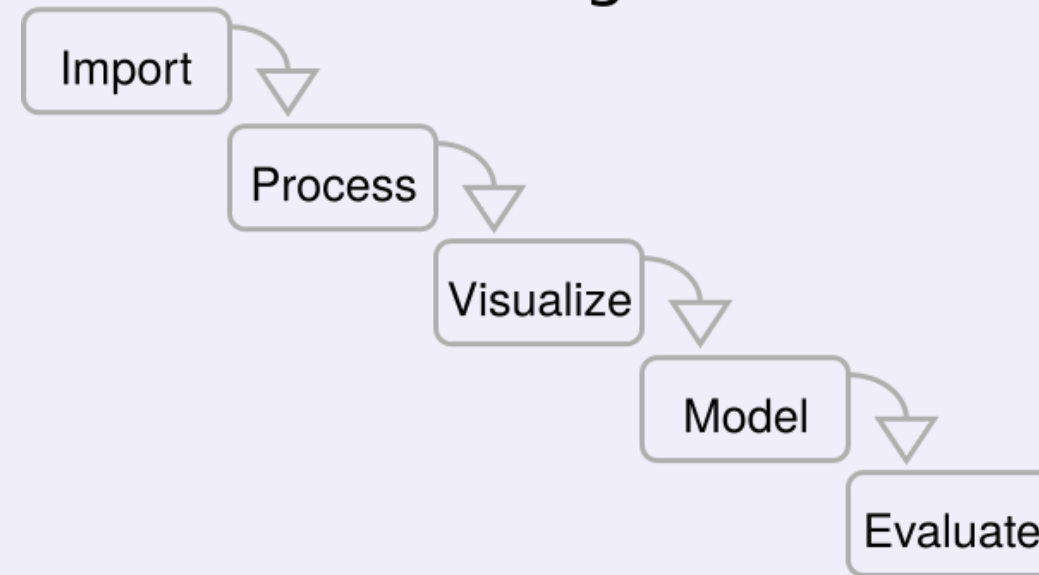


Move from continuous targets to discrete decisions with calibrated probabilities.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3 Classification Basics**
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Classification is a supervised learning task used to assign a new observation to a predefined category based on learned patterns from labeled data.

Machine learning workflow

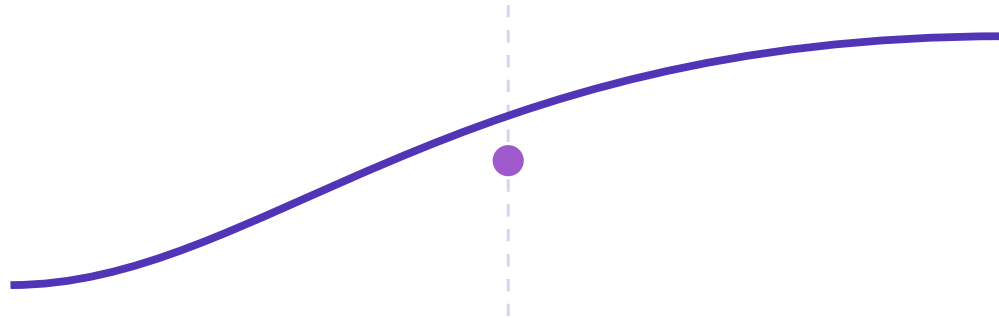


Output is a class label (e.g., 0/1, Yes/No).

Used in spam detection, medical diagnosis, fraud detection, and customer churn prediction.

● Logistic Regression: Definition

Logistic Regression is a supervised algorithm for binary classification that estimates the probability of class membership.



Produces probabilities in the range $[0, 1]$.

Final class is determined using a threshold (commonly 0.5).

Models the relationship between features and log-odds of the target.

Why Not Linear Regression for Classification?

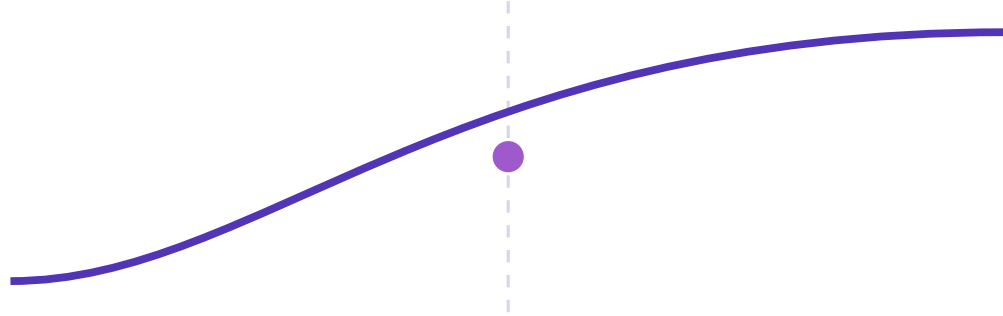
Linear regression predicts continuous values rather than discrete classes.

Predicted values can be less than 0 or greater than 1, which is invalid for probabilities.

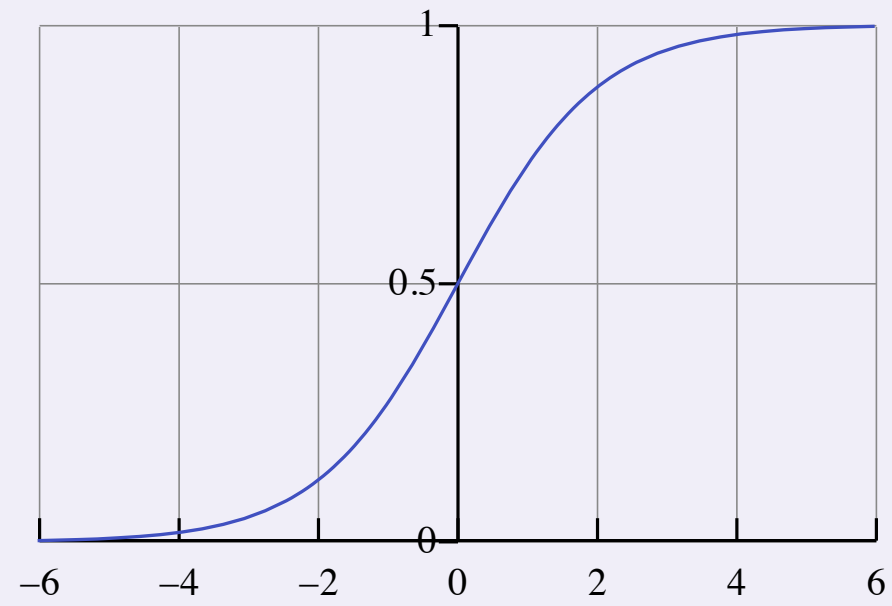
It does not naturally provide a robust classification boundary.

Logistic Regression solves this by mapping the linear score to probability through a sigmoid function.

● Sigmoid Mapping and Decision Threshold



$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad z = \beta_0 + \beta^T x$$

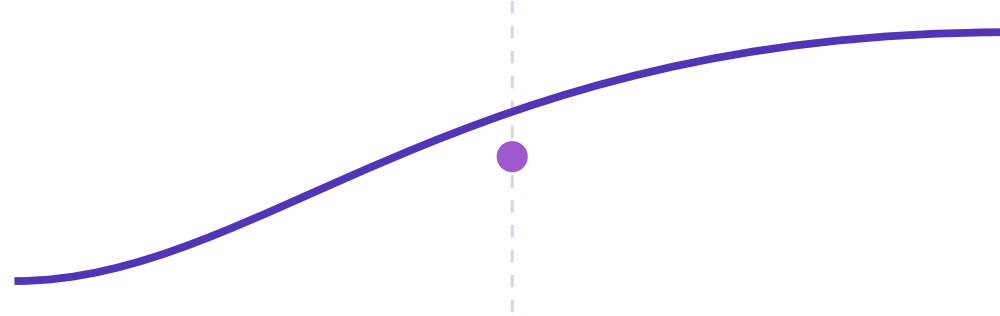


Sigmoid converts any real-valued score into a valid probability.

If probability ≥ 0.5 , predict class 1; otherwise class 0.

Threshold can be adjusted based on business/clinical needs.

● Key Assumptions for Logistic Regression



$$\text{Odds} = \frac{p}{1-p}, \quad \text{Logit}(p) = \log\left(\frac{p}{1-p}\right)$$

Observations should be independent.

Predictors should not have severe multicollinearity.

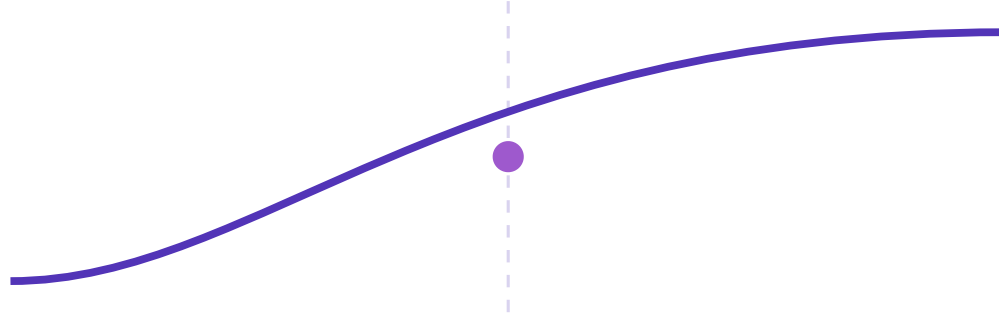
Relationship should be approximately linear in the log-odds space.

● Maximum Likelihood Estimation (MLE)

Logistic Regression parameters are estimated by maximizing the likelihood of observing the true class labels.

- Different curves correspond to different parameter values.
- The optimal model is the one with the highest likelihood (or lowest log-loss).

● Logistic Regression: Strengths and Limits



Strength: calibrated probabilities and clear feature coefficients.

Limit: linear decision boundary in feature space (use polynomials or other models for complex boundaries).

Strengths

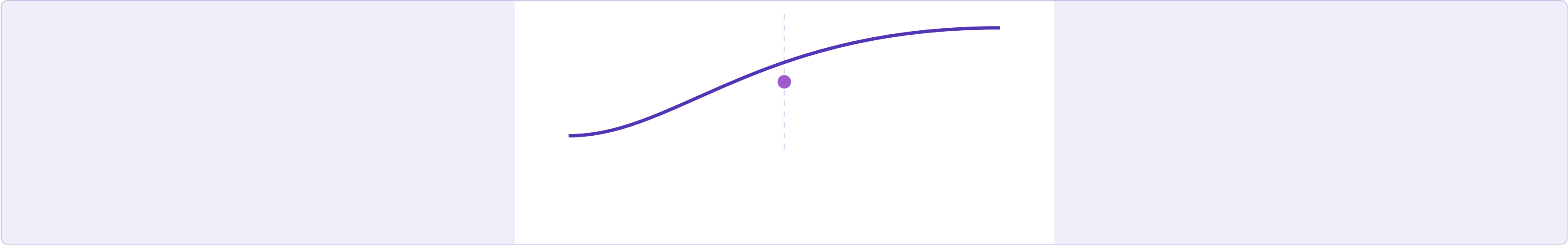
High interpretability of coefficients and odds ratios.

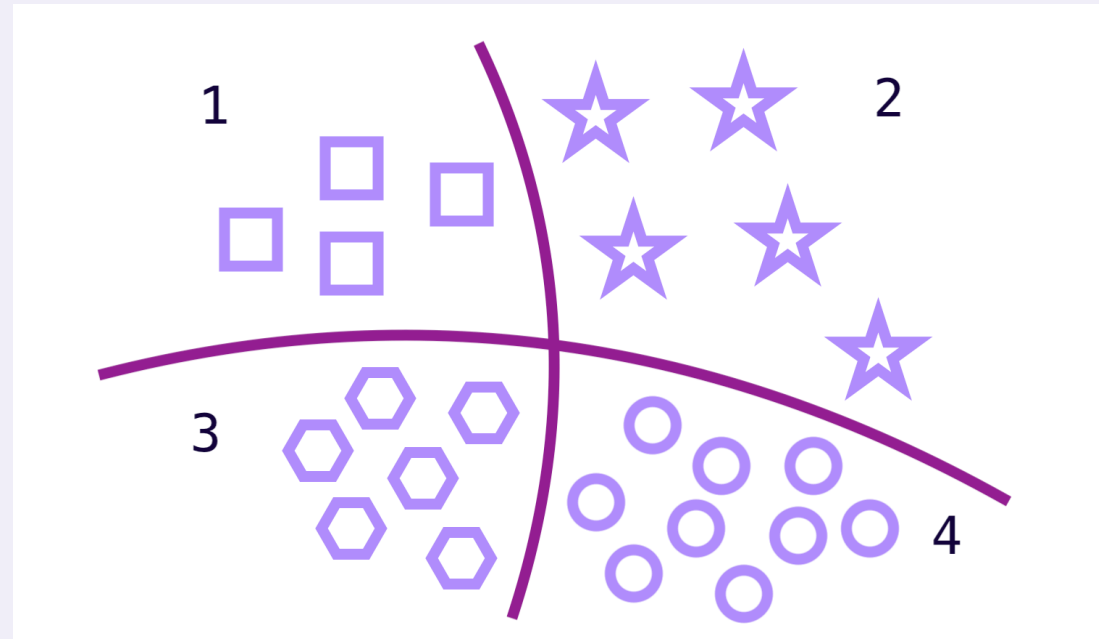
Fast training and strong baseline for binary tasks.

Limitations

May underperform on complex non-linear boundaries.

Sensitive when classes strongly overlap.





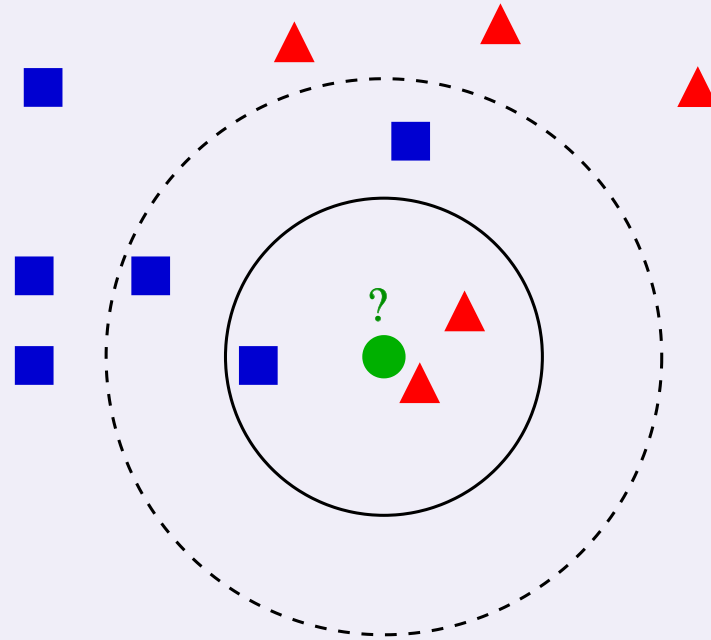
■ Multinomial (Softmax) Logistic Regression directly handles multiple classes.

■ One-vs-All (OvA) trains one binary classifier per class.

■ Final class is selected by highest predicted probability.

● K-Nearest Neighbors (K-NN): Core Idea

K-NN is an instance-based, non-parametric algorithm that classifies a sample using the majority class among its nearest neighbors.



- Choose K.
- Compute distance to all training points.
- Select K nearest points and apply majority voting.

K-NN: Choosing K and Distance Metric

Small K: lower bias, higher variance (sensitive to noise).

Large K: smoother boundary, may miss local structure.

Common metrics: Euclidean, Manhattan, Hamming.

Use cross-validation to choose K and scale numeric features before distance-based modeling.

K-NN: Strengths and Limitations

Strength: no training phase — lazy learning, simple baseline.

Limit: slow inference, curse of dimensionality, needs feature scaling.

Strengths

Simple and intuitive.

Effective on smaller, well-separated datasets.

Naturally supports multiclass classification.

Limitations

Prediction can be slow on large datasets.

Sensitive to irrelevant features and class imbalance.

Performance drops in high-dimensional spaces.

Section Takeaways

1. Logistic regression outputs probabilities — pick thresholds for the business cost.
2. K-NN is lazy learning: performance depends on scale and dimensionality.
3. Confusion matrix beats accuracy on imbalanced data.

Think: What threshold minimizes cost for your use case?

WEEK 1 · SESSION 4 · Classical ML

4

Naive Bayes, Trees, Evaluation

Naive Bayes

Decision trees

Random Forest

Precision / recall / F1

▸ 14 slides · ~22 min

Where Are We in the Bootcamp?

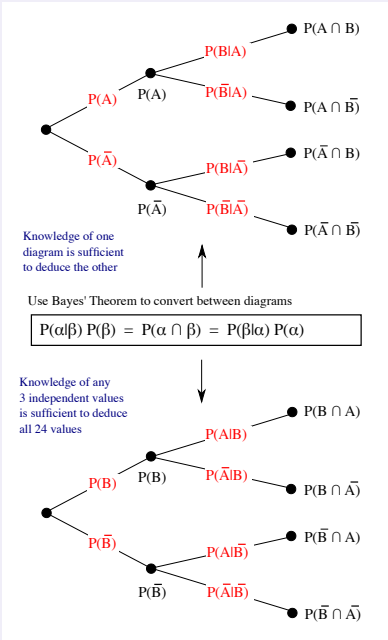


Combine probabilistic models with interpretable tree ensembles.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4 Naive Bayes & Trees**
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Core Probability Rule

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$



$P(A|B)$: posterior probability after observing evidence B.

$P(A)$: prior belief before seeing evidence.

$P(B|A)$: likelihood of evidence if class A is true.

$P(B)$: evidence probability (normalization term).

This theorem updates beliefs when new evidence appears.

$$\hat{y} = \arg \max_{c \in \mathcal{C}} P(c) \prod_{j=1}^m P(x_j \mid c)$$

Compute class posterior for each class: $P(\text{Class} \mid X)$.

Estimate prior and feature likelihoods from training data.

Compare posterior scores across classes.

Assign the class with the highest posterior probability.

For numerical stability, implementations often use log-probabilities.



Why Is It Called 'Naive'?

Conditional Independence Assumption

Naive Bayes assumes features are conditionally independent given the class label.

- | Simplifies parameter estimation and speeds up training.
- | Works well in text classification and high-dimensional sparse data.
- | Can still perform well even when independence is not fully true.

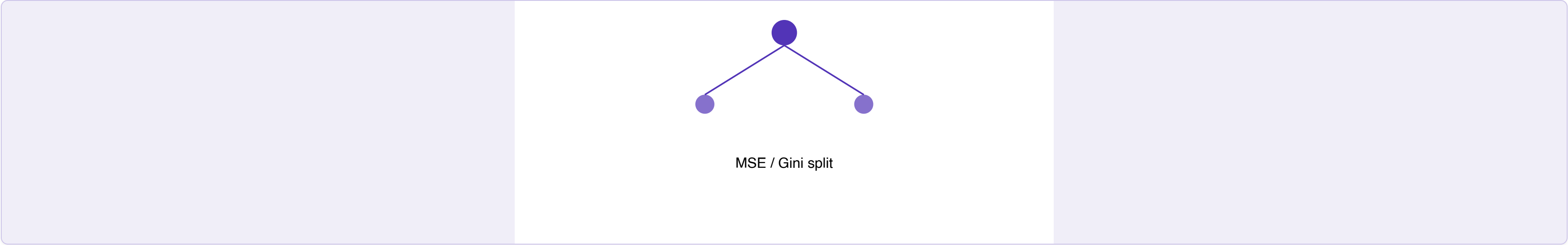
When comparing classes for the same X , $P(X)$ is constant and can be ignored.

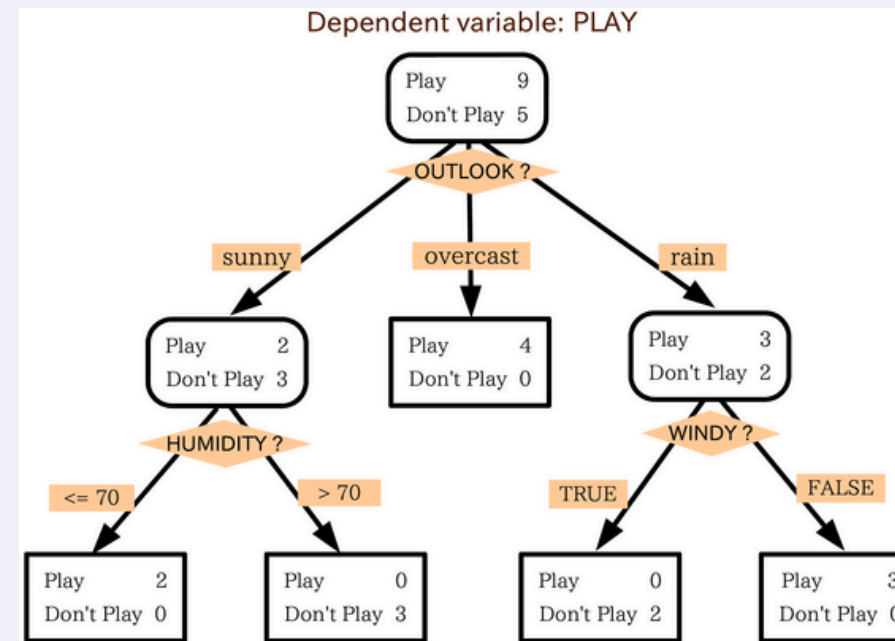
- Handle zero-frequency with smoothing (e.g., Laplace smoothing).
- Scale and preprocessing depend on feature type and variant.

Variant	Typical Data Type	Key Idea
Gaussian NB	Continuous	Assumes each feature follows a Gaussian distribution.
Multinomial NB	Count-based	Common for document word counts.
Bernoulli NB	Binary	Uses binary presence/absence features.

Decision Tree Classification (CART)

Decision Tree Classification predicts class labels by recursively splitting feature space into purer class regions.





Internal nodes represent feature-based decisions.

Branches represent decision outcomes.

Leaf nodes output final class prediction.

How Splits Are Chosen in Classification Trees

- Choose split that best separates classes at each node.
- Stop splitting using rules like max depth or minimum samples.
- Gini impurity and entropy measure node purity — lower is better.
- Splits are chosen greedily to maximize information gain.

Gini Impurity (CART default)

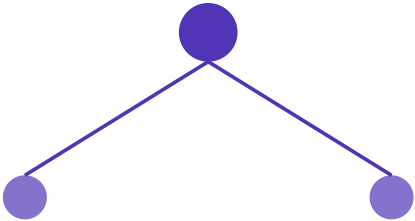
$$Gini(S) = 1 - \sum_{k=1}^K p_k^2$$

Entropy and Information Gain

$$Entropy(S) = - \sum_{k=1}^K p_k \log_2 p_k$$

Better splits reduce impurity and increase class purity in child nodes.

Decision Tree: Classification vs Regression



MSE / Gini split

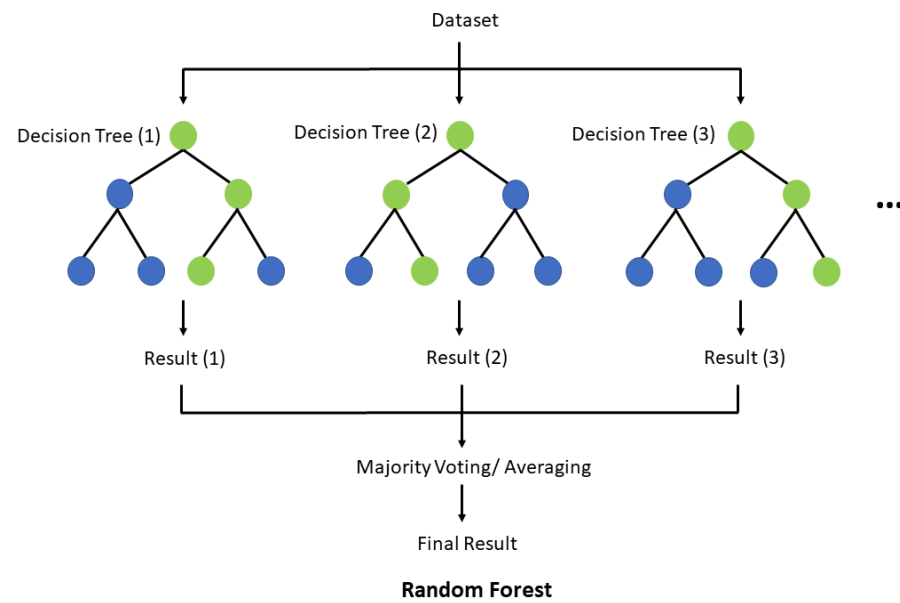
ASPECT	CLASSIFICATION TREE	REGRESSION TREE
Target	Categorical class	Continuous value
Split criterion	Gini / Entropy	MSE / MAE
Leaf output	Class label or class probability	Numeric mean/median

Both use recursive partitioning, but optimize different objectives.

Random Forest Classification

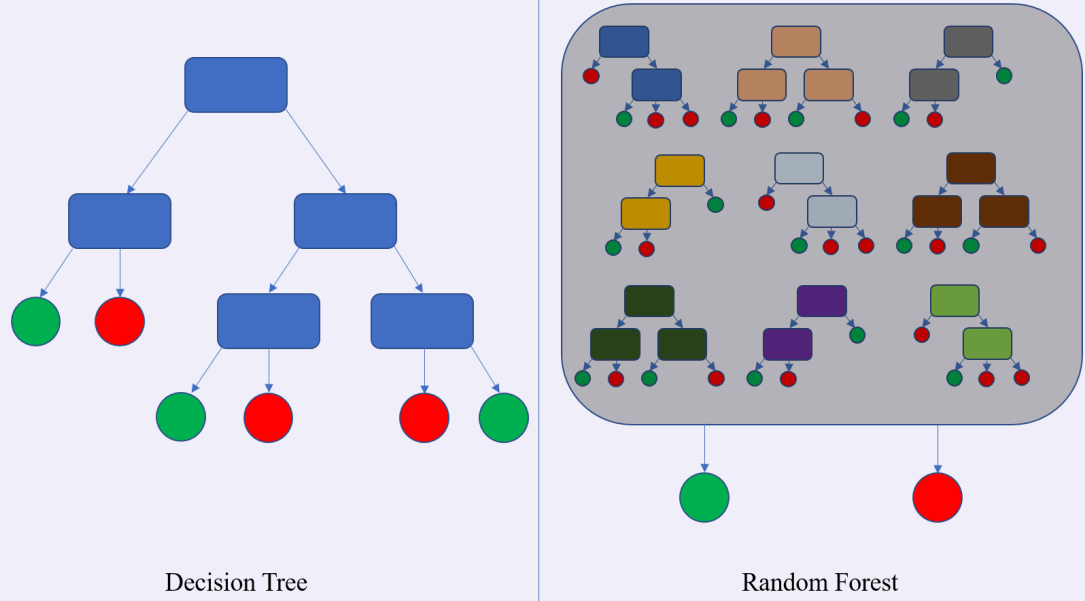
Ensemble Learning by Bagging

$$\hat{y}_{RF}(x) = \text{mode}\{\hat{y}_1(x), \hat{y}_2(x), \dots, \hat{y}_T(x)\}$$



- Sample bootstrap data and train many trees.
- Each tree predicts a class for the new sample.
- Final class is selected by majority voting.

Why Random Forest Often Outperforms a Single Tree



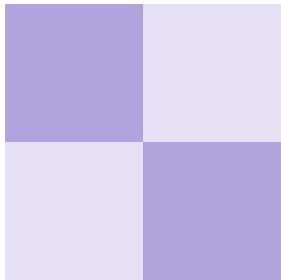
PROPERTY	SINGLE DECISION TREE	RANDOM FOREST
Variance	High	Lower (averaging effect)
Overfitting risk	Higher	Lower
Interpretability	High	Moderate
Predictive robustness	Sensitive to data noise	More stable

Random Forest improves generalization by combining diverse weak learners.

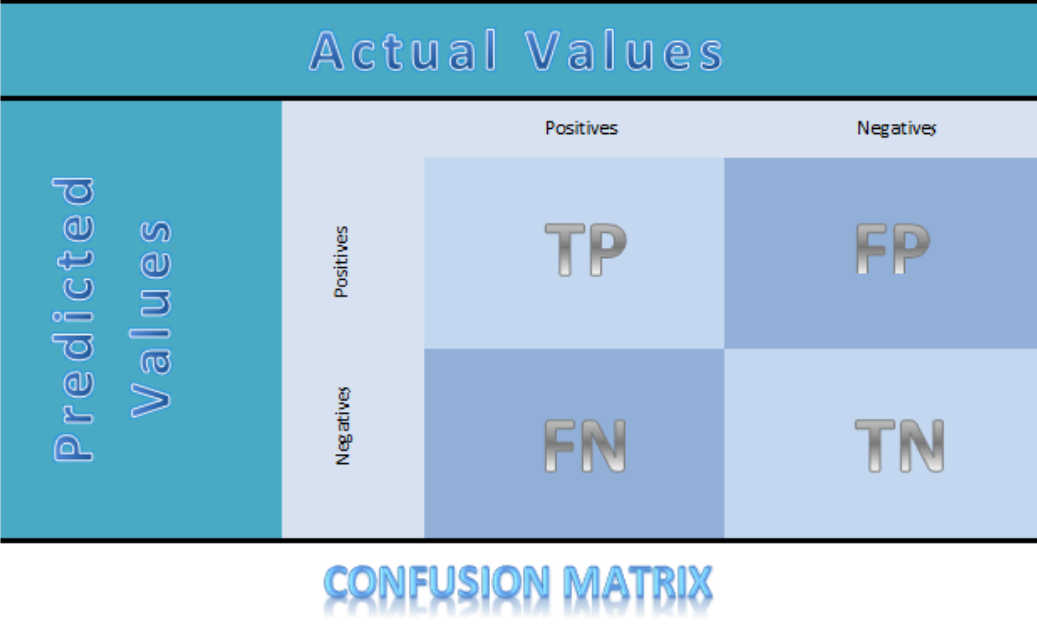
● Classification Errors: FP and FN

- False Positive (Type I): model predicts positive, but actual is negative.
- False Negative (Type II): model predicts negative, but actual is positive.
- Error impact depends on domain costs (e.g., healthcare vs spam filtering).

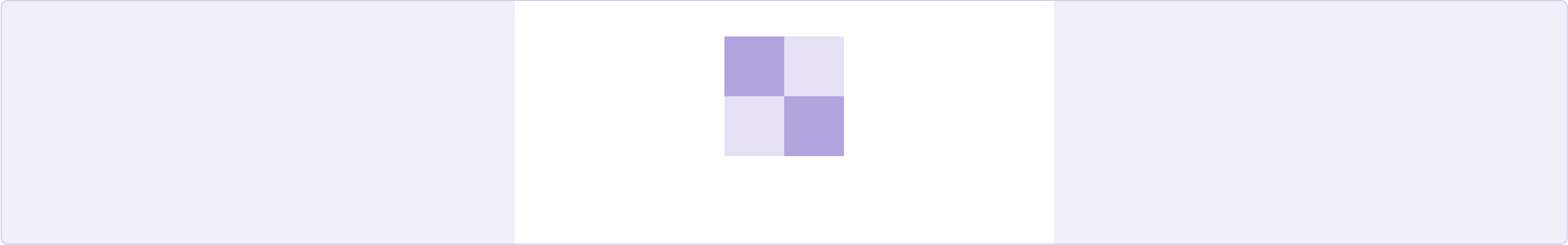
Model evaluation should consider error trade-offs, not accuracy alone.

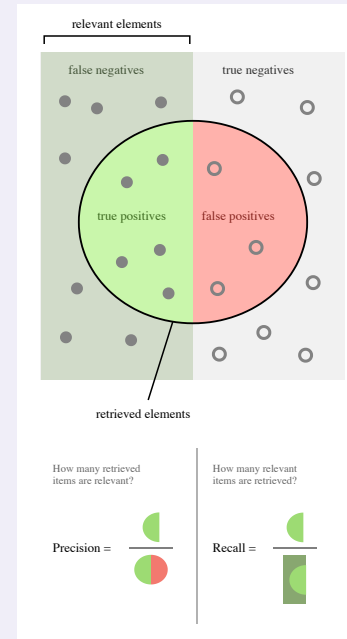


$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$



METRIC	MEANING
TP	Correctly predicted positives
TN	Correctly predicted negatives
FP	Incorrectly predicted positives
FN	Incorrectly predicted negatives





Precision: of predicted positives, how many are truly positive.

Recall: of actual positives, how many were detected.

F1-score: harmonic mean balancing precision and recall.

Formulas

$$Precision = \frac{TP}{TP + FP}, \quad Recall = \frac{TP}{TP + FN}$$

F1

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Use F1 especially when class distribution is imbalanced.

Section Takeaways

1. Naive Bayes assumes feature independence — fast baseline for text.
2. Decision trees overfit; Random Forests reduce variance via bagging.
3. Precision/recall/F1 reflect different failure costs.

Think: When is interpretability worth more than a few points of AUC?

WEEK 1 · SESSION 5 · Classical ML

5

SVM

Kernel Methods

Margin maximization

Kernel trick

Support vectors

Linear vs RBF

▸ 11 slides · ~18 min

Where Are We in the Bootcamp?

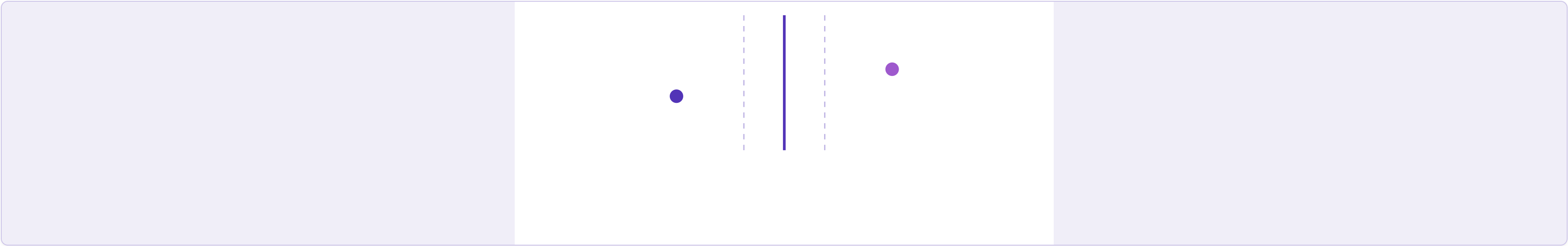


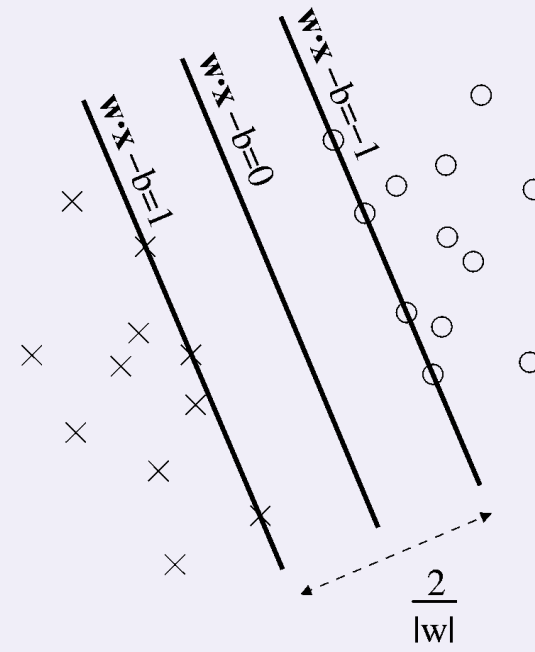
Find optimal margins and lift linear boundaries with kernels.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5 SVM & Kernels**
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Support Vector Machine (SVM): Core Idea

SVM is a supervised method that finds the best separating boundary (hyperplane) between classes.

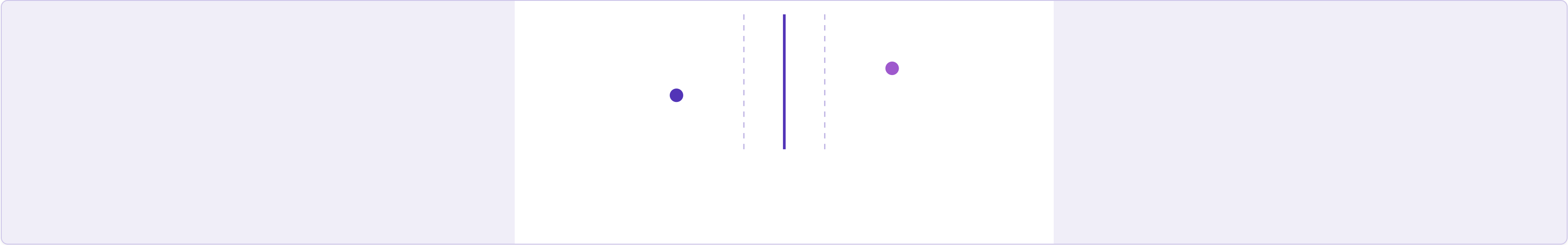




It maximizes the margin between classes.

Only support vectors (closest points) determine the boundary.

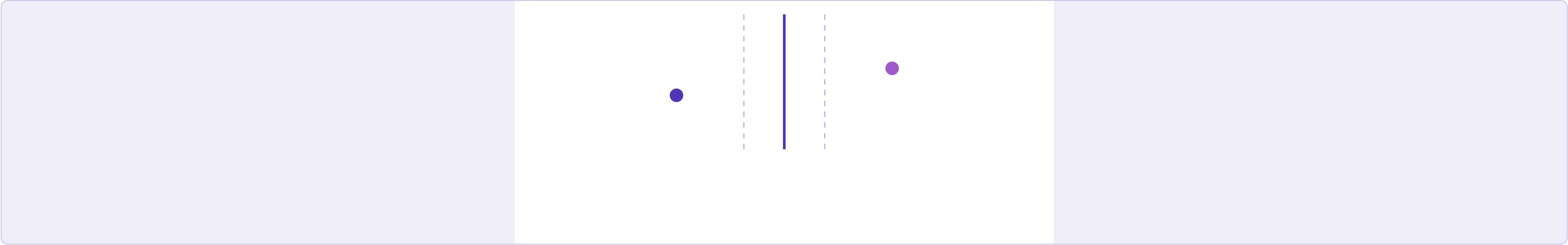
A larger margin usually improves generalization.



$$f(x) = w^T x + b, \quad \hat{y} = \text{sign}(f(x))$$

- New points are classified by the side of the hyperplane they fall on.
- Hard margin: no misclassification allowed (strict separation).
- Soft margin: allows violations for better robustness with overlap/noise.

Soft-margin SVM adds penalty C to control margin-violation tradeoff.

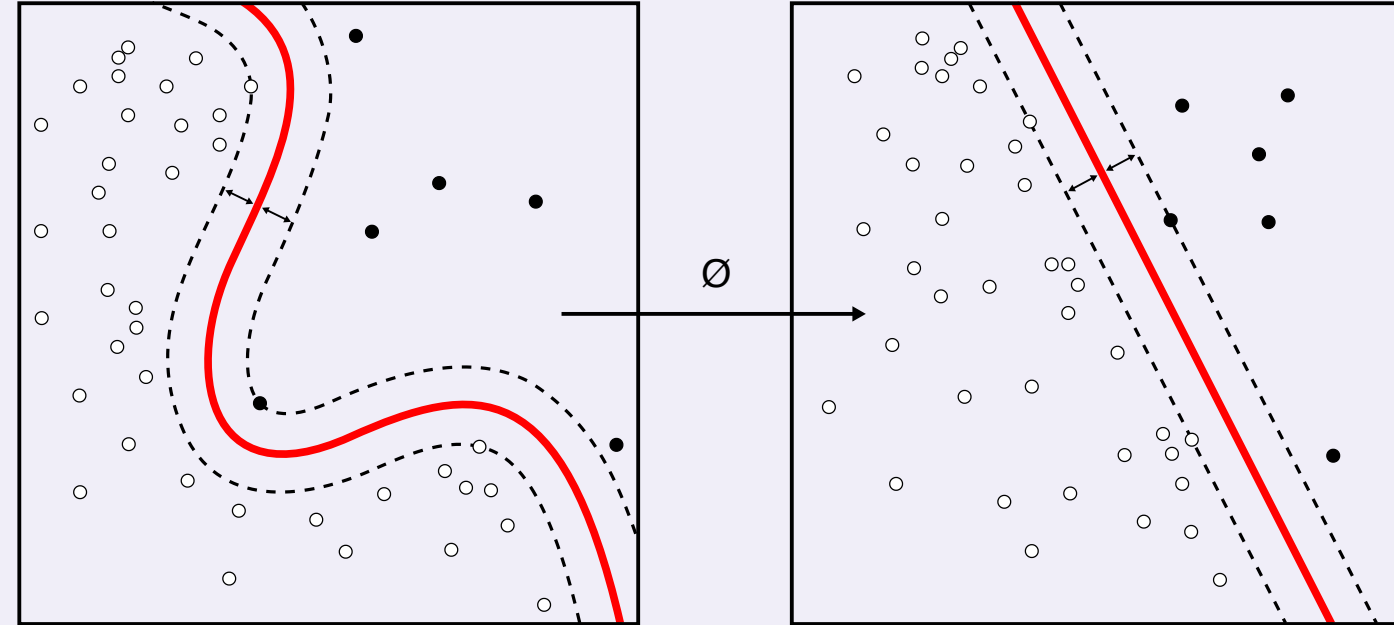


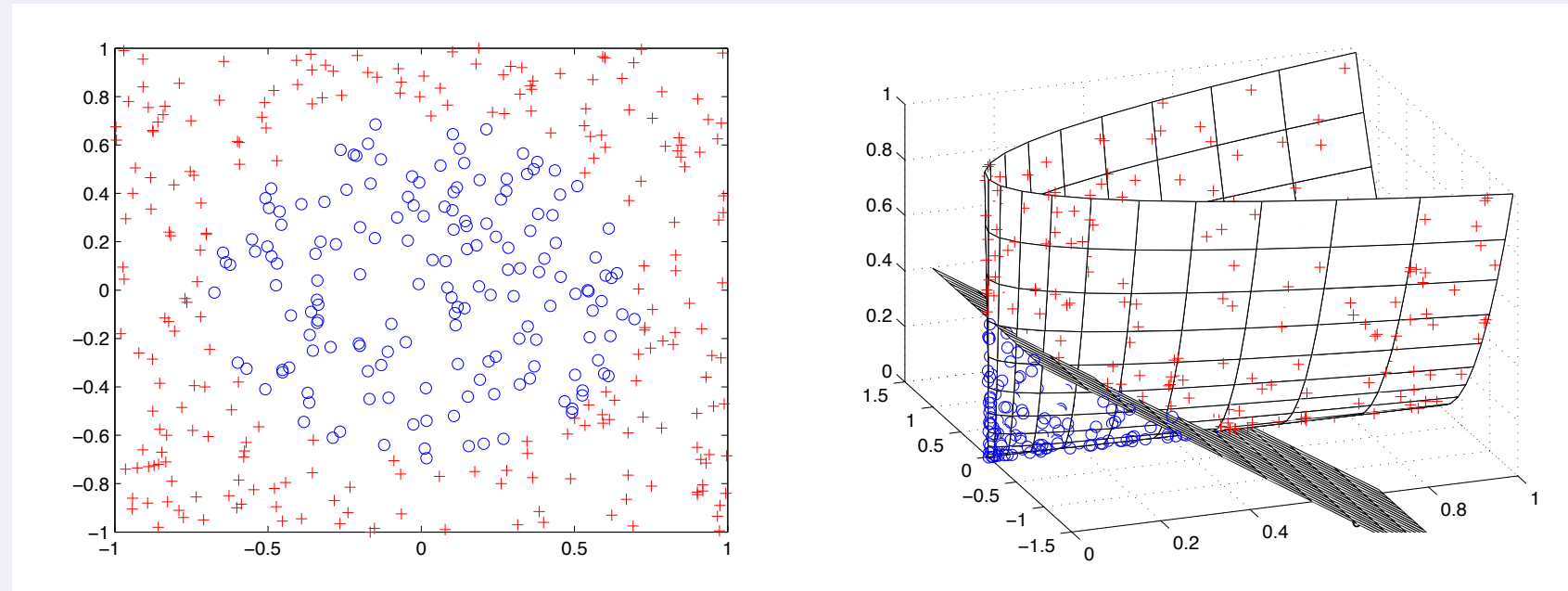
- Common in text classification and image-related features.
- Often effective on small-to-medium datasets.

CAPABILITY	WHY IT MATTERS
Maximum-margin principle	Better robustness to small perturbations.
Kernel support	Handles non-linear class boundaries.
High-dimensional performance	Works well in text and sparse feature spaces.

Kernel Trick: From Non-Linear to Linear Separation

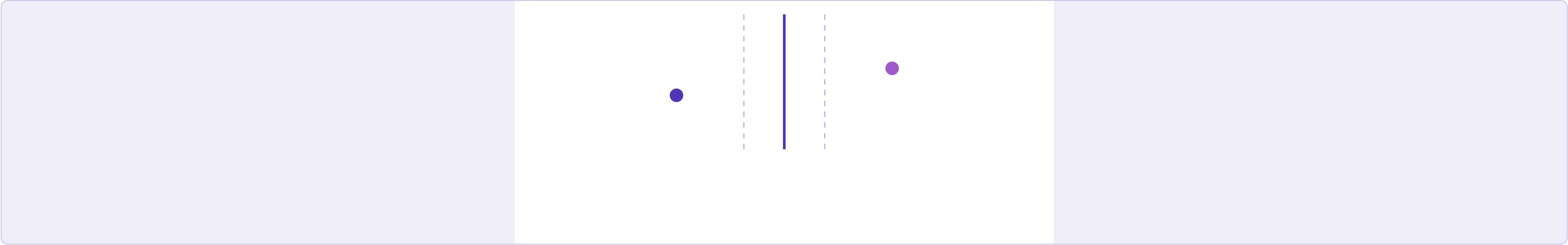
When data is not linearly separable in original space, SVM uses kernel functions to separate it in a transformed feature space.





Avoids explicit high-dimensional mapping in many cases.

Computes similarity using kernel function $K(x_i, x_j)$.



$$K(x, l_i) = \exp\left(-\frac{\|x - l_i\|^2}{2\sigma^2}\right)$$

If x is close to landmark l_i , similarity is near 1.

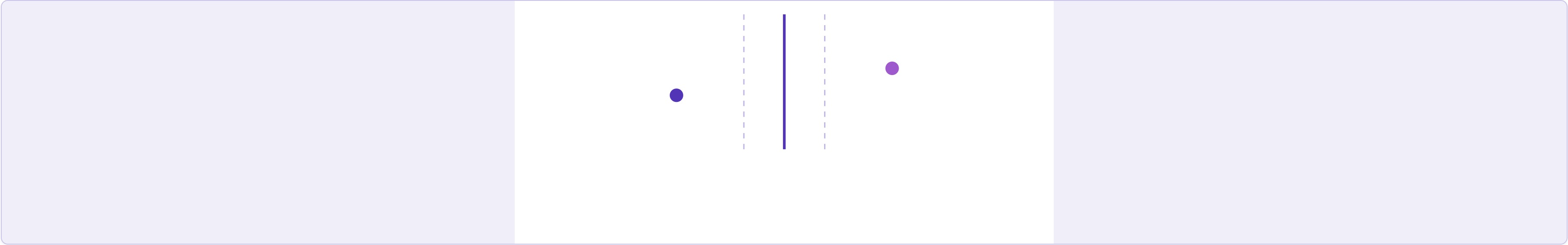
If x is far, similarity approaches 0.

RBF is a common default for non-linear SVM.

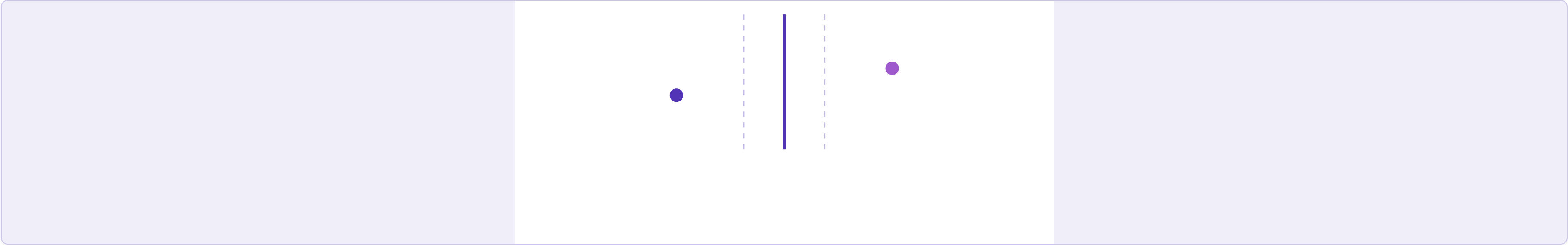
Effect of sigma

Large sigma: smoother, wider decision boundary (higher bias).

Small sigma: tighter, complex boundary (higher variance/overfitting risk).



Kernel	Typical Use	Notes
Linear	High-dimensional sparse data	Fast and interpretable margin.
RBF (Gaussian)	General non-linear patterns	Strong baseline in many tasks.
Polynomial	Polynomial-like interactions	Degree controls complexity.
Sigmoid	Neural-style boundary behavior	Less common in practice.



Use: medium-sized data, clear margin, high-dimensional text.

Avoid: massive datasets, heavy noise without kernel tuning.

Use SVM When

| Data is high-dimensional or moderately sized.

| Classes are reasonably separable.

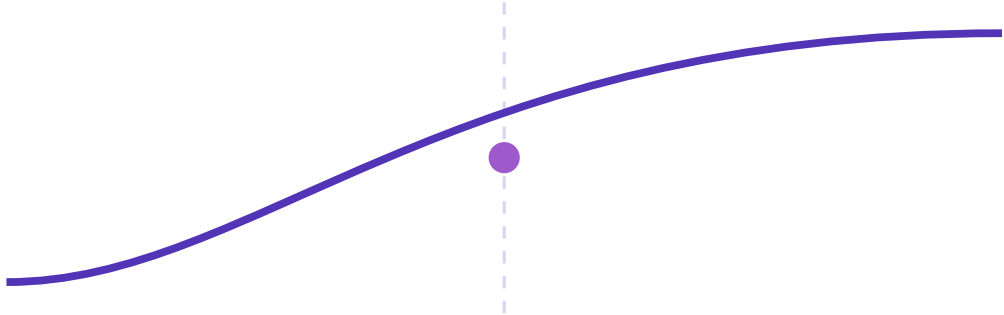
| You need non-linear boundaries via kernels.

Avoid SVM When

| Dataset is very large (training can be expensive).

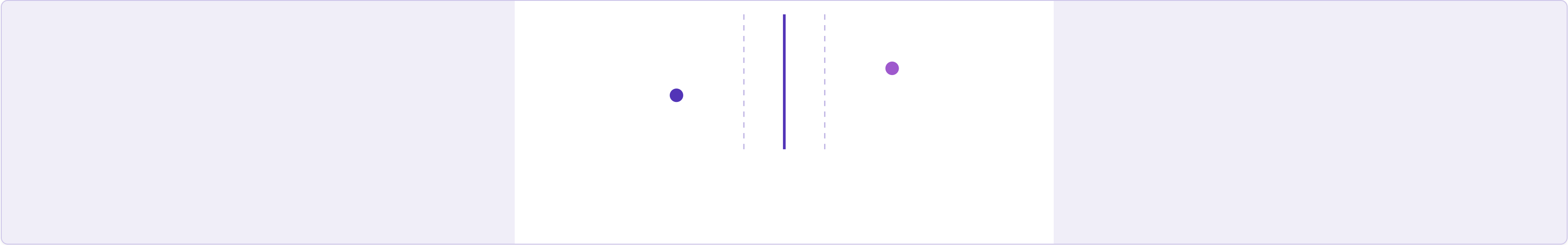
| Data is extremely noisy with weak class structure.

| Feature count and sample size make kernel matrix too costly.



ASPECT	SVM	LOGISTIC REGRESSION	K-NN
Primary task	Classification (and SVR)	Classification	Classification
Non-linearity	Yes, with kernels	Limited in linear form	Yes (distance-based)
Compute profile	Can be heavy on large data	Usually efficient	Heavy at prediction
Best for	Complex margins, high-dimensional spaces	Fast interpretable baseline	Local neighborhood patterns

Model choice should be validated with cross-validation and error tradeoff metrics.



PARAMETER	ROLE	TYPICAL TUNING DIRECTION
C	Penalty for margin violations	Higher C -> stricter fit, lower C -> smoother margin
Kernel	Similarity function	Start with RBF, compare with linear/polynomial
gamma (RBF)	Locality of influence	Higher gamma -> tighter boundary
degree (poly)	Polynomial complexity	Increase only when needed

Scale features before SVM, especially for RBF/polynomial kernels.

Section Takeaways

1. SVM maximizes margin — support vectors define the boundary.
2. Kernel trick maps features implicitly to higher dimensions.
3. SVMs shine on medium data with clear margin structure.

Think: Why might an RBF kernel overfit on tiny datasets?

Clustering

PCA

K-Means Hierarchical clustering PCA Silhouette & elbow

▸ 11 slides · ~18 min

Where Are We in the Bootcamp?



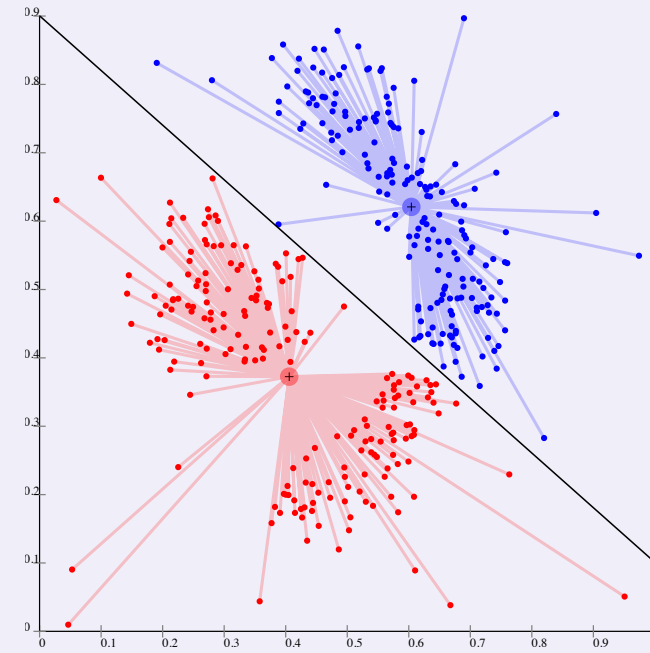
Discover structure without labels and compress high-dimensional views.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6 Clustering & PCA**
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Clustering Overview

Unsupervised Learning for Structure Discovery

ETRA



Clustering groups similar observations without target labels.

It is used for customer segmentation, anomaly discovery, and data exploration.

Cluster quality depends on feature scaling, distance metric, and algorithm assumptions.

In unsupervised settings, evaluation combines metrics with domain interpretation.

K-Means Clustering: Core Idea

Partition Data into K Compact Groups



$$J = \sum_{i=1}^K \sum_{x \in C_i} \|x - \mu_i\|^2$$

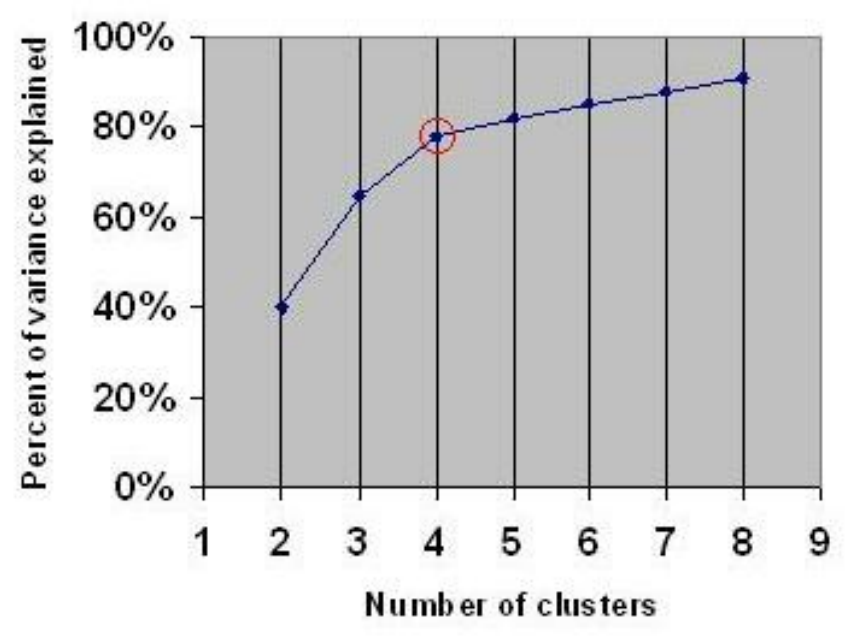
Initialize K centroids, assign each point to the nearest centroid, then update centroids.

Repeat assignment and update until cluster memberships stabilize.

K-Means favors spherical, similarly sized clusters in Euclidean space.

The objective minimizes within-cluster variance (WCSS/inertia).

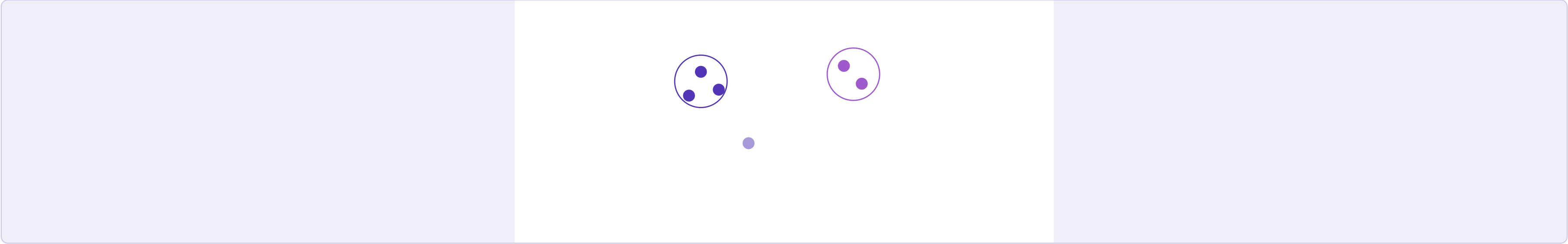
Selecting K: Elbow and Silhouette

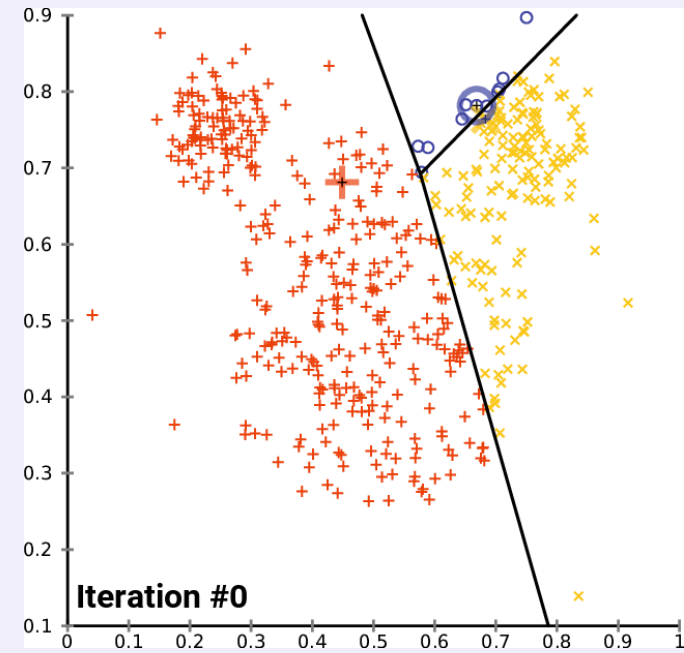


METHOD	WHAT TO INSPECT	INTERPRETATION
Elbow	WCSS vs number of clusters	Choose the knee where improvement starts diminishing
Silhouette	Mean silhouette score in [-1, 1]	Higher values indicate better separation and compactness

Use both metrics and business context; the mathematically best K is not always operationally best.

A Better Start than Pure Random Seeds



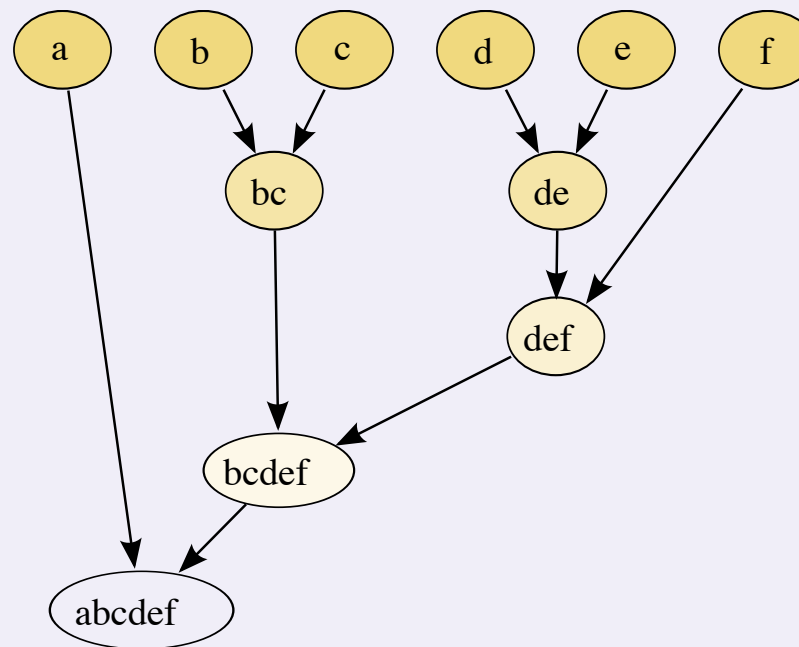


Pick the first centroid randomly from data points.

Choose subsequent centroids with probability proportional to squared distance (D^2).

Then run standard K-Means iterations.

K-Means++ reduces poor initialization risk and often converges to lower inertia.

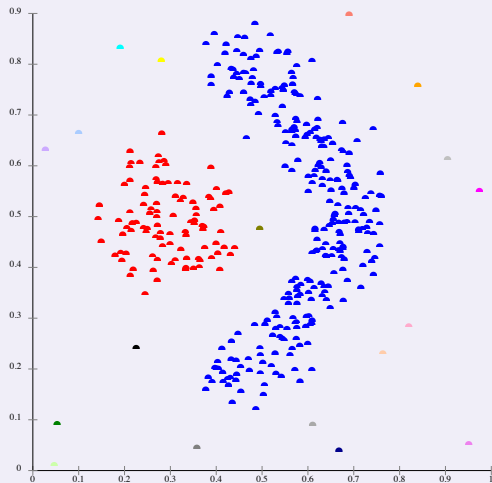


Agglomerative clustering starts with one point per cluster and merges progressively.

Divisive clustering starts with one global cluster and splits recursively.

The dendrogram stores merge history and supports multi-resolution analysis.

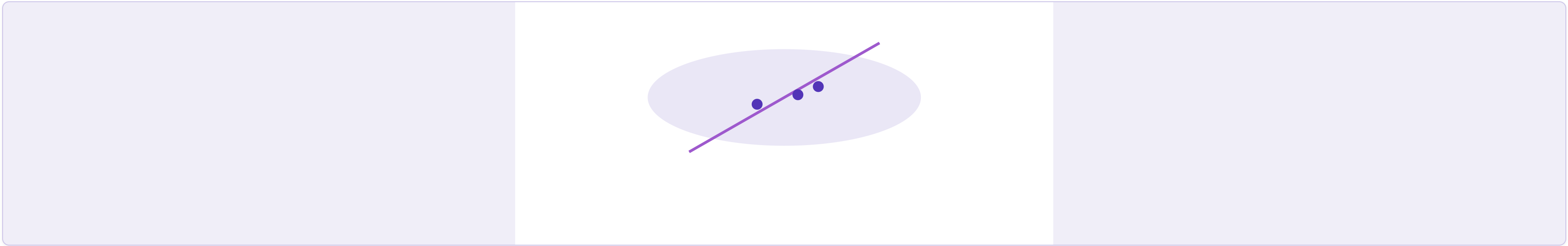
Unlike K-Means, hierarchical methods can reveal nested structure in data.



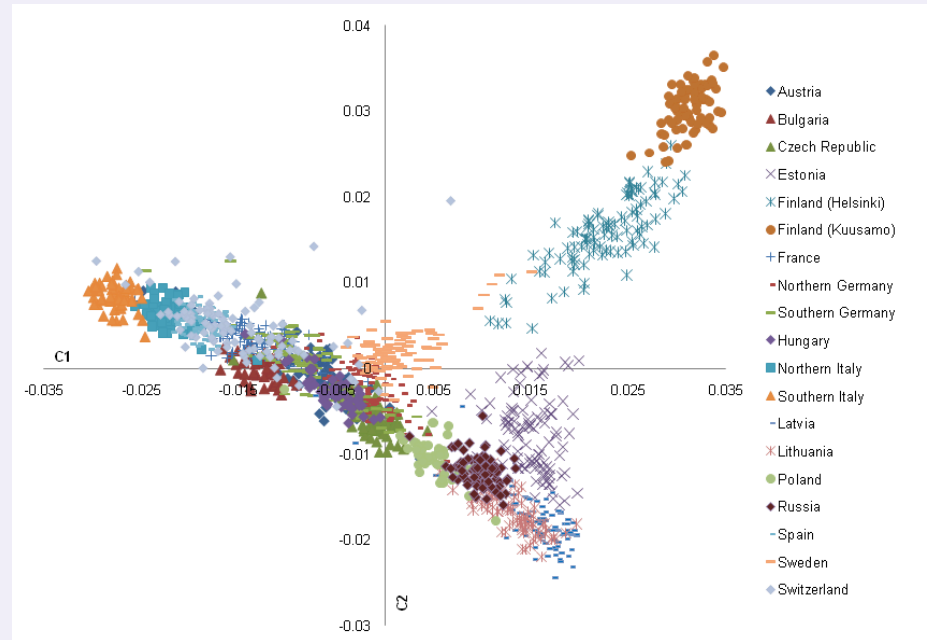
Linkage	Distance between clusters	Typical behavior
Single	Minimum pairwise distance	Can chain elongated clusters
Complete	Maximum pairwise distance	Produces compact, tighter clusters
Average	Mean pairwise distance	Balanced tradeoff between single/complete
Ward	Increase in within-cluster variance	Often forms homogeneous clusters

Choose a dendrogram cut threshold where vertical gaps are most pronounced.

Project Data onto High-Variance Directions



$$\Sigma = \frac{1}{n-1} X^\top X, \quad \Sigma v_j = \lambda_j v_j$$

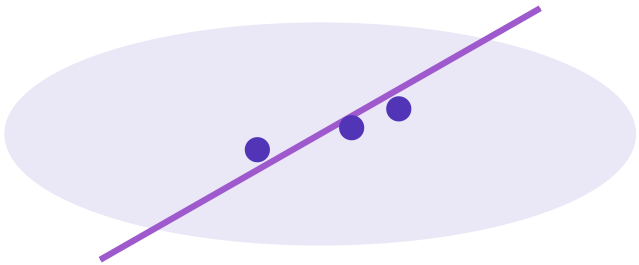


PCA transforms correlated features into orthogonal principal components.

Components are ordered by explained variance (largest eigenvalues first).

Keeping top components reduces noise and computational cost.

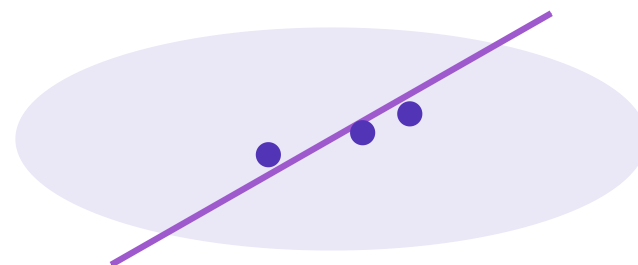
Standardize features before PCA when original scales differ substantially.



Explained Variance Ratio_j = $\frac{\lambda_j}{\sum_{m=1}^p \lambda_m}$

STEP	ACTION	OUTCOME
1	Standardize features	Comparable feature scales
2	Compute covariance matrix and eigenpairs	Ranked principal directions
3	Select top-k components	Compressed feature representation
4	Project data onto selected components	Lower-dimensional inputs

Select k using cumulative explained variance (e.g., 90-95%) and downstream model performance.



Scale features, run PCA, then cluster in reduced space.

Use 2D/3D PCA projections to visually inspect cluster separability.

Compare multiple algorithms (K-Means, Hierarchical, DBSCAN) before finalizing.

Validate with silhouette score, stability checks, and domain plausibility.

Combining PCA with clustering often improves interpretability and runtime on high-dimensional data.

Section Takeaways

1. Clustering needs domain sense — algorithms optimize geometry, not meaning.
2. PCA finds variance directions; scale features before applying it.
3. Elbow/silhouette guide k — always validate clusters qualitatively.

Think: How would you validate clusters with no ground truth?

WEEK 2 · SESSION 1 · Deep Learning

7

Deep Learning Neural Networks

Perceptron & MLP

Backpropagation

CNNs

RNNs & regularization

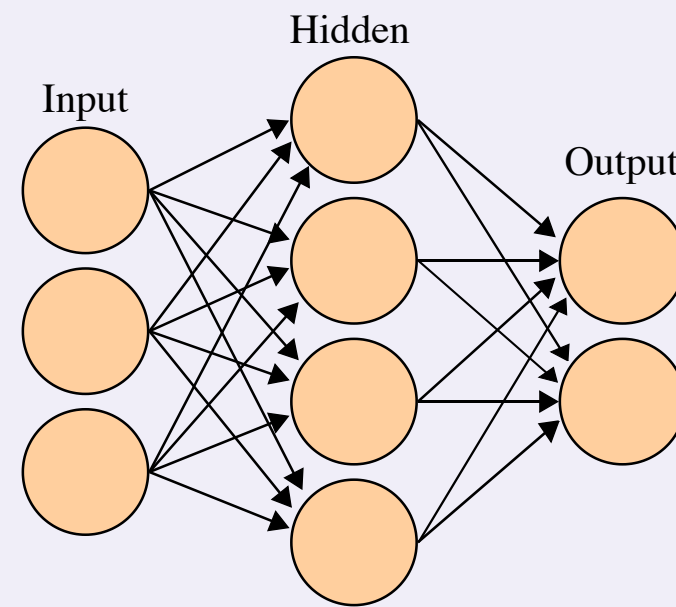
▸ 30 slides · ~48 min

Where Are We in the Bootcamp?

Stack layers to learn hierarchical representations from data.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7 Deep Learning**
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Each phase answers a different question: Phase 1 — why depth and what a neuron actually computes; Phase 2 — how training stays stable and generalizes; Phase 3 — which architecture matches images vs sequences; Phase 4 — generation and reconstruction.



Four phases: foundations → optimization → architectures → autoencoders.

Each phase has a lab-style checkpoint — pause at phase boundaries.

Focus on deep learning basics — phase breaks are intentional.

Phase 1 focus: understand the “why” and the mathematical mechanics of a single neuron before scaling to layers and data.

Phase 1: Foundations & Neural Core

Why depth, neurons, and forward pass

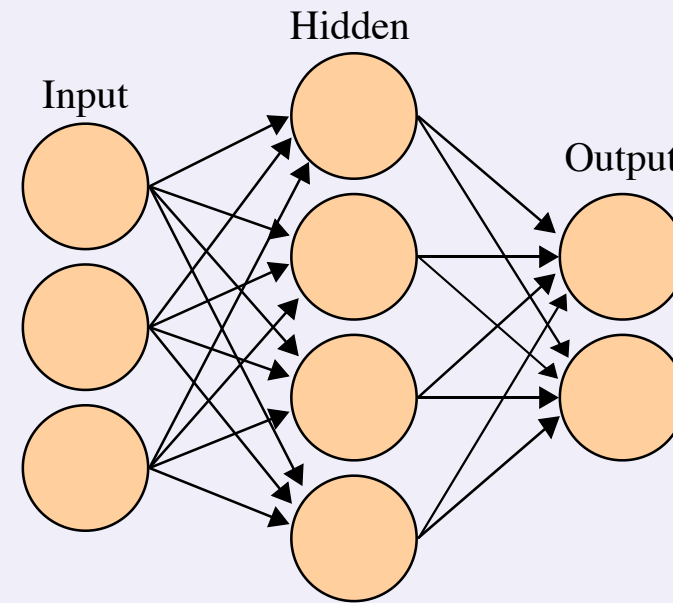
- Goal: understand what a neuron computes and why stacking layers enables hierarchical features.
- Topics: perceptron, activations, MLP, forward propagation, and loss functions.
- Exit check: explain $z = w \cdot x + b$ and why nonlinearity is required between layers.

Phase 1 roadmap

BLOCK	QUESTION IT ANSWERS
Perceptron & MLP	What does one layer compute?
Forward pass	How does inference flow through the graph?
Loss & gradients	What are we optimizing?

What Is Deep Learning?

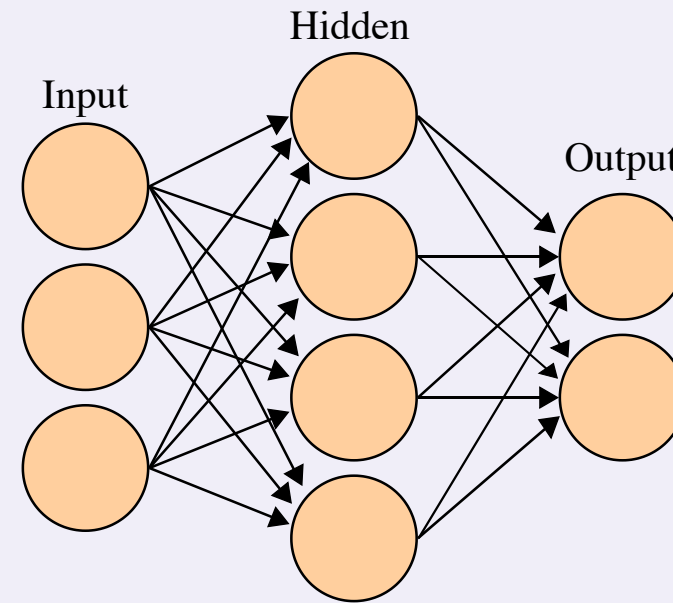
Definition in One Sentence — Then the Details



- Deep learning learns a parameterized mapping from inputs to outputs by stacking nonlinear transformations (layers). “Deep” means multiple such transformations—hierarchical features, not one giant linear rule.
- Training = pick a loss that measures mistakes, then adjust weights with gradients so average loss drops on data (generalization to unseen data is the real goal).
- Unlike fixed pipelines of hand-crafted features, the network learns representations that are tuned to the task (edges → textures → parts → objects in vision, etc.).

Three ingredients you always have

INGREDIENT	WHAT IT MEANS
Architecture	How signals flow (MLP, CNN, RNN, Transformer, ...)
Loss / objective	What “better” means for your task (MSE, cross-entropy, ...)
Optimizer + data	How you search parameters and what examples you show



Artificial intelligence (AI): the broad field—rules, search, optimization, learning, planning—anything that aims at intelligent behavior.

Machine learning (ML): a subfield of AI where behavior improves from data via a learning algorithm (not only hand-written rules).

Deep learning (DL): a subfield of ML using deep neural networks—multiple learned nonlinear layers; representation learning is central.

Containment (conceptual)

SET	CONTAINS
AI	Rule systems, classical search, ML, knowledge bases, ...
ML	Linear models, trees, kernel SVMs, shallow nets, deep nets
DL	CNNs, RNNs/GRU/LSTM, Transformers, GANs, VAEs, ...

Saying “we used AI” is vague; “we used supervised DL with a CNN” is precise.

Use-Case Fit: When Deep Learning Shines — and When to Skip It

Match Method to Data, Risk, and Budget

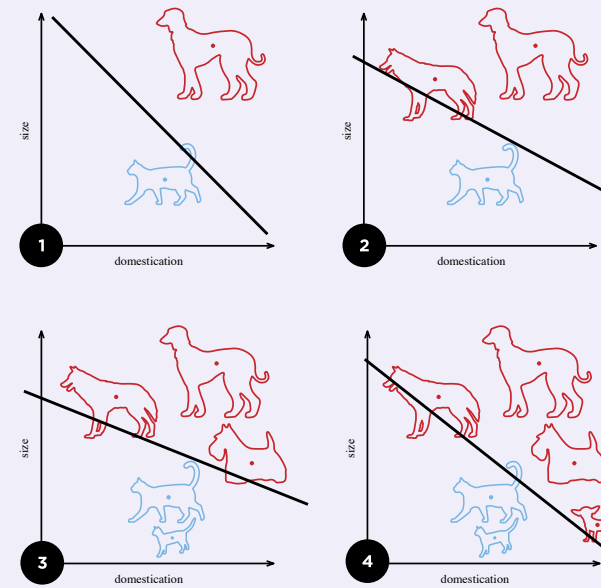
- DL is strongest when: large data (or strong pre-training), high-dimensional raw inputs (images, text, speech), and compositionality you want learned end-to-end.
- Think twice when: tiny tabular datasets, strict interpretability requirements, tight latency on CPU without optimization, or simple baselines already solve the problem.
- Always compare against strong baselines (logistic regression, gradient boosting, k-NN on engineered features)—DL must earn its complexity on validation metrics you care about.

SITUATION	TYPICAL FIRST CHOICE	WHY
Large labeled images / audio / text	Pre-trained DL + fine-tune	Raw inputs; hierarchical features
Small tabular (< few k rows)	Trees / linear + regularization	Lower variance, fast iteration, interpretability
Need causal or legal explanation	Simpler model or hybrid	Auditability and stability
Plenty of GPUs + big data	Larger nets + careful tuning	Capacity + regularization can scale
Edge device, millisecond latency	Small distilled / quantized models	Memory and power limits

Single Neuron (Perceptron): Math, Weights, and Bias

One Affine Map + One Nonlinearity

$$z = w^\top x + b, \quad a = f(z), \quad \hat{y} = a \text{ (or further processing)}$$



Affine part (see formula above): z is the weighted sum of inputs plus b —each x_i is multiplied by w_i . The bias b shifts the decision boundary (same orientation, normal w); it does not rotate the separating hyperplane.

Weights encode which patterns excite the neuron; training adjusts w and b so that on labeled data the loss decreases.

With no hidden layer and a monotonic activation, the decision boundary in input space is linear (a hyperplane)—powerful but limited.

SYMBOL	SHAPE INTUITION	ROLE
x	Feature vector (length d)	One training example's inputs
w	Same length as x	Direction + strength of sensitivity to each feature
b	Scalar	Offset / threshold in logit space
f	Maps $\mathbb{R} \rightarrow \mathbb{R}$ (per neuron)	Introduces nonlinearity (or identity)

Weight Initialization: Why Zeros Break Training

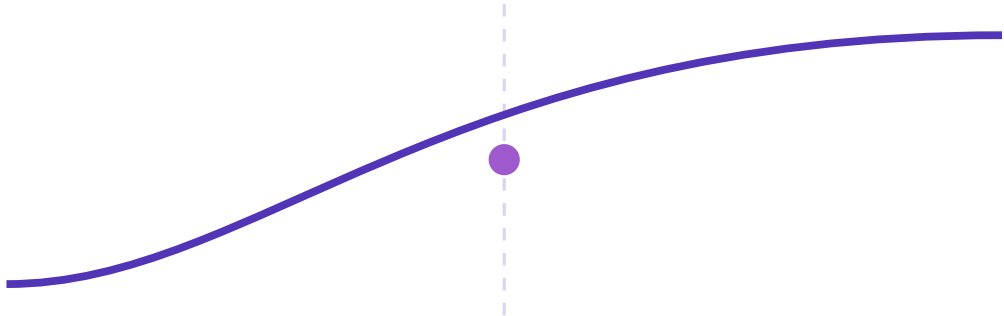
Symmetry Must Be Broken

- If all weights in a layer start equal (especially zero), every hidden unit receives the same gradient and stays identical forever—symmetry never breaks.
- Random initialization makes each unit walk a different path in parameter space so they can specialize as feature detectors.
- Frameworks use formulas like He (Kaiming) for ReLU: $\text{variance} = 2/n_i n$, and Xavier (Glorot) for Tanh: $\text{variance} = 2/(n_i n + n_{out})$ to keep activation scale stable.

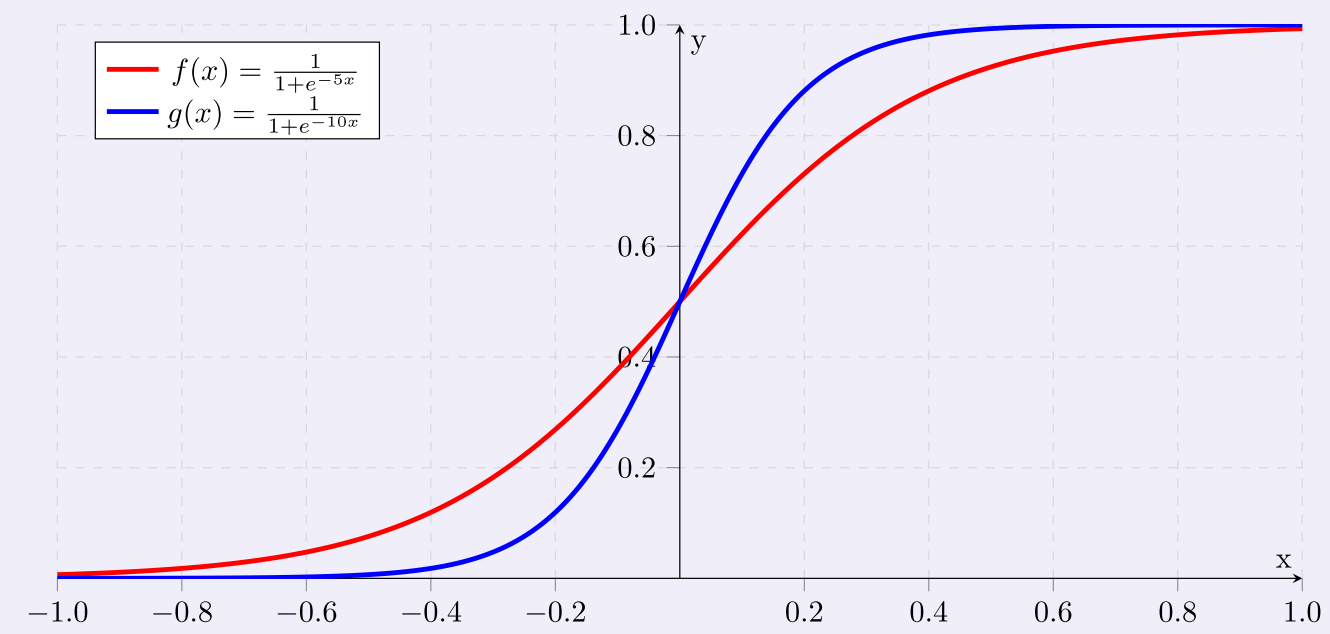
INIT SCHEME	PROBLEM IT AVOIDS	BEST SUITED FOR
All zeros	Hidden units stay identical; no representation diversity	None (always avoid for weights)
Xavier (Glorot)	Vanishing or exploding variance across layers	Sigmoid / Tanh activations
He (Kaiming)	Variance drop due to ReLU zeroing out half the signals	ReLU / Leaky ReLU activations

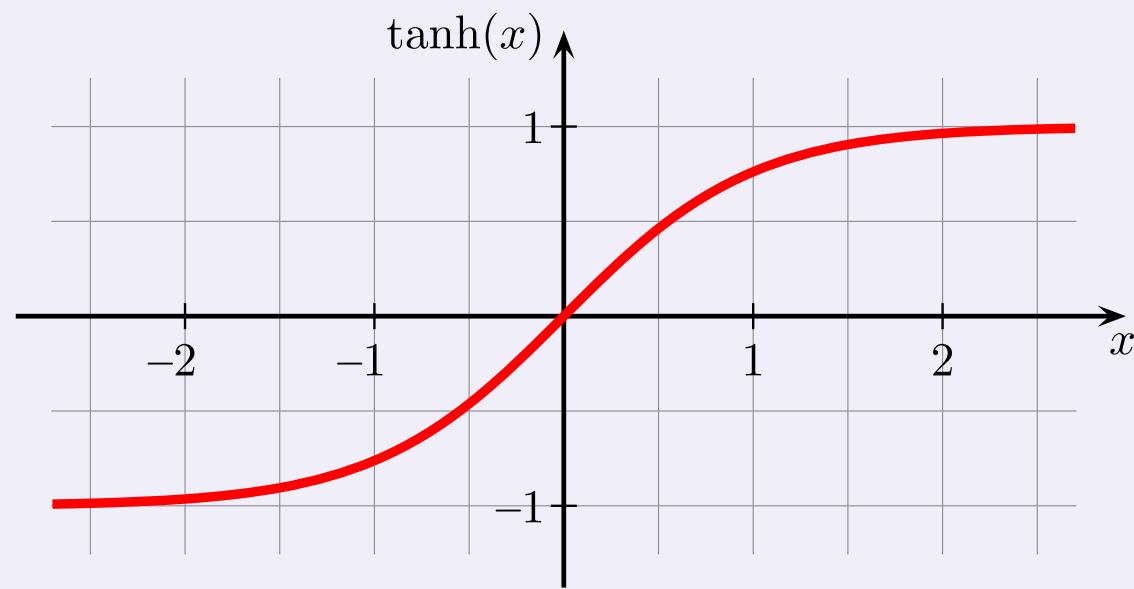
● Activation Functions: Sigmoid, Tanh, and ReLU

Nonlinearity is the Key to Network Depth



$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \quad \text{ReLU}(z) = \max(0, z)$$





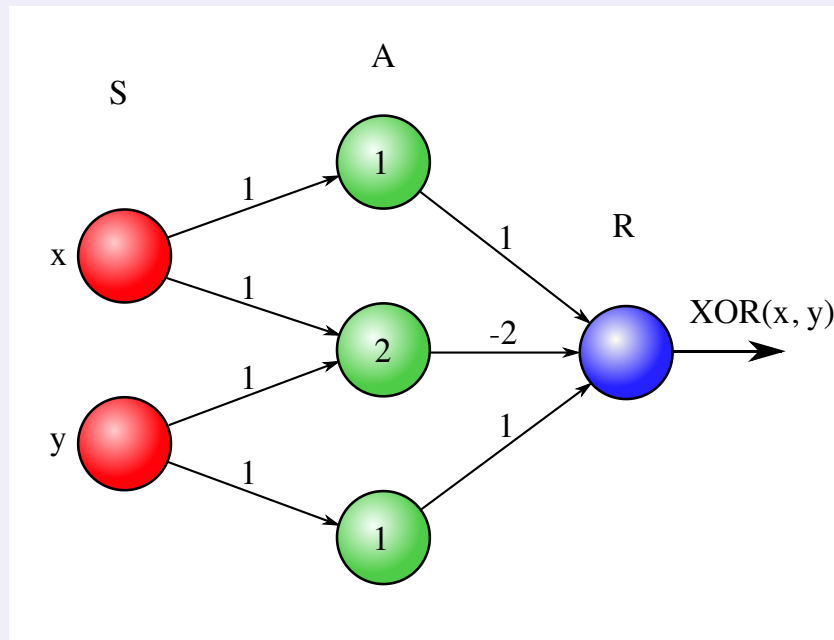
Sigmoid: maps inputs to $(0, 1)$ —useful for probabilities, but suffers from vanishing gradients when z is very large or very small (saturation).

Tanh: zero-centered, maps inputs to $(-1, 1)$ —often nicer than sigmoid for hidden layers but still saturates.

ReLU: modern default for deep stacks—cheap to compute, avoids vanishing gradient on positive domain; leaky ReLU / ELU are variants.

FUNCTION	RANGE	PRIMARY USE-CASE
Sigmoid	(0, 1)	Binary classification output layer, gating networks
Tanh	(-1, 1)	Hidden layers (recurrent networks)
ReLU	[0, ∞)	Default choice for hidden layers in MLPs and CNNs

● The XOR Problem: Why Hidden Layers Are Logically Necessary



XOR is not linearly separable in the 2D input plane: no single line separates class 1 from class 0 on the four corners.

A single perceptron computes a half-space; XOR requires nonlinear mixing of inputs—exactly what a hidden layer provides.

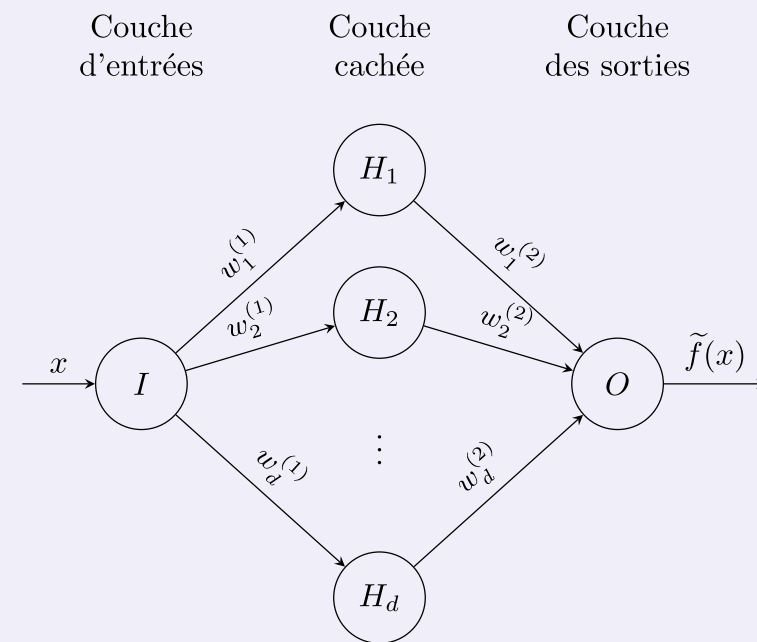
Hidden units reparameterize inputs into a space where the task becomes linearly separable—this is the core intuition for representation learning.

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

From Perceptron to MLP (Multi-Layer Perceptron)

Depth = Composed Nonlinear Features

$$h = \phi(W_1x + b_1), \quad \hat{y} = \tau(W_2h + b_2)$$

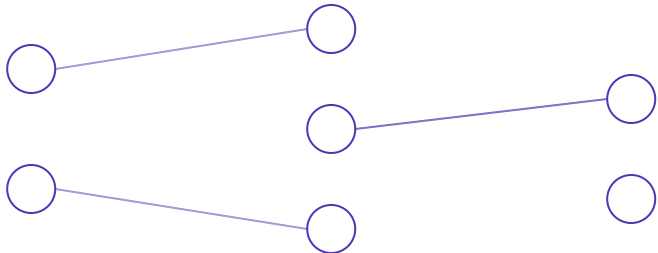


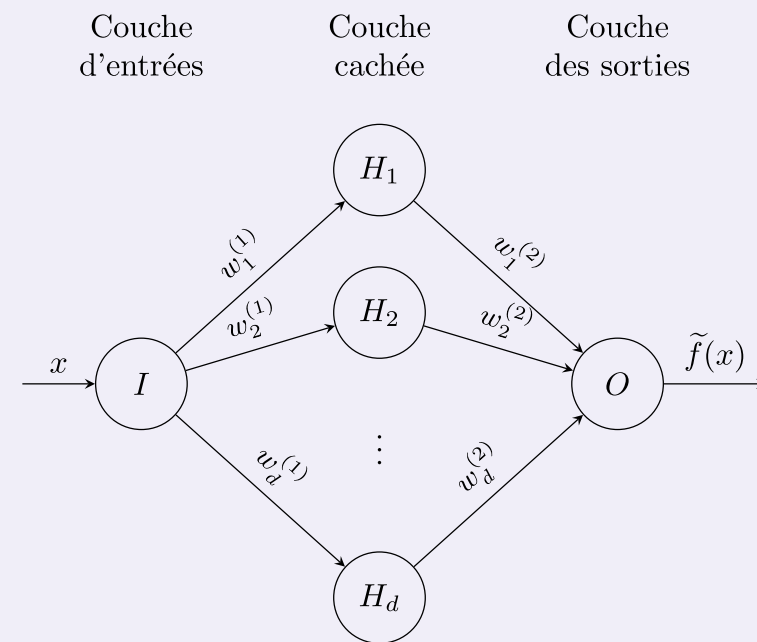
Each hidden layer applies an affine map followed by a nonlinearity, producing new features h as functions of the original inputs.

Width (units per layer) and depth (number of layers) trade off capacity, data needs, compute, and optimization difficulty.

Universal approximation (broad strokes): one sufficiently wide hidden layer can approximate many continuous functions on compact domains—in practice depth + inductive bias wins.

HYPERPARAMETER	WHAT IT CONTROLS
Hidden width	How many independent nonlinear features per layer
Depth	How many levels of composition (edges→textures→...)
Skip / residual	Ease of optimization in very deep nets





Forward pass: feed input x through each layer in order—compute logits or scores at the end (then softmax for class probabilities if needed).

Implementation-wise this is a computational graph: each op (matmul, ReLU, ...) knows how to propagate values forward.

Training repeats forward passes on mini-batches for efficiency and gradient noise that can help generalization.

STEP	OPERATION	OUTPUT CARRIES...
1	Linear: $z = Wx + b$	Weighted sum at each unit
2	Activation: $a = f(z)$	Nonlinear feature for next layer
3	Repeat / head	Class logits or regression value

Loss Functions: MSE vs Cross-Entropy

Match the Loss to the Output and Task

Regression — Mean Squared Error (MSE)

$$\mathcal{L} = \frac{1}{N} \sum_i \|\hat{y}_i - y_i\|^2$$

Penalizes large errors heavily (quadratic); differentiable everywhere—classic for continuous targets.

Assumes Gaussian-like noise in a probabilistic story; outliers can dominate—Huber loss is a robust alternative.

Classification — Cross-entropy

$$\mathcal{L} = -\frac{1}{N} \sum_i \sum_c y_{ic} \log \hat{p}_{ic} \quad (\text{often with logits + log-softmax for stability})$$

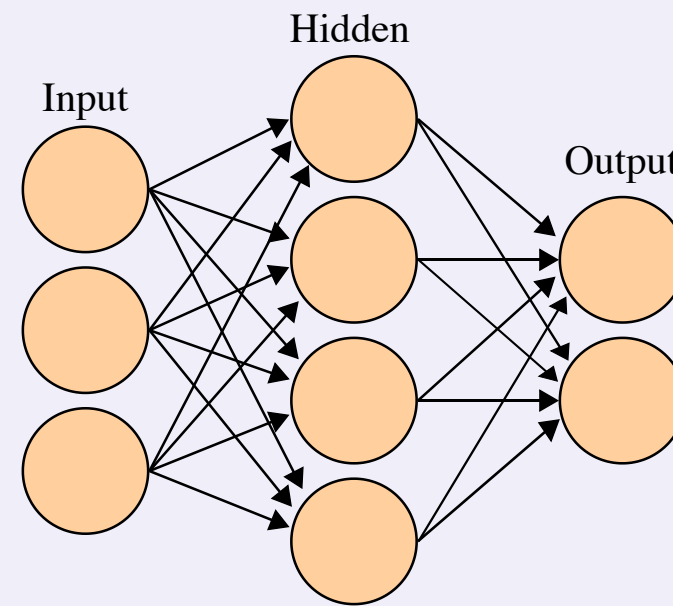
Compares predicted probabilities $\hat{p}$ to true distribution (one-hot or soft labels).

Pairs naturally with softmax at the output; gradients are well-behaved when implemented in log-space.

TASK	COMMON HEAD + LOSS
Regression	Linear output + MSE / Huber
Binary classification	Sigmoid + binary cross-entropy
Multi-class	Softmax + cross-entropy

● Backpropagation: Chain Rule on the Computational Graph

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w} \quad (\text{chain along paths})$$



- Backprop applies the chain rule systematically: compute adjoints $\partial \mathcal{L} / \partial$ each intermediate tensor from outputs backward.
- Reverse-mode autodiff (backprop) is efficient when many parameters feed one scalar loss—exactly the neural net case.
- Modern frameworks (PyTorch, JAX, TensorFlow) build the graph (explicit or traced) and implement this reliably—you still debug shapes, numerical stability, and vanishing gradients.

IDEA	MEANING
Shared subexpressions	Same forward values reused; backward visits each edge once
Topological order	Visit nodes after their successors—dynamic programming on the graph

Understanding backprop = understanding which paths carry gradient and which activations saturate.

Phase 2: Optimization & Training Strategy

Stable Optimization + Generalization

- Phase 2 is about the art of training: diagnosing fit, controlling gradients, picking optimizers, normalizing, and regularizing.
- Good models are not only “big”—they match capacity to data, monitor validation faithfully, and fail visibly when something is wrong.

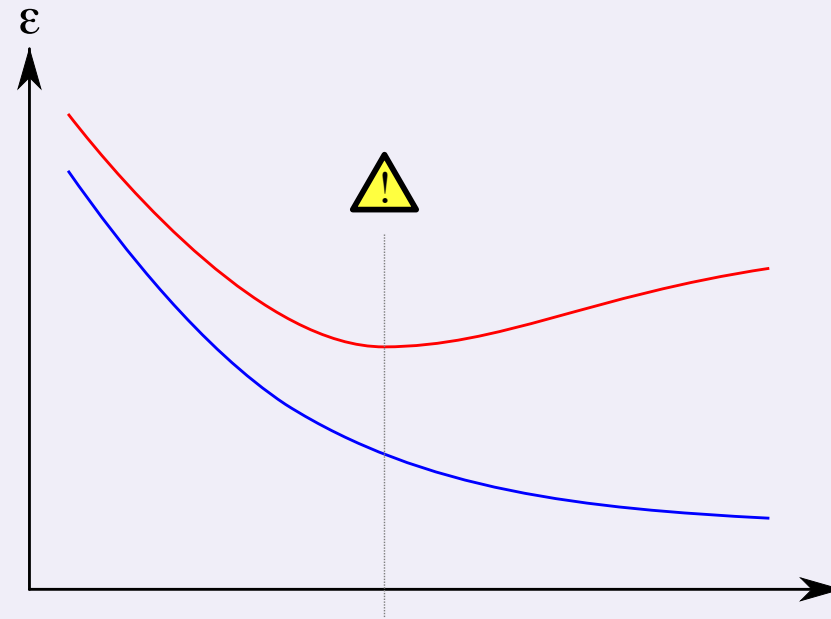
What you will control in practice

LAYER	EXAMPLES
Optimization	SGD, Adam, gradient clipping
Stabilization	Batch Norm, residual paths, initialization
Generalization	Dropout, weight decay, early stopping



Training Diagnostics: Underfitting, Good Fit, and Overfitting

Read Train vs Validation Behavior



Underfitting: both train and validation error stay high—the model is too simple, features are insufficient, or optimization is stuck (LR too low, wrong loss).

Good fit: train and validation errors are both low and track each other; small gap is normal if train is slightly better.

Overfitting: train error keeps dropping while validation worsens or plateaus badly—the model memorizes idiosyncrasies (noise, augment leakage).

REGIME	TRAIN LOSS	VAL LOSS / METRIC	TYPICAL RESPONSE
Underfitting	High	High	More capacity, better features, longer train, higher LR (carefully)
Good fit	Low	Low, close to train	Keep regularization; ship or iterate on data
Overfitting	Very low	Worse / diverging gap	More data, dropout, wd, simpler model, early stop

Vanishing and Exploding Gradients

Depth Multiplies Jacobians Layer by Layer

Vanishing: backprop multiplies many factors < 1 (saturated sigmoid/tanh, poor init)—early layers get tiny updates and learn slowly.

Exploding: repeated factors > 1 or large weights produce huge parameter updates—loss spikes, NaNs.

Fixes are layered: architecture (residual paths), activations (ReLU), initialization (He/Xavier), normalization (BN/LN), and optimization (clipping).

SYMPTOM	LIKELY CAUSE	MITIGATION
Early layers barely move	Vanishing chain through saturated units	ReLU, He init, residuals, norm
Loss → NaN quickly	Exploding activations or LR too high	Clip grads, lower LR, check loss scaling
Stable then sudden blow-up	Rare but real—mixed precision, bad data batch	Finite checks, gradient clipping

Learning rate, Momentum, and Adaptive Steps

- SGD + momentum: accumulates updates in a velocity vector to dampen oscillations in ravines and accelerate along consistent directions.
- Adam: tracks both first-moment (momentum) and second-moment (recent squared gradients) of gradients to adapt learning rates per parameter.
- SGD is often preferred in computer vision for generalization, while Adam is the default starting point for Transformers and general MLPs.

OPTIMIZER	CORE MECHANISM	WHEN TO USE
Vanilla SGD	Step in negative gradient direction	Rarely used alone today
SGD + Momentum	Adds fraction of previous step direction	Standard for CV models
Adam	Adapts learning rate per parameter based on gradients	Robust default for text/tabular/MLPs

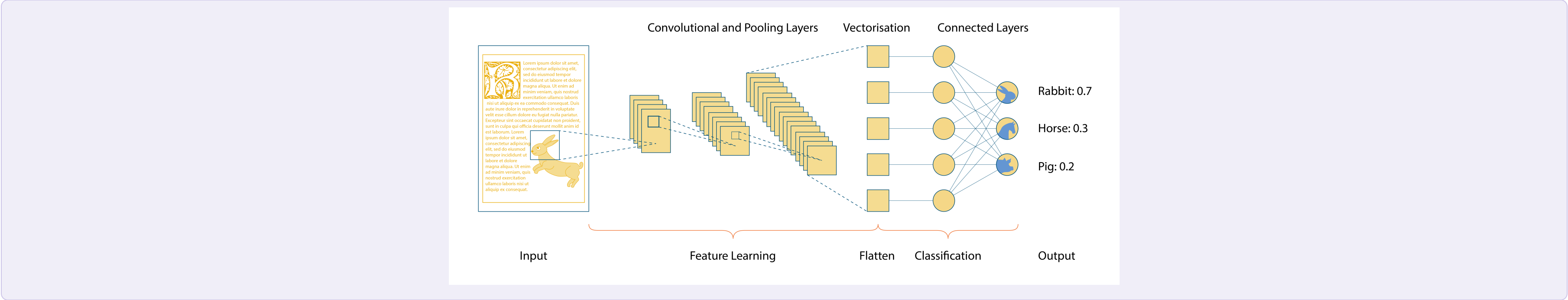
Learning rate (LR) is the single most important hyperparameter to tune (typically searched on a log grid, e.g., 1e-4, 1e-3, 1e-2).

Normalization: Batch Normalization

Stabilize Activations and Speed Up Deep Training

ETRA





Batch Norm (BN): normalizes activations across the mini-batch for each channel, then learns scale γ and shift β .

It stabilizes internal activations, allowing higher learning rates and reducing dependency on precise initialization.

Alternative is Layer Norm (LN) which normalizes across features per example—standard in Transformers and sequence models.

METHOD	NORMALIZATION DIMENSION	TYPICAL USE-CASE
Batch Norm	Across batch and spatial dimensions	CNN classifiers, ResNets
Layer Norm	Across features for a single token/example	Transformers, RNNs

Regularization: Dropout, Weight Decay, and Early Stopping

ETRA

Constrain Capacity to Prevent Overfitting



Dropout

Randomly zero out hidden units during training with probability p (e.g., 0.2-0.5) so neurons cannot co-adapt.

Acts as a powerful implicit ensemble method; disabled during inference (evaluation mode).

Weight Decay (L2 regularization)

Adds a penalty to the loss proportional to the squared sum of weights to keep weights small and smooth.

Early stopping

Monitor validation loss during training and stop when it ceases to improve for a certain number of epochs (patience).

TECHNIQUE	PRIMARY ACTION	INFERENCE BEHAVIOR
Dropout	Randomly drops connections	Turned OFF (weights scaled)
L2 Weight Decay	Shrinks weights toward zero	Turned ON (weights are static)
Early Stopping	Halts training at lowest val loss	N/A (model is checkpointed)

Phase 3: Specialized Architectures (Vision & Sequences)

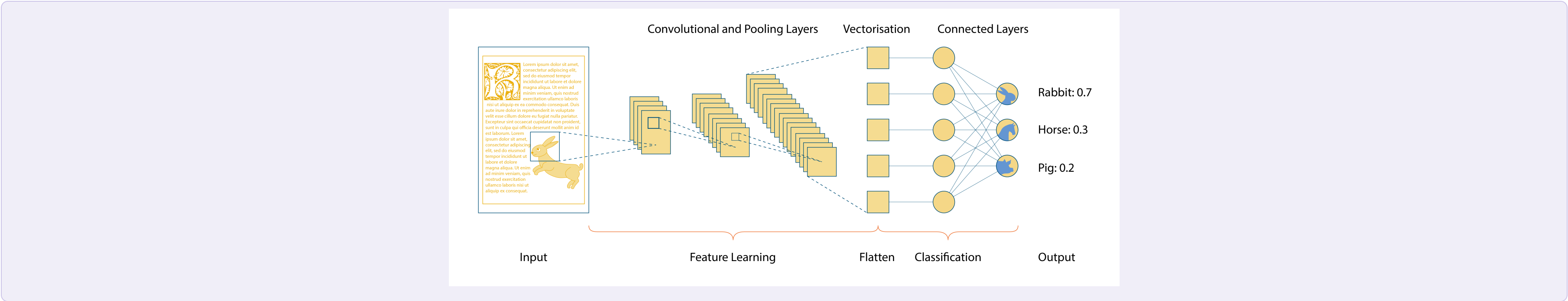
Images vs Ordered Sequences

- Images have 2D locality and translation symmetry—convolutions exploit shared weights across space.
- Sequences have order—recurrence, causal convolutions, or attention carry context across time or position.
- Choosing the right inductive bias beats blindly scaling the wrong architecture.

MODALITY	CORE STRUCTURE	REPRESENTATIVE LAYERS
Image / video frames	Spatial grids	Conv2D, pooling, residual blocks
Text / speech / sensors	Ordered tokens or time steps	RNN/GRU/LSTM, causal conv, self-attention

Image Data: How Computers “See” Pixels

Tensors, Channels, and Local Structure



A color image is usually a tensor of shape $H \times W \times 3$ (RGB); each entry is an intensity discretized into 8 bits (0–255) before normalization.

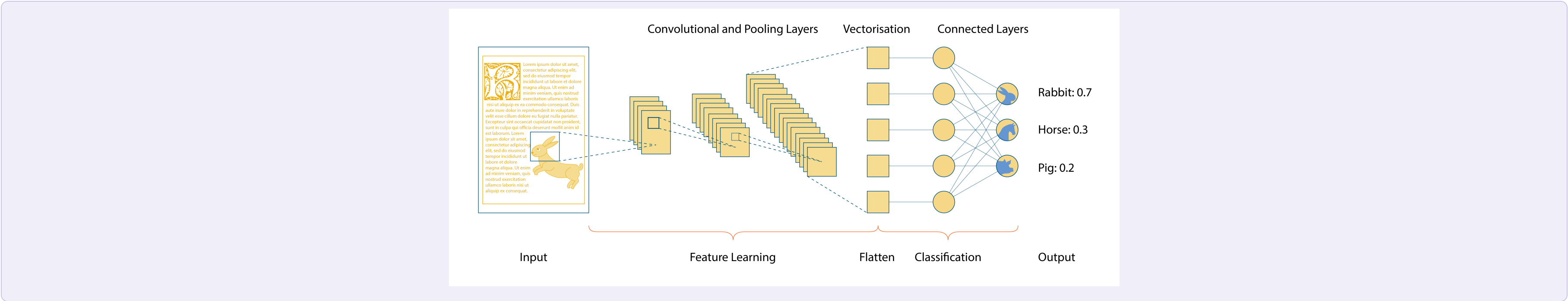
Meaning lives in local neighborhoods: small patches reveal edges; larger contexts reveal objects—hierarchical composition.

Augmentation (crop, flip, color jitter) teaches invariance; normalization (e.g., ImageNet mean/std) stabilizes optimization.

STAGE	WHAT HAPPENS
Raw sensor	Quantized intensities per channel per pixel
Preprocess	Resize, crop, normalize, augment
Model input	Batch × C × H × W tensor fed to stem convolutions

Convolution: Kernels, Edge Filters, and Feature Maps

Sliding Linear Filters + Nonlinearity





A kernel is a small learnable matrix (e.g., 3×3) that responds to local patterns; multiple kernels produce a stack of feature maps.

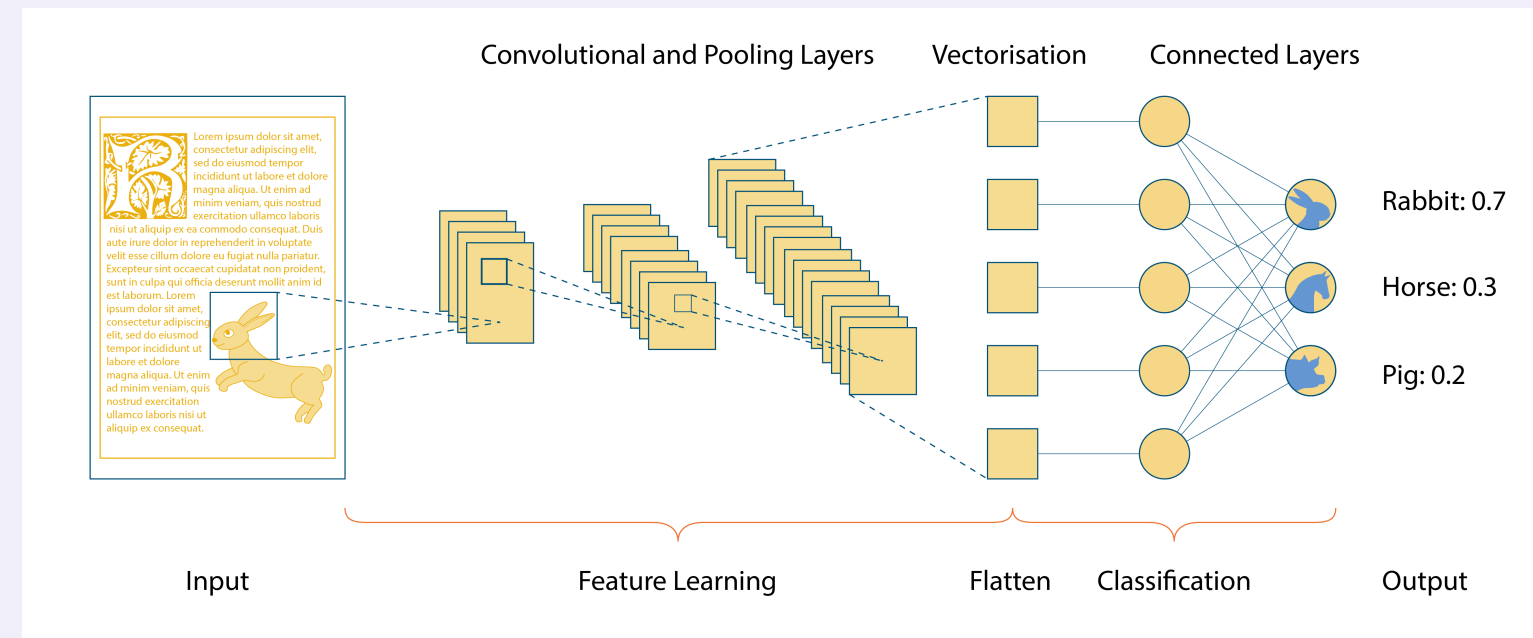
Classical edge detectors (Sobel, shown) are fixed kernels; CNNs learn task-specific filters from data.

Deep stacks widen the receptive field: early layers $\approx$ edges/textures; late layers $\approx$ object parts and semantics.

HYPERPARAMETER	EFFECT
Kernel size	Immediate neighborhood size (3×3 common)
Stride	Downsample spatially when >1
Padding	Keep spatial size (same padding) or control border

$$y = \mathcal{F}(x) + x$$

(residual block learns $\mathcal{F}$, identity carries gradient)



Standard CNN: Alternates Conv2D, Activation (ReLU), and Pooling (Max Pool) layers to extract spatial features and downsample.

AlexNet (2012): First large-scale CNN to win ImageNet; used ReLU, dropout, and multi-GPU training.

ResNet (2015): Introduced skip (shortcut) connections, allowing gradients to flow back easily through identity mappings. Enables training networks with 100+ layers.

Landmark ideas (simplified)

MODEL / BLOCK	IDEA YOU SHOULD REMEMBER
Conv + MaxPool	Feature extraction + spatial downsampling
AlexNet	Scale + ReLU + data aug on GPUs
ResNet	Residual mapping $F(x)+x$; skip paths help gradients

Transfer Learning with Pretrained Models

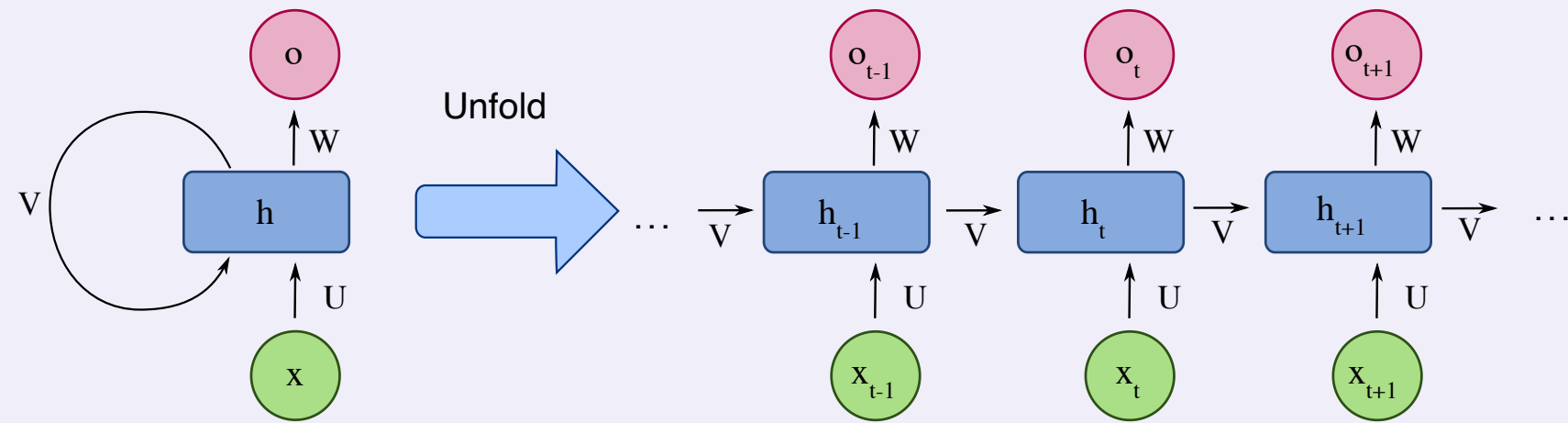
Reuse Features, Adapt the Head

- Pretraining on millions of labeled images (e.g. ImageNet) learns general low/mid-level filters (edges, textures) transferable to new domains.
- Typical recipe: replace the final classifier head; freeze the pretrained backbone for small datasets, or fine-tune with a very small learning rate for larger datasets.
- Transfer learning dramatically reduces training time and required labeled samples for a new vision task.

Target Dataset Size	Strategy
Small target dataset	Freeze backbone, train head only
Medium dataset	Fine-tune top blocks with a smaller learning rate
Large target dataset	Full fine-tune of all layers with a small learning rate

One Transition, Unfolded Through Time

$$h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b)$$



Sequence modeling: text, audio, and sensor streams have order; shuffling them breaks their semantic meaning.

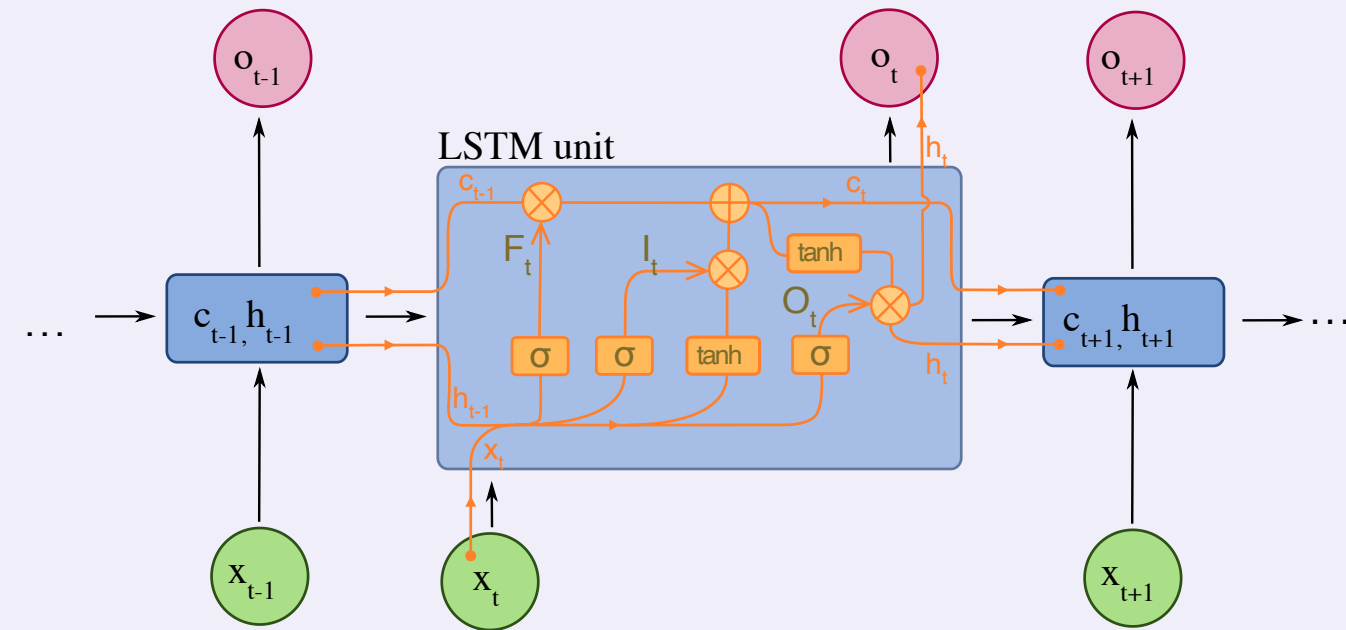
Recurrent Neural Networks (RNN): maintain a recurrent state vector h_t that acts as a memory summary of the past.

The same weights $W_h h$ and $W_x h$ are reused across all timesteps (parameter efficiency), but long-range dependencies suffer from vanishing/exploding gradients.

FEATURE	LIMITATION
Reuses parameters	Sequential execution (cannot parallelize steps)
Tracks state h_t	Cannot hold long-term memories in practice

LSTM: Gates for Long-Term Memory

Forget, Input, Output — Cell State Flows Linearly



Cell state c_t can accumulate information with an additive path—mitigates vanishing signal compared with plain tanh RNNs.

Forget gate f_t decides how much past cell content to erase; input gate i_t and candidate $\tilde{g}$ control new information written.

Output gate o_t filters what becomes hidden state h_t exposed to the next layer or timestep.

GATE	ROLE (INTUITION)
f_t	Erase irrelevant history from the cell
i_t	Allow new candidate information in
o_t	Expose part of the cell as h_t

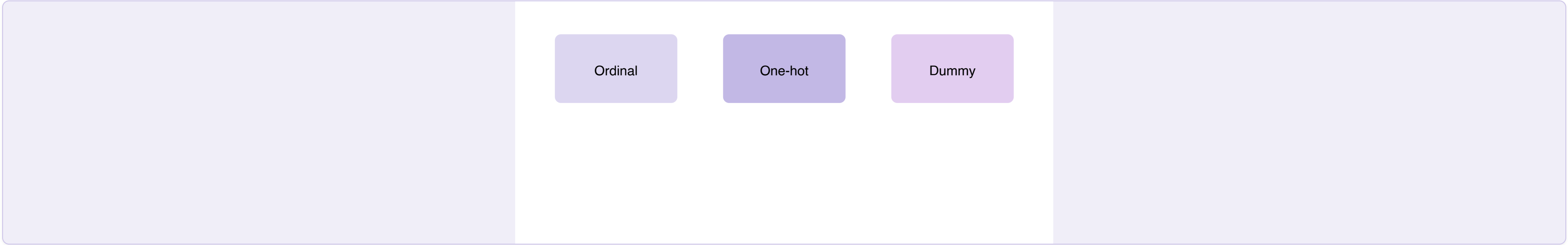
Phase 4: Generative Models & Reconstruction

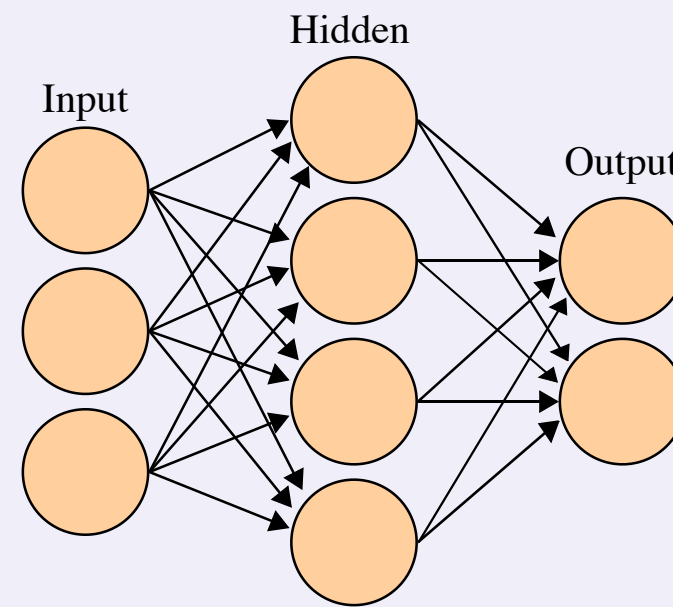
- Phase 4 explores unsupervised representation learning and basic generative frameworks.
- Autoencoders are the foundation here: they learn to compress inputs to a low-dimensional bottleneck (latent space) and reconstruct them.

CONCEPT	INPUT	GOAL
Autoencoder	Unlabeled data x	Compress and reconstruct $\hat{x} \approx x$

Autoencoders (AE): Compression and Reconstruction

Encoder → Bottleneck → Decoder





Encoder f maps input x to a low-dimensional code $z = f(x)$; decoder g maps z back to $\hat{x} \approx x$.

The bottleneck forces a compressed representation—useful for denoising (train on noisy $\rightarrow$ clean), anomaly detection (high recon error = abnormal).

Linear AE with MSE relates to PCA when constraints align—nonlinear AE learns curved manifolds.

VARIANT	IDEA
Denoising AE	Corrupt input, reconstruct clean—robust features
Sparse AE	Penalty on activations—encourages informative sparse codes
Contractive AE	Penalize sensitivity of code to small input changes—smoother latent map

Section Takeaways

1. Neural nets learn features — depth helps but needs regularization.
2. Activations + initialization control gradient flow.
3. CNNs for grids, RNNs for sequences — match architecture to structure.

Think: What symptom tells you the network is underfitting vs overfitting?

WEEK 3 · SESSION 1 · NLP

8

NLP Workshop: Foundations to Seq2Seq

NLP pipeline

Text preprocessing

Bag-of-words

SpaCy & NLTK

▸ 32 slides · ~51 min

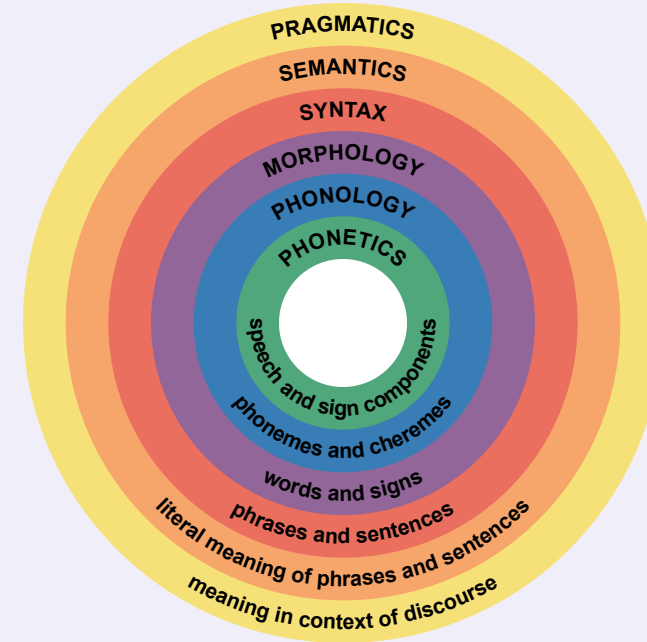
Where Are We in the Bootcamp?



Week 3 Session 1 — NLP goals, regex, normalization, and hands-on text cleaning.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8 NLP Fundamentals**
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16** MLOps

Why Natural Language Is Difficult for Machines



Natural language is ambiguous, context-dependent, and full of idioms.

Meaning often depends on pragmatics, domain, and cultural background.

The same sentence can map to multiple valid interpretations.

NLP systems must model form, meaning, and context simultaneously.

EXPRESSION	POSSIBLE INTERPRETATION A	POSSIBLE INTERPRETATION B
The bank is closed	Financial institution is closed	River bank area is inaccessible
It's cold	Low temperature	Emotionally distant behavior
The chicken is ready to eat	Food is ready to be eaten	Animal is ready to eat food

Robust NLP requires disambiguation using surrounding textual and situational context.

Regular Expressions (Regex): Core Idea

Regex is a compact pattern language for matching and transforming text.

Typical uses include validation, extraction, and rule-based cleanup.

Useful for emails, phone numbers, logs, and normalization pipelines.

PATTERN	MEANING	EXAMPLE MATCH
\d+	one or more digits	2026
\w+	one or more word chars	nlp_model1
^[A-Z]	starts with uppercase letter	Hello
\S+	one or more non-space chars	token123

Text Cleaning — Conceptual Steps & Python Code

Conceptual stages matched with Python regex implementations

STAGE	PYTHON IMPLEMENTATION	GOAL
1. Decode & Clean HTML	<code>html.unescape(text) & re.sub(r'<[^>]+>', '', text)</code>	Keep readable text only
2. Remove Contacts/URLs	<code>re.sub(r'https?://\S+ www\.\S+', '', text)</code>	Drop URL metadata
3. Remove Mentions/Socials	<code>re.sub(r'@\w+ #\w+', '', text)</code>	Remove handles & hashtags
4. Drop Non-ASCII & Emoji	<code>text.encode('ascii', 'ignore').decode('ascii')</code>	Drop emoji/special symbols
5. Normalize Whitespace	<code>re.sub(r'\s+', ' ', text).strip()</code>	Normalize extra spaces

A regex-based cleaning function is usually the first stage in an NLP preprocessing pipeline.

Text Cleaning — Worked Example

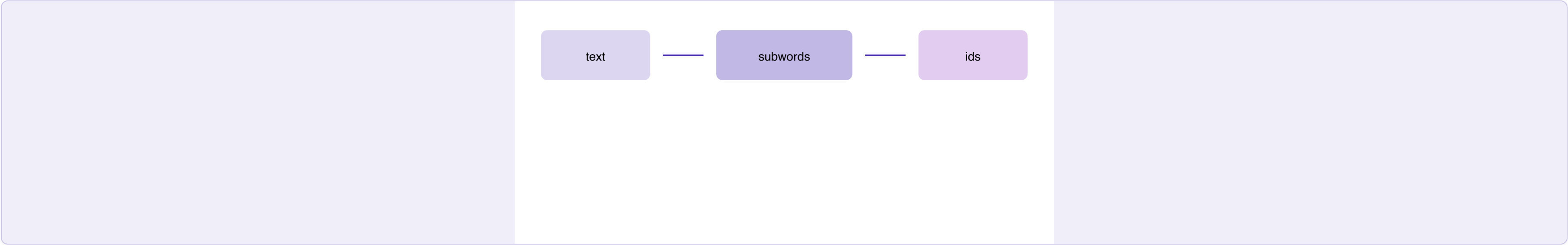
Trace of raw input transitioning to clean text

STAGE	OPERATION PERFORMED	TEXT SNAPSHOT
0. Raw Input	Original raw string	John: "AI is amazing!!!! 🤖🔥 " Visit: https://ai.com @john #AI
1. Clean HTML	Decode entities & remove tags	John: "AI is amazing!!!! 🤖🔥 " Visit: https://ai.com @john #AI
2. Remove URLs	Remove web addresses	John: "AI is amazing!!!! 🤖🔥 " Visit: @john #AI
3. Remove Socials	Remove @mentions and #hashtags	John: "AI is amazing!!!! 🤖🔥 " Visit:
4. Drop Emoji/Symbols	ASCII-only encoding & cleanup	John AI is amazing!!!! Visit

Text cleaning produces standard, noise-free input for tokenizer components.

Tokenization Granularities (Word, Char, Subword)

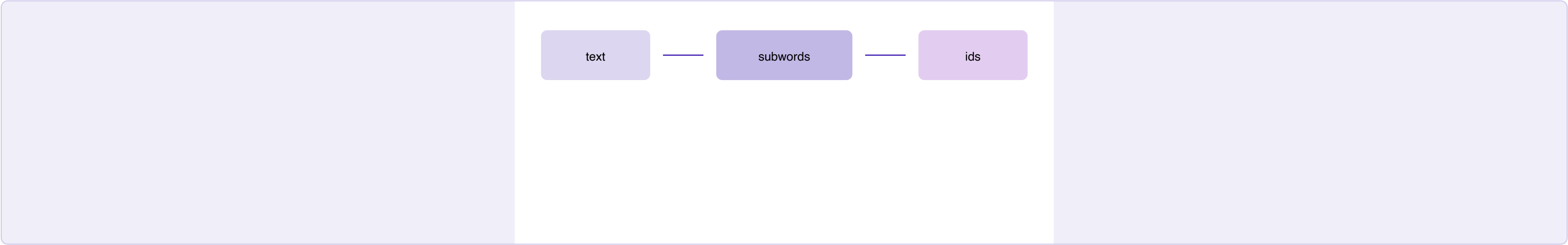
Comparing different levels of text splitting



granularity	Strength	Tradeoff	Example Output (for 'running')
Word / Space-based	Human-readable tokens	Weak on rare/compound words (OOV)	['running']
Character-based	No unknown words (no OOV)	Very long sequences, loss of meaning	['r', 'u', 'n', 'n', 'i', 'n', 'g']
Subword-based	Handles rare words, compact vocab	Requires tokenizer training/tuning	['run', '##ning']

Subword tokenization is the industry standard for modern LLMs and neural NLP models.

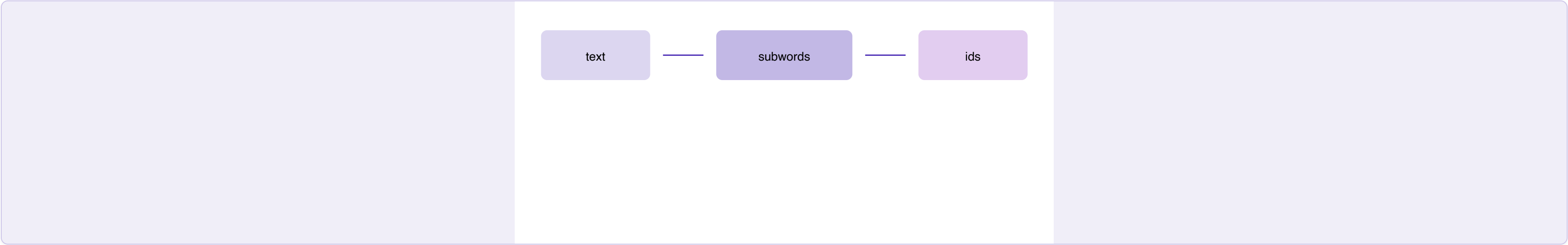
Same sentence, word / char / subword splits side by side



TOKENIZER TYPE	PYTHON IMPLEMENTATION	RESULT ON "I'M RUNNING TO THE STORE!"
Word	word_tokenize(text)	['I', "'m", 'running', 'to', 'the', 'store', '!']
Character	list(text)	['I', "'", 'm', ' ', 'r', 'u', 'n', 'n', 'i', 'n', 'g', ' ', 't', 'o', ' ', 't', 'h', 'e', ' ', 's', 't', 'o', 'r', 'e', ' ', '!', '']
Subword	model_tokenizer.tokenize(text)	['I', "'m", 'run', '##ning', 'to', 'the', 'store', '!']
Sentence	sent_tokenize(text)	["I'm running to the store!"]

Subword Tokenization (LLMs) — Why It Wins

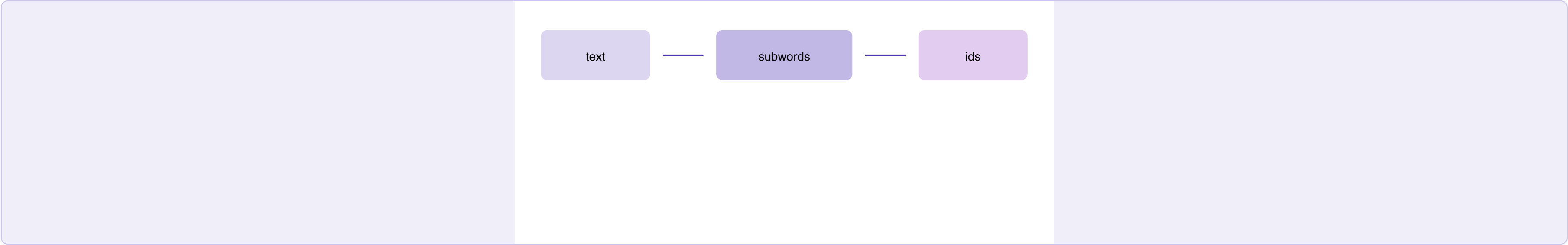
How subwords solve core vocabulary challenges



WORD TOKENIZER ISSUE	SUBWORD SOLUTION	EXAMPLE WORKED OUT
Huge vocabulary size	Reuse frequent morphemes/pieces	unbelievably → un + ##believe + ##ably
Out-of-Vocabulary (OOV) words	Break unknown forms into known units	ChatGPT → ['Chat', 'G', 'PT']
Morphological variants	Shared roots reduce feature sparsity	running/runs/runner share 'run'

BPE Walkthrough & Inference Example

How Byte Pair Encoding learns merges and splits unseen words

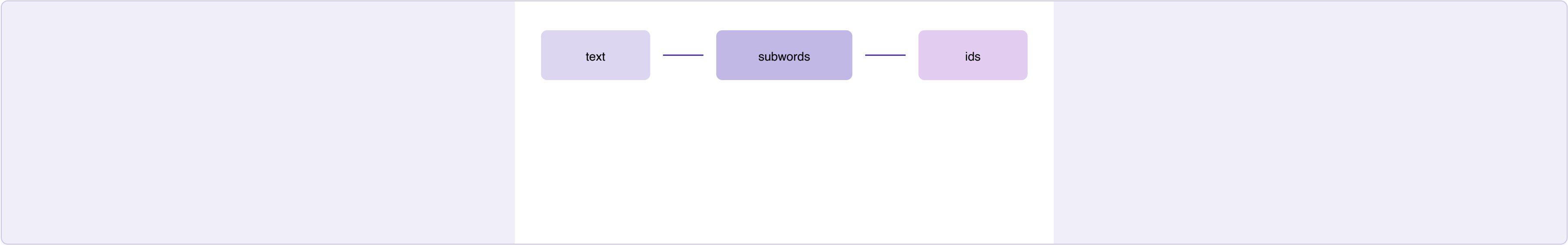


- Walkthrough: start with chars (l, o, w, e, s, t, r) -> merge l+o (lo) -> merge lo+w (low) -> compact vocabulary.
- Inference on 'lowered the lowest curtain': splits rare words to ['low', 'er', 'ed'] and ['low', 'est'] while keeping 'the' and 'curtain' whole.

WORD	SUBWORD OUTPUT	EXPLANATION
lowered	['low', 'er', 'ed']	Decomposes into learned root and suffix subwords
lowest	['low', 'est']	Decomposes into base 'low' and suffix 'est'

BPE repeatedly merges frequent adjacent units until target vocabulary size is reached.

Filtering low-information tokens from texts



- Function words (the, is, and) dominate word counts but carry minimal topic signal.
- Removing stop words reduces dimensionality for TF-IDF & Bag-of-Words.
- Critical Rule: NEVER remove stop words for language models (GPT/BERT), translation, or sentiment analysis where negation ('not') and syntax carry the core meaning.

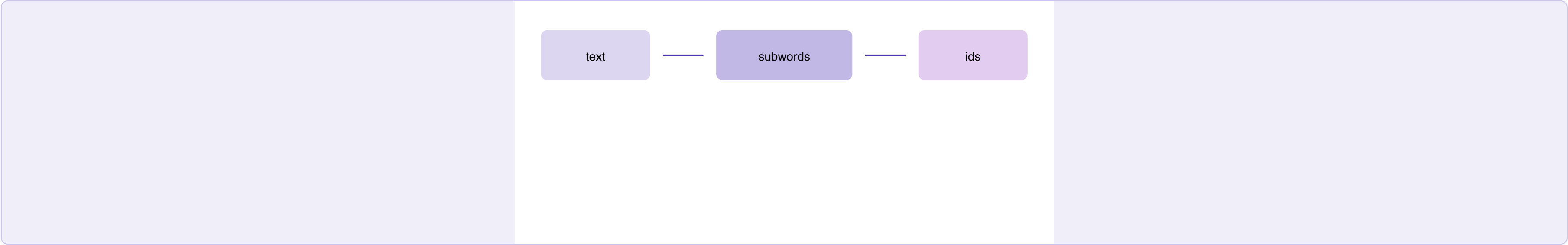
Task Category	Stop Word Policy	Reasoning
Search & Topic Modeling	Remove	Improves speed and focus on key words
Large Language Models	Keep	Syntactic and semantic flow is vital
Sentiment Analysis	Keep	Negations like 'not' reverse polarity

Normalizing words to their base forms

ASPECT	STEMMING	LEMMATIZATION
Mechanism	Rule-based suffix chopping (heuristic)	Dictionary lookup + morphological analysis
Speed	Very fast, computationally cheap	Slower, requires resource lookup
Output	Can be non-words (e.g., study -> studi)	Always dictionary words (e.g., study -> study)
POS Awareness	No	Yes (e.g., saw -> see or saw depending on POS)

Lemmatization preserves semantics better than aggressive stemming.

Grammar Labels per Token



- Universal categories include NOUN, VERB, ADJ, ADV, DET, ADP, PROPN, NUM, ...
- POS feeds lemmatization quality and downstream relation extraction.
- SpaCy exposes coarse pos_ and fine-grained tag_ plus dependency dep_.

POS	ROLE	QUICK EXAMPLES
NOUN / PROPN	Things and names	city, Tesla
VERB / AUX	Actions and helpers	run, is
ADJ / ADV	Modifiers	quick, quickly
DET / ADP	Structure words	the, in

Named Entity Recognition (NER)

Typed Spans over Text

- Labels include PERSON, ORG, GPE, DATE, MONEY, PERCENT, PRODUCT, EVENT, ...
- Useful for indexing, compliance redaction, financial news graphs, and search facets.
- Small models err on edge cases — always spot-check domain text.

LABEL	EXAMPLES
PERSON	Tim Cook, Jensen Huang
ORG / GPE	Apple, Texas
MONEY / PERCENT	\$430 billion, 3.5%

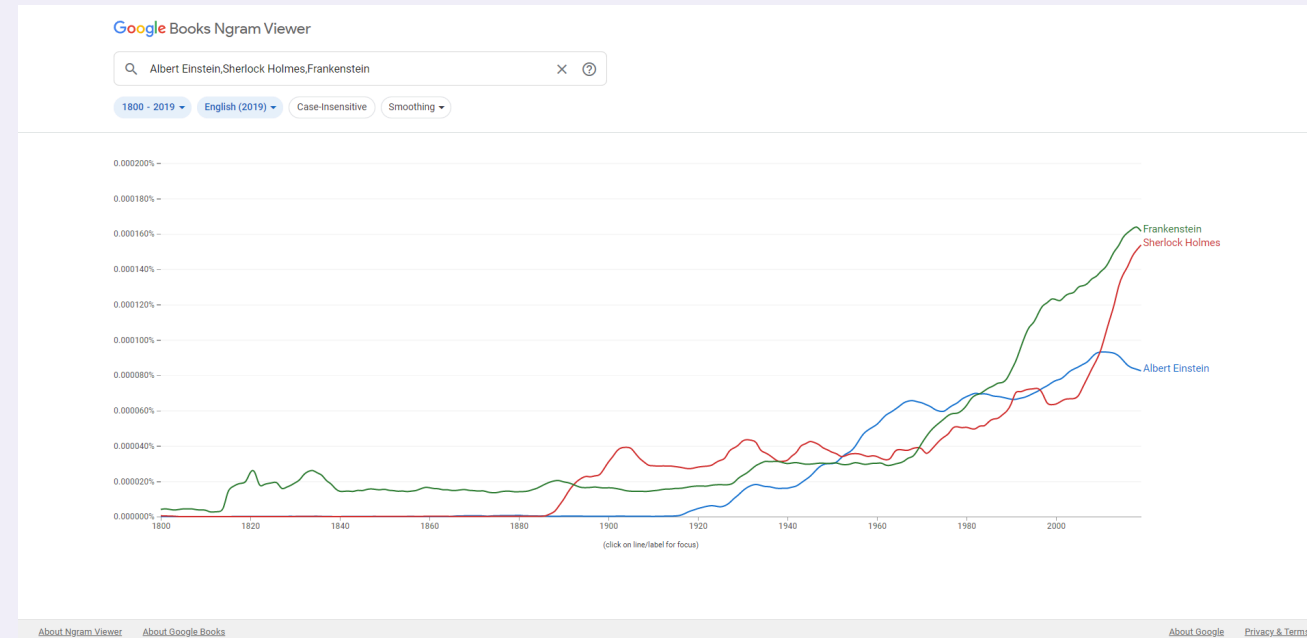
● Full SpaCy Pipeline Functionally

ETRA

- One doc object: tokens, lemmas, POS, entities, noun_chunks, sents.
- Typical export: clean token list, lemma bag, entity list, POS histogram.
- Use length-based heuristics (e.g. longest sentence) only as weak importance cues.

NLP Language Modeling with N-grams

From Count-Based Prediction to Evaluation



Language models estimate probabilities over word sequences.

N-gram models predict the next token from a limited context window.

This module connects N-gram intuition to modern LLM generation.

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1})$$

- | A language model can score full sentences or predict the next word.
- | Exact estimation over full histories is intractable for real corpora.
- | Approximation strategies include N-grams, smoothing, and neural LMs.



Worked Bigram Example

Given counts: $C(\text{"want to"}) = 608$, $C(\text{"want"}) = 927$

$$P(\text{"to"} \mid \text{"want"}) = \frac{608}{927} \approx 0.656$$

Interpretation: in this corpus, "to" follows "want" about 65.6% of the time.

Count quality depends strongly on corpus domain and size.

Why Use Log Probabilities

- Multiplied many small probabilities causes numerical underflow.
- Log transform converts multiplication into stable addition.
- Sequence scoring becomes computationally robust and efficient.

ORIGINAL SPACE	LOG SPACE
$P = p_1 * p_2 * ... * p_k$	$\log P = \log p_1 + \log p_2 + ... + \log p_k$
Very small numbers	Numerically stable sums

● Perplexity (PP): Interpretation

$$\text{PP}(W) = P(w_1, \dots, w_T)^{-\frac{1}{T}} = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid h_t) \right)$$

Perplexity is the average branching uncertainty of the model.

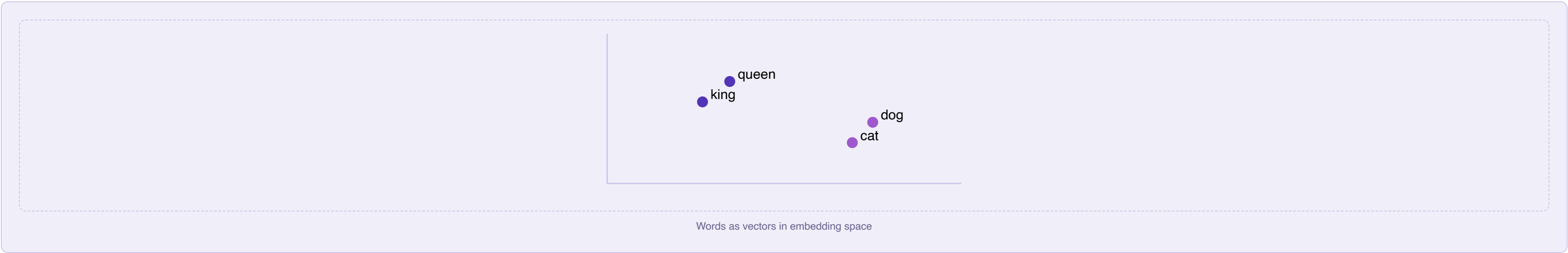
Lower PP means better predictive confidence on unseen text.

Use identical test sets when comparing models.

Raw sentence probabilities are length-sensitive; perplexity normalizes by token count.

PROPERTY	STATIC EMBEDDINGS (WORD2VEC/GLOVE)	CONTEXTUALIZED EMBEDDINGS (ELMO/BERT/GPT)
Vector per word	One fixed vector	Different vectors per context
Polysemy handling	Weak	Strong
Context direction	Usually local/global corpus only	Bidirectional or autoregressive sequence context
Task transfer	Moderate	High with pretraining + fine-tuning

Contextualization is a core enabler for modern LLM quality.



- Static embeddings: one vector per word type.
- Contextual: vector depends on surrounding tokens (ELMo, BERT-style).

Architecture Layers

Token/base embedding layer maps input ids to dense vectors.

Context encoder layers (often Transformer blocks) enrich each token with sequence context.

Task head uses contextual features for prediction.

Layer Semantics

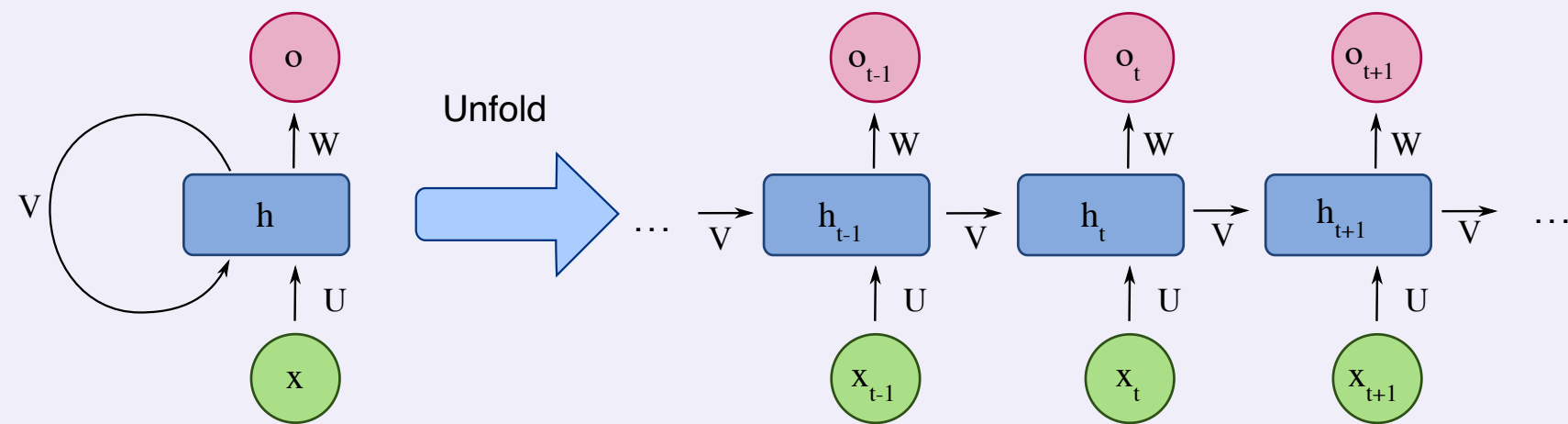
Lower layers capture local syntax/patterns.

■ Middle layers improve sense disambiguation.

■ Upper layers capture richer semantics and task-specific signals.

RNN Mechanism and Weight Sharing

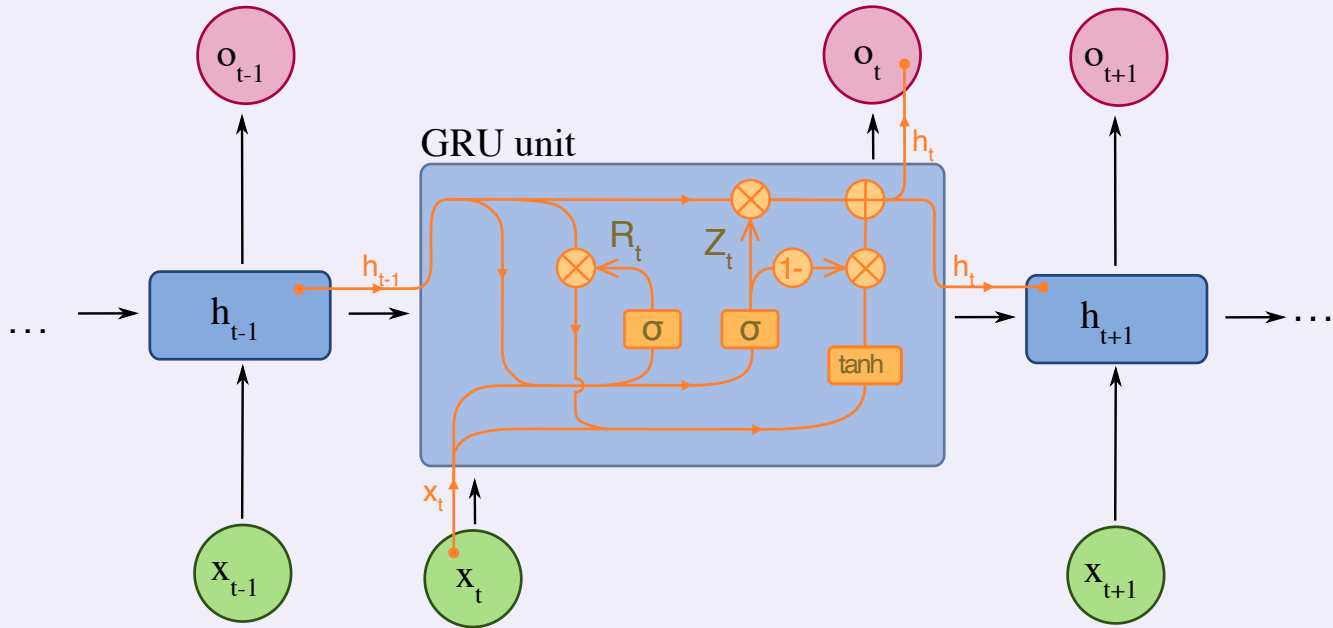
$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$



- At time t , the RNN consumes current input and previous hidden state.
- The same cell and weights are reused at every time step.
- This recurrence captures sequential information in variable-length text.

PATTERN	MAPPING	EXAMPLE TASK
One-to-One	single input -> single output	basic regression/classification
One-to-Many	single input -> sequence output	image captioning
Many-to-One	sequence input -> single output	sentiment classification
Many-to-Many	sequence input -> sequence output	machine translation

Sequence-aware patterns are where recurrent models provide clear value.

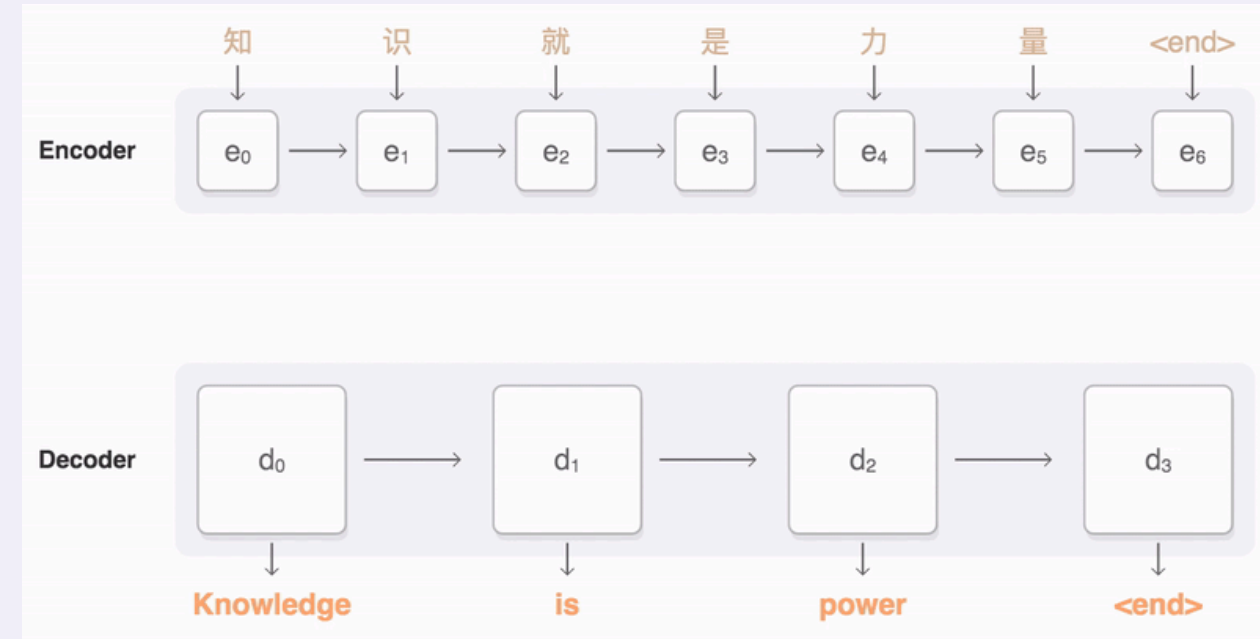


MODEL	VANISHING GRADIENT RISK	KEY STRENGTH	TYPICAL LIMITATION
Vanilla RNN	High	Simple architecture	Poor long-range memory
GRU	Lower	Efficient gating with fewer parameters	Can still degrade on very long contexts
BiRNN	Medium	Uses past and future context	Higher compute and memory
LSTM	Low (relative)	Strong long-term dependency handling	Heavier than GRU

Ordinal

One-hot

Dummy

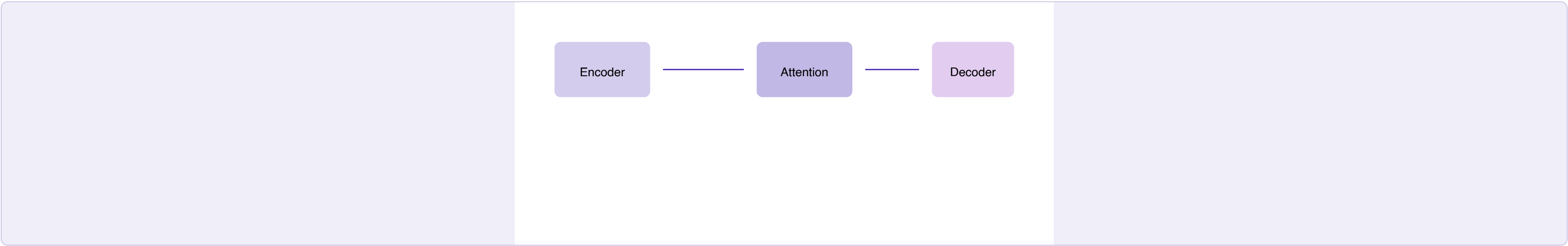


Seq2Seq maps variable-length input sequences to variable-length outputs.

Encoder-decoder models were foundational for neural machine translation.

This module covers training, decoding, bottlenecks, attention, and metrics.

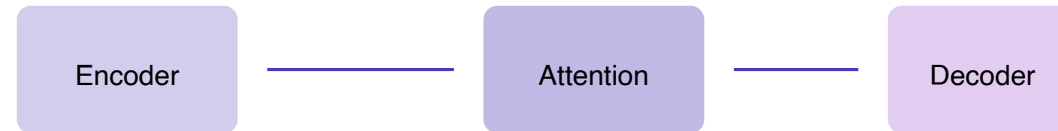
Inference: Greedy Decoding vs Beam Search



METHOD	DECISION RULE	STRENGTH	RISK
Greedy	Pick top token each step	Fast and simple	May miss globally better sequence
Beam Search	Track top-k hypotheses each step	Better global sequence quality	Higher compute; can become generic/bland

Typical beam sizes are moderate (e.g., 4-10) to balance quality and cost.

● Information Bottleneck in Basic Seq2Seq



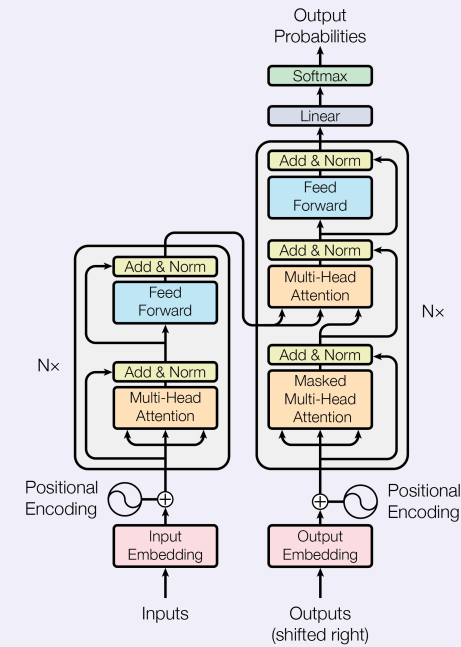
Compressing a full source sentence into one fixed vector can lose detail.

Longer/complex inputs worsen the bottleneck effect.

Decoder needs different source details at different output steps.

This motivates attention over all encoder hidden states.

● Attention as the Bottleneck Solution

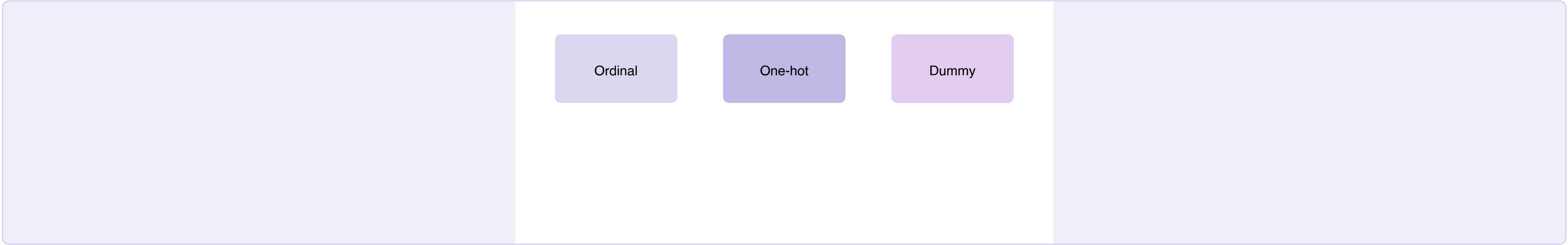


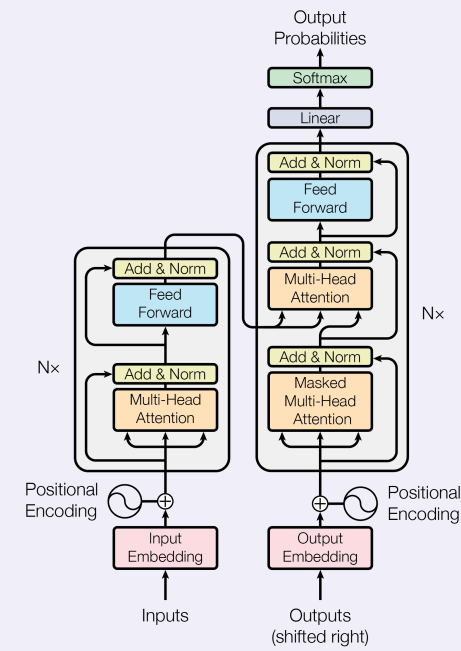
Decoder attends to relevant encoder positions at each generation step.

Dynamic alignment improves translation adequacy and fluency.

Attention laid the foundation for transformer-dominant NMT systems.

Metric	Orientation	What it Measures	Common Limitation
BLEU	Precision-oriented	n-gram overlap from candidate to reference	Weak semantic/syntactic sensitivity
ROUGE-N	Recall-oriented	n-gram overlap from reference to candidate	Can reward lexical overlap over meaning
F1 (with overlap metrics)	Balance precision/recall	Harmonic tradeoff view	Still limited on deep semantics





Encoder builds contextual token representations from source text.

Decoder generates target tokens autoregressively with attention to encoder states.

Foundation architecture for translation, summarization, and many LLM pipelines.

Attention mechanisms reduce reliance on recurrence for sequence modeling.

Section Takeaways

1. Language is ambiguous — preprocessing choices persist through the pipeline.
2. Clean raw text before tokenization; regex handles structured patterns.
3. The full NLP pipeline: ingest → clean → tokenize → analyze → model.

Think: Which cleaning step would break downstream NER if skipped?

WEEK 4 · SESSION 1 · GenAI

14

GenAI: BERT, T5, GPT, Attention

Transformers

BERT & GPT

Fine-tuning

LLM scaling

▸ 22 slides · ~35 min

Where Are We in the Bootcamp?



Pretrain at scale, fine-tune for tasks, generate with decoders.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14 Generative AI**
- **S15** RAG Systems
- **S16** MLOps

Generative AI: Core Concepts

BERT, T5, GPT, LLMs, and Attention (Focused Edition)

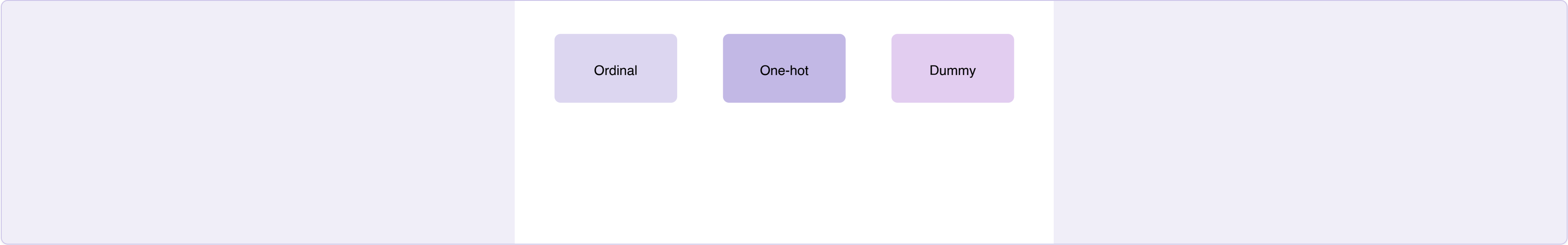
ETRA

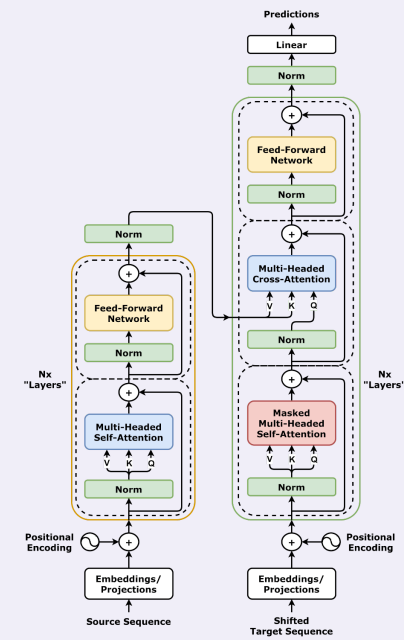
- This section extracts the most important ideas into a compact teaching flow.
- Focus areas: transfer learning, model families, attention mechanics, scaling, and practical deployment.
- Designed for bootcamp delivery within roughly 20 slides.

STAGE	DATA TYPE	GOAL
Pretraining	Large mostly unlabeled corpora	Learn general language representations
Fine-tuning	Task-labeled dataset	Adapt model to specific downstream objective
Inference	User prompt/context	Generate or classify outputs for real tasks

Transfer learning reduces labeled data needs and accelerates convergence.

Strategy	What Changes	Pros	Tradeoff
Feature-based	Freeze pretrained backbone; use embeddings as input features	Fast and lightweight	Less task adaptation
Fine-tuning	Update model weights (often with a task head)	Higher task performance	More compute and tuning effort





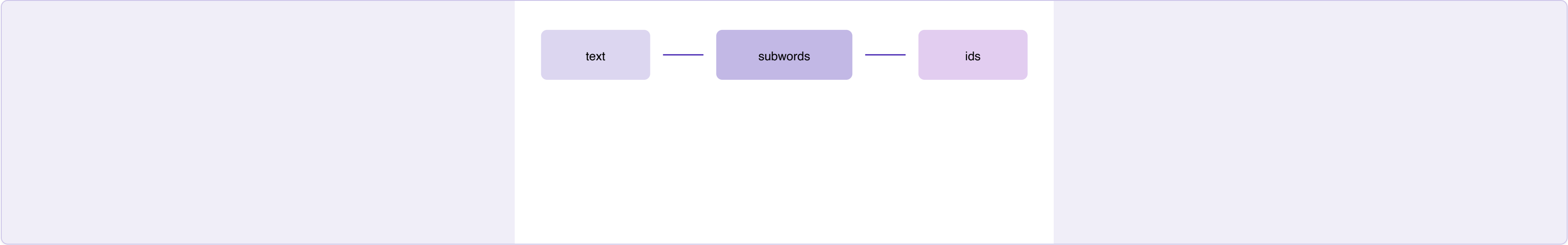
BERT learns contextual token representations using both left and right context.

Strong for understanding tasks: classification, NER, QA, and retrieval-oriented use cases.

Pretrained at scale (Wikipedia + BooksCorpus), then adapted to downstream tasks.

OBJECTIVE	HOW IT WORKS	WHAT IT TEACHES
MLM (Masked Language Modeling)	Mask subset of tokens and predict originals	Bidirectional contextual semantics
NSP (Next Sentence Prediction)	Predict whether sentence B follows sentence A	Inter-sentence coherence signals

MLM and NSP were jointly used in original BERT training setup.



- | [CLS] is prepended and its final embedding is used as a sequence-level summary.
- | [SEP] separates sentence segments and marks boundaries.
- | In classification tasks, [CLS] passes through a lightweight prediction head.

Token design helps unify single- and pair-sentence tasks.

Arabic Adaptation Example: AraBERT

- AraBERT follows BERT-base style architecture with Arabic-specific preprocessing.
- Segmentation and Arabic-tailored tokenization improve vocabulary coverage.
- Domain/language adaptation quality can matter more than raw parameter count.

Component	Adaptation Benefit
Farasa-style segmentation	Better handling of prefixes/suffixes and morphology
Arabic-focused vocabulary	Lower fragmentation and stronger lexical coverage
Arabic corpus curation	Improved contextual understanding in target language



T5: Unified Text-to-Text Framework

ETRA

T5 reframes every NLP task as text input -> text output.

Task instruction is included as a prefix (e.g., "summarize:", "translate:").

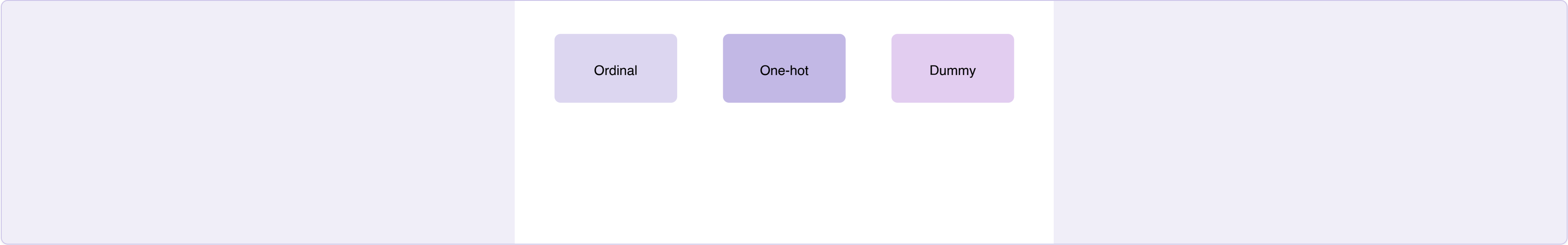
One shared architecture/training recipe serves multiple tasks.

This design strongly influenced modern prompt-based LLM usage.

T5 Pretraining Objective: Span Corruption

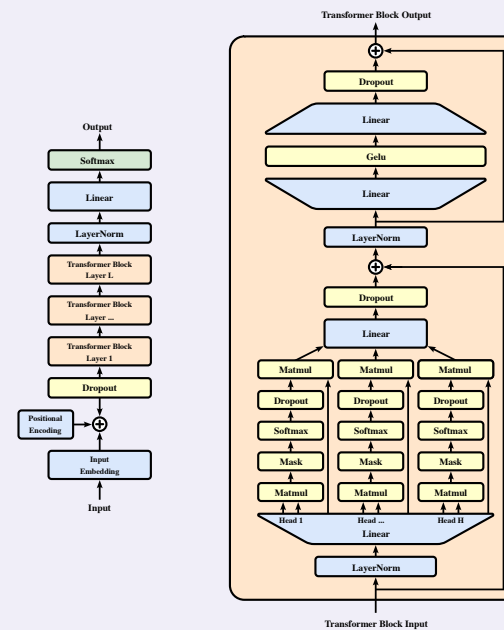
- Remove contiguous text spans from input and replace with sentinel tokens.
- Train model to reconstruct missing spans autoregressively.
- Encourages stronger generative behavior than token-only masking.

MODEL	CORRUPTION STYLE	TYPICAL STRENGTH
BERT	Random masked individual tokens	Language understanding tasks
T5	Contiguous span corruption	Generation + multitask transfer



ATTENTION TYPE	VISIBILITY	USED IN
Fully visible	All tokens attend to all tokens	Encoder representations
Causal	Token attends to past only	Autoregressive generation
Prefix-causal hybrid	Full source access + causal target decoding	Seq2Seq decoding in T5

GPT: Decoder-Only Autoregressive Generation



GPT predicts next token from left context only (causal modeling).

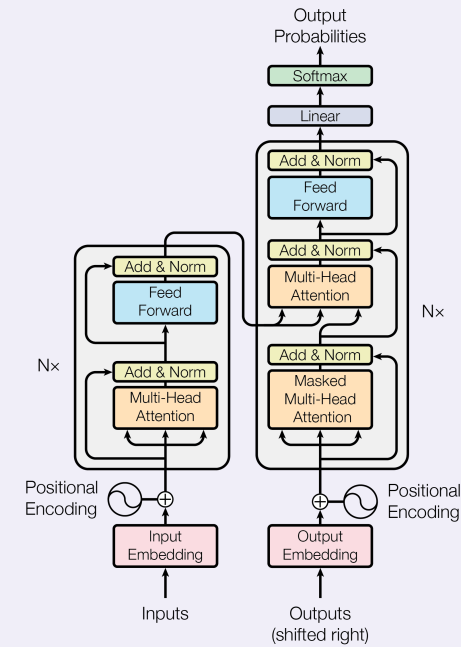
Uses masked self-attention in decoder stack for no-lookahead generation.

Excellent for open-ended completion, instruction following, and synthesis.

GENERATION	APPROX. PARAMETERS	KEY THEME
GPT-1	117M	Proof of transfer-learning viability
GPT-2	1.5B	Large-scale web pretraining for generation
GPT-3	175B	Few-shot in-context capabilities at scale

Model utility scales with data, architecture, and training compute quality.

● Attention: Why It Was a Breakthrough



Attention solves fixed-vector bottlenecks in sequence transduction.

At each step, model dynamically focuses on most relevant source positions.

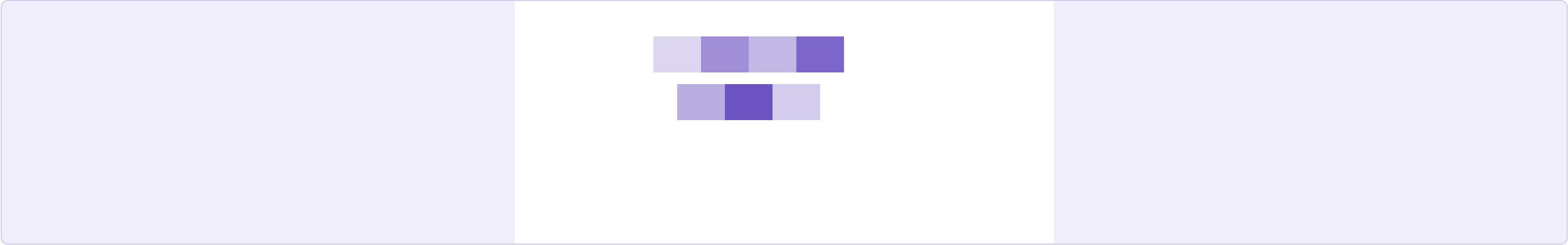
Entire mechanism is differentiable and learned end-to-end.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^{\top}}{\sqrt{d_k}} \right) V$$

- Compute relevance scores between query state and source states.
- Apply softmax to obtain attention weights (probability distribution).
- Return weighted sum of value/source states as contextual output.

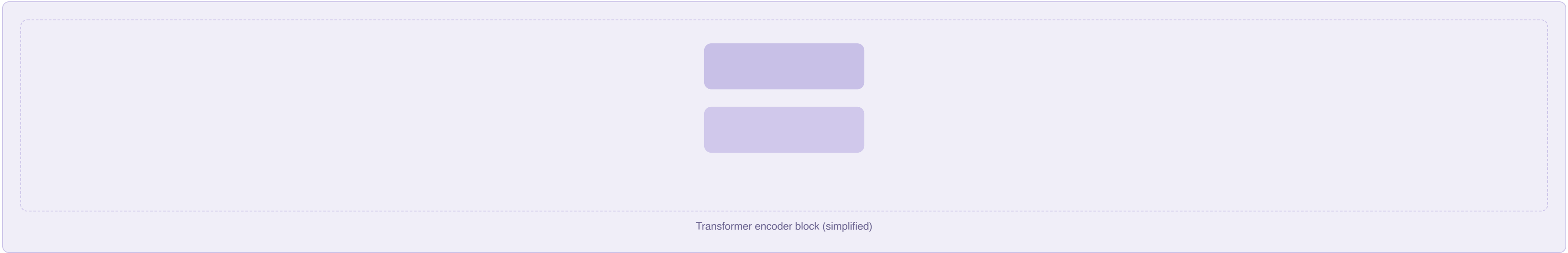
VARIANT	SCORING FUNCTION	TYPICAL PLACEMENT
Luong	Bilinear/dot-style	Attention after decoder state update
Bahdanau	MLP/additive score	Attention integrated before decoder update

Both approximate alignment; design choice affects speed and accuracy tradeoffs.



- Query asks for relevant context; Key indexes available context; Value carries content.
- Self-attention lets each token aggregate information from other tokens in sequence.
- Multi-head attention captures different linguistic relations in parallel.

Different heads can specialize in syntax, agreement, locality, or semantics.



- Decoder cannot access future tokens during generation.
- Causal mask enforces left-to-right consistency and prevents information leakage.
- Enables parallel training over full target sequence while preserving autoregressive objective.

BOTTLENECK	COMPLEXITY	PRACTICAL IMPACT
Attention map computation	$O(L^2)$	Time/memory explode as context grows
Activation storage (training)	$O(N \cdot L \cdot d_{model})$	High VRAM demand for deep long-context models

Long-context efficiency is now a central LLM engineering topic.

Scaling Laws and Compute-Optimal Training

- Performance improves with data, model size, and compute, but with diminishing returns.
- Compute-optimal training balances parameter count and token budget.
- Chinchilla-style insight: many large models are under-trained relative to their size.

PRINCIPLE	IMPLICATION
Fixed compute budget	Tune model size and data volume jointly
Data-quality filtering	Small fraction of tokens may dominate useful learning
Chinchilla ratio (rule of thumb)	Roughly ~20 training tokens per parameter



Generative AI Practical Takeaways

ETRA

Choose architecture by task type: encoder-only, encoder-decoder, or decoder-only.

Prefer transfer learning and fine-tuning over training from scratch when possible.

Use attention-aware designs and decoding strategy (greedy/beam) based on product constraints.

Plan for memory/compute early: quantization, context limits, and deployment targets.

For domain-heavy use cases, adapt with specialized corpora and evaluation protocols.

Strong GenAI systems come from balanced choices across data, architecture, compute, and evaluation.

Section Takeaways



- 1. Encoder models understand; decoder models generate; seq2seq models both.
- 2. Attention lets models focus on relevant context per token.
- 3. Scaling laws trade compute for capability — plan responsibly.

Think: Which GenAI risks matter most for your organization's data?

WEEK 4 · SESSION 2 · GenAI

15

RAG

Retrieval-Augmented Generation

Vector stores

Chunking & retrieval

Augmented prompts

RAG evaluation

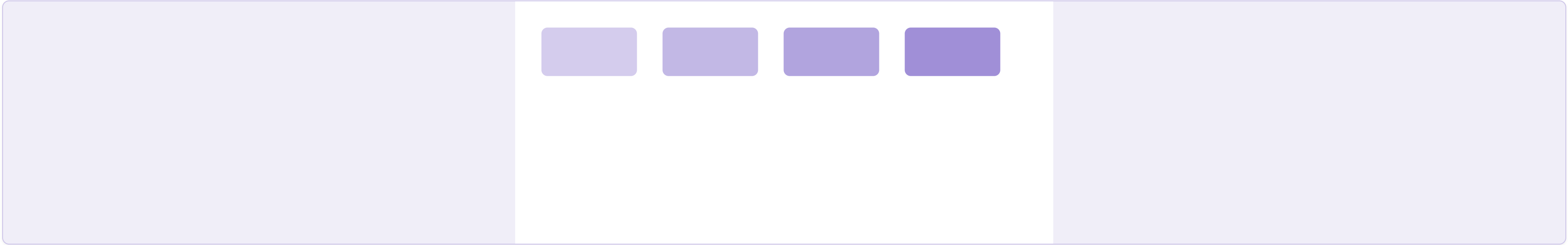
▸ 14 slides · ~22 min

Where Are We in the Bootcamp?



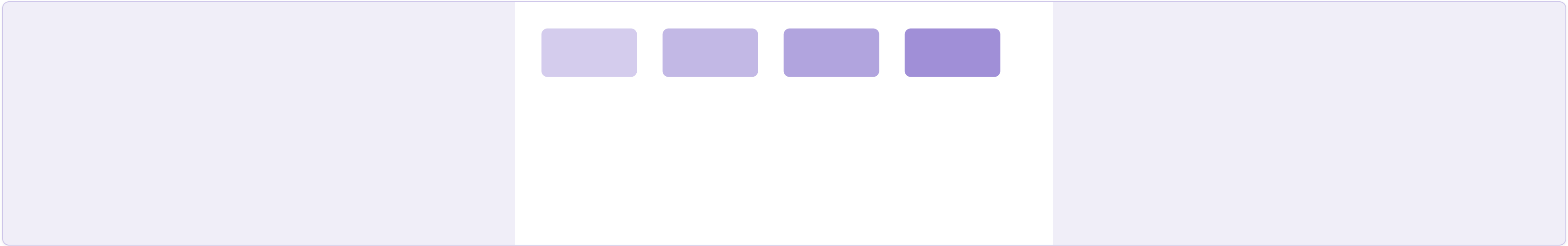
Ground LLM answers in your documents with retrieval-augmented generation.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15 RAG Systems**
- **S16** MLOps



- LLMs alone lack private, up-to-date, or domain-specific knowledge.
- RAG retrieves relevant documents at query time and injects them into the prompt.
- Reduces hallucination risk and lets you cite sources in enterprise settings.

Index → Retrieve → Augment → Generate



- Offline: chunk documents → embed → store in a vector index.
- Online: embed user query → retrieve top-k chunks → build augmented prompt.
- LLM generates answer conditioned on retrieved context.

Think of RAG as search + synthesis, not search alone.

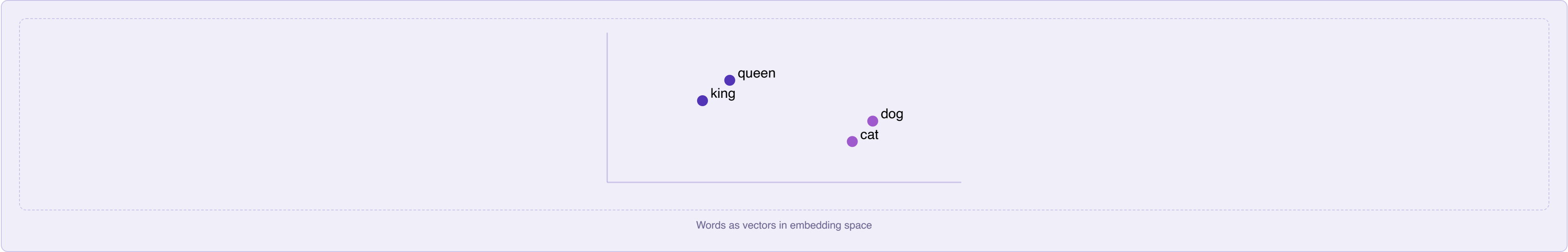
● Chunking Strategies

ETRA

Fixed-size windows (e.g., 512 tokens) — simple but may split semantics.

Semantic / paragraph chunks — better coherence, variable length.

Overlap between chunks improves recall for boundary facts.



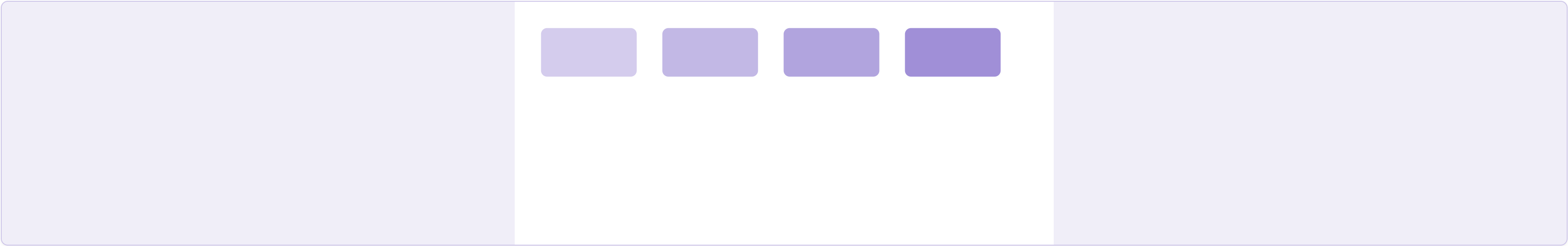
COMPONENT	ROLE	EXAMPLES
Embedding model	Maps text to dense vectors	OpenAI, Cohere, sentence-transformers
Vector DB	Similarity search at scale	FAISS, Pinecone, Chroma, pgvector
Metadata	Filter by tenant, date, ACL	JSON fields on each chunk

Recall@k: did the right chunk appear in top results?

Hybrid search (BM25 + vectors) often beats either alone.

Re-rankers can sharpen top results before the LLM sees them.

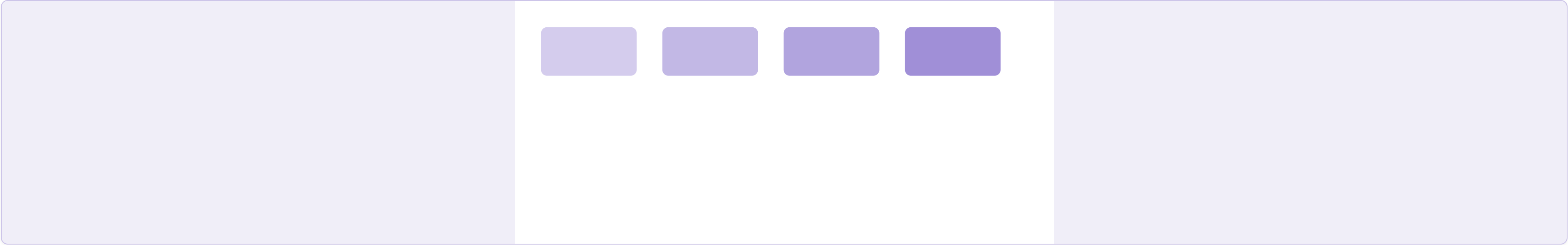
A robust template states rules, lists sources, then asks the question.



- | Instruct the model to answer only from provided context.
- | Include chunk boundaries and source IDs for traceability.
- | Handle empty retrieval with a safe fallback message.

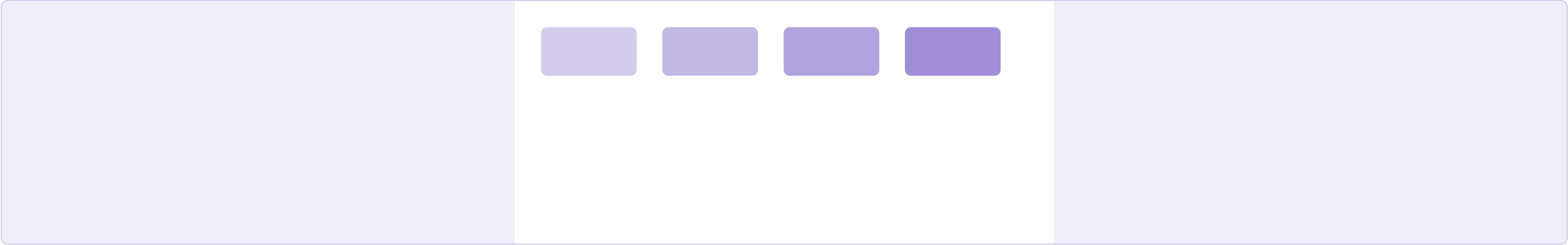
Faithfulness & Hallucination

- Even with context, models may invent facts — evaluate faithfulness.
- Ask the model to quote supporting spans when possible.
- Human review for high-stakes domains (legal, medical, finance).

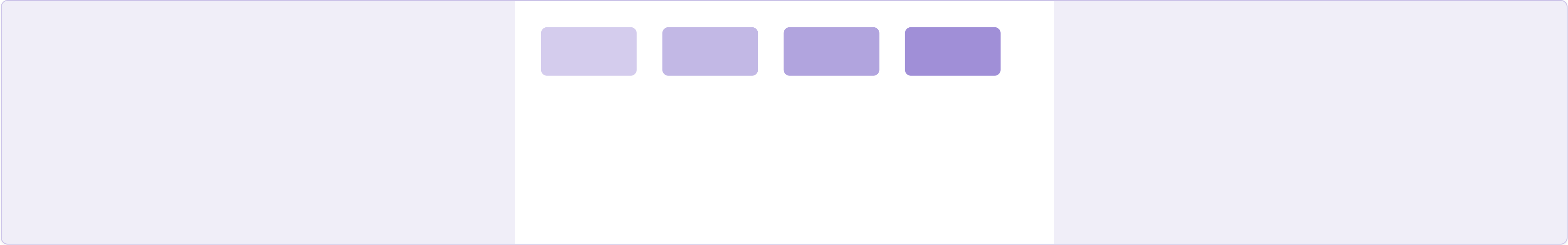


Metric	What it measures
Retrieval recall	Correct chunk in top-k
Answer relevance	Response addresses the user question
Faithfulness	Claims supported by retrieved text
Latency / cost	End-to-end SLA and token usage

- Chunks too large — dilute relevance; too small — lose context.
- Stale index — documents updated but embeddings not refreshed.
- ACL leaks — retrieving chunks the user should not see.



APPROACH	BEST WHEN	TRADEOFF
RAG	Knowledge changes often; need citations	Retrieval quality is critical
Fine-tuning	Style/format/domain phrasing	Expensive refresh cycle
Both	Regulated domain + brand voice	Higher ops complexity



- Embed the user query and every document chunk.
- Retrieve the top chunk via similarity search.
- Inject retrieved text into the LLM prompt before generation.



Production Checklist

- | Versioned ingestion pipeline with chunk + embed + index jobs.
- | Observability: log queries, retrieved IDs, latency, user feedback.
- | Governance: PII scrubbing, retention policy, and access-controlled indexes.

Section Takeaways

1. RAG = retrieve relevant chunks → augment prompt → generate answer.
2. Chunking strategy and embeddings quality dominate retrieval recall.
3. Evaluate faithfulness — models can still hallucinate with context.

Think: What document boundary would cause harmful retrieval for your domain?

WEEK 4 · SESSION 3 · GenAI

16

MLOps

Production ML Systems

CI/CD for ML

Model registry

Monitoring & drift

Production serving

▸ 15 slides · ~24 min

Where Are We in the Bootcamp?



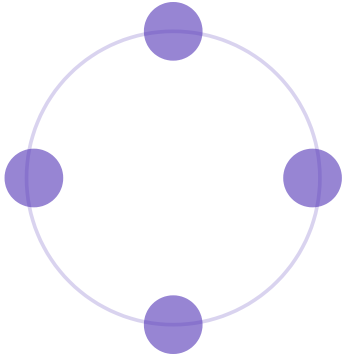
Ship models reliably: versioning, monitoring, and automated pipelines.

- **S1** Foundations & Preprocessing
- **S2** Regression Models
- **S3** Classification Basics
- **S4** Naive Bayes & Trees
- **S5** SVM & Kernels
- **S6** Clustering & PCA
- **S7** Deep Learning
- **S8** NLP Fundamentals
- **S9** Tokenization
- **S10** Text Analysis & NER
- **S11** Language Modeling
- **S12** Embeddings & RNNs
- **S13** Seq2Seq & NMT
- **S14** Generative AI
- **S15** RAG Systems
- **S16 MLOps**

What is MLOps?

From Notebook to Owned Service

ETRA



- | MLOps applies DevOps discipline to data, models, and experiments.
- | Goal: reproducible training, safe deployment, and continuous monitoring.
- | Bridges data science, engineering, and operations teams.

- | Data pipelines — ingestion, validation, feature stores.
- | Training pipelines — experiment tracking, model registry.
- | Serving — APIs, batch scoring, edge deployment.
- | Monitoring — data drift, concept drift, performance decay.

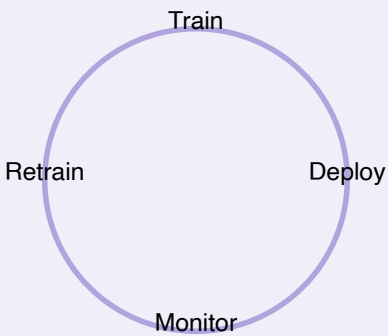
ARTIFACT	WHY VERSION IT
Code + config	Recreate training exactly
Training data snapshot	Audit and debug drift
Model weights	Rollback and compare
Environment	Same dependencies in train and serve

- | Log parameters, metrics, artifacts, and git SHA per run.
- | Compare runs side-by-side before promoting a model.
- | Tools: MLflow, Weights & Biases, Neptune, cloud-native trackers.

Model Registry & Promotion

ETRA

- Stages: staging → production → archived.
- Promotion gates: metric thresholds, bias checks, sign-off.
- Never deploy straight from a laptop without registry metadata.



MLOps continuous delivery loop

- CI: unit tests + data/schema validation + training smoke tests.
- CD: deploy container or endpoint when registry stage changes.
- Feature pipelines may need separate release cadence from models.

TYPE	DEFINITION	EXAMPLE SIGNAL
Data drift	Input distribution changes	New user demographics
Concept drift	$P(y x)$ changes	Fraud patterns evolve
Label drift	Labeling policy changes	Stricter QA on positives

Monitoring in Production

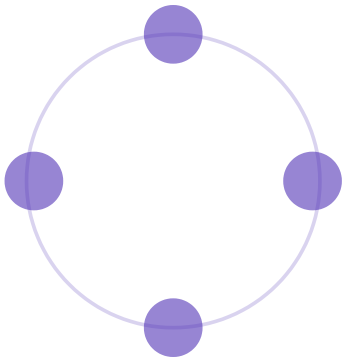
- Track latency, error rate, throughput at the serving layer.
- Track prediction distribution and feature statistics over time.
- Alert on SLA breaches and statistical tests vs training baseline.

Centralize features for training and serving to prevent train-serve skew.

- | Same feature definitions online and offline.
- | Point-in-time correctness for historical training sets.
- | Reuse across teams — reduces duplicated ETL.

- Batch scoring for reports and nightly jobs.
- Real-time REST/gRPC for interactive products.
- Shadow / canary deploys to compare new model safely.

- Log hyperparameters and evaluation metrics per training run.
- Tag runs with git SHA and data version for auditability.
- Push artifacts to MLflow, W&B, or your cloud tracker in production.



- Pick one model from the bootcamp and draw its production path.
- List 3 metrics you would monitor week one after launch.
- Identify the single biggest reproducibility gap in your current workflow.

Section Takeaways

1. MLOps connects code, data, and models with reproducible pipelines.
2. Monitor drift — data and concept drift degrade silent production models.
3. CI/CD for ML includes data validation, not just unit tests.

Think: Which production incident would your current process miss?

Conclusion

Key Takeaways

- Good ML pipelines start with strong preprocessing and proper validation.
- Model choice depends on data structure, interpretability needs, and scale.
- For classification, evaluate with confusion matrix, precision, recall, and F1.
- For unsupervised tasks, combine clustering diagnostics with PCA-based interpretation.

For deep learning, balance architecture power with regularization and validation discipline.

For NLP, robust preprocessing and tokenization choices strongly shape model outcomes.

Tokenizer design directly impacts OOV handling, sequence length, and model quality in NLP.

For language modeling, probability estimation quality and perplexity tracking are critical.

For sequence tasks, contextual representations and attention mechanisms are key drivers of quality.

For MT/NLG, combine BLEU and ROUGE with qualitative semantic validation.

For modern GenAI, balance architecture choice, scaling strategy, and compute efficiency.

RAG grounds LLMs in your documents; MLOps ships and monitors models reliably.

Use cross-validation and hyperparameter tuning for robust generalization.

Thank you. Questions and discussion are welcome.